

NHDF-CCD

Cradle-to-Grave

Support-Mapped Rigid-Convex Continuous Collision Detection

Version 0.6.0 - Reference, Validation, Integration, Operations, and Retirement

Release thesis. A reusable convex support interface, auditable GJK separation bounds, and a conservative linear-plus-angular motion envelope can extend the CCD core beyond v0.5 primitive profiles while preserving explicit uncertainty, finite resources, deterministic replay, and non-authoritative failure states.

Tom Klootwijk

Supplied author attribution; civil metadata is not independently verified and is not a cryptographic credential.

1 September 2026

Document status: executable research reference. This report does not claim peer review, exact-real completeness, production certification, GPU device validation, patent status, medical efficacy, or universal physical law.

Abstract

The NHDF source sequence began as an ordered, self-referential operator vocabulary spanning a log-polar lookup table, local implicit boundaries, parity, branching, kinematic propagation, analytic sweep, projection, and feedback. The formal v0.1 specification made the decisive non-degenerate correction: “SDF zero on all definable states” must be a family of local zero-level constraints rather than one identically zero global field [1, 2]. The commute transcript then connected the architecture to literal engine Continuous Collision Detection (CCD), but initially overclaimed that one SDF lookup and a bit flip solved general CCD [3]. Versions 0.4 and 0.5 replaced that claim with bounded broad and narrow phases, analytic time-of-impact profiles, certificates, explicit unsupported states, and reproducible tests.

Version 0.6 continues that roadmap. It introduces a common convex support-mapping layer for spheres, capsules, boxes, triangles, and finite convex hulls; an audit-oriented Gilbert–Johnson–Keerthi (GJK) distance query that reports lower and upper separation bounds; conservative advancement under quadratic translation and constant angular velocity; exact linear sphere–sphere and sphere–capsule profiles; and a deterministic swept-AABB bounding-volume hierarchy (BVH). The implementation is supplied in Python and independently written C++17. A complete exact sphere-pair CUDA kernel, portable data layout, and host harness are included, while device-level equivalence remains explicitly open.

The generated validation corpus contains 4350 authoritative narrow-phase cases, 100 diagnostic convex-QP cross-checks, and broad-phase scenes up to 2000 bodies. In this corpus, there were no observed definitive false negatives or false positives in the analytic CCD checks, no GJK bound violations over tolerance, no unresolved analytic cases, and no checked BVH pair-set disagreements. Endpoint-only checking missed 1034 collisions recovered by CCD. These are implementation observations on seeded binary64 data, not a proof over every real or floating-point input.

The wider optical, quantum, chemical, biological, medical, fabrication, SAR, AI, public-safety, and cosmological material is retained under a separate A0–A5 adapter qualification protocol [4]. CCD validation does not transfer to another domain merely because the same notation is reused.

Release decision

Decision: publish as v0.6.0 reference candidate

The Python, C++17, schema, host-GPU-layout, analytic-oracle, deterministic replay, and bounded-resource gates pass in the recorded build. The release is suitable for continued research, controlled engine integration, and independent replication. It is not a production collision library and does not yet satisfy the C2 GPU-equivalence gate.

Most important semantic change

For the general rigid-convex profile, **HIT** means first entry into the declared contact band, not an exact real-arithmetic zero-distance root. The certificate reports the last separated sample and the first accepted/refined contact-band sample. Specialized exact profiles may report an analytic geometric time of impact.

Hard evidence boundary

The release validates a bounded software CCD profile. It does not validate therapeutic claims, biological Cross-Kerr gates, universal physical substrate claims, zero-power photonic computing, identity-derived cryptography, or autonomous police authority. Those concepts remain independent adapter hypotheses requiring their own governing laws, experiments, uncertainty models, and safety reviews.

Contents

Abstract	i
Release decision	ii
I Provenance, Claims, and Lifecycle	1
1 Roadmap continuation and release identity	2
1.1 Why v0.6 centers on support-mapped rigid convex CCD	2
1.2 Layered identity	2
1.3 Release identifiers	2
2 Source lineage and epistemic separation	3
2.1 Source set	3
2.2 What is source-derived	3
2.3 What is release engineering	3
2.4 Claim ledger	3
3 Cradle-to-grave lifecycle	4
II Literal CCD Formalization	5
4 Problem definition	6
4.1 Pairwise continuous collision query	6
4.2 Why endpoint tests fail	6
4.3 Supported profile	6
5 Typed operator chain for CCD	7
5.1 State threading	7
5.2 Relation to parity and zero sets	7
6 Convex support mappings	8
6.1 Definition	8
6.2 Shape contracts	8
7 GJK distance with auditable bounds	9
7.1 Simplex upper bound	9
7.2 Support-plane lower bound	9

7.3	Finite-precision terminal states	9
8	Motion model and conservative point speed	10
8.1	Translation	10
8.2	Rotation	10
8.3	Lipschitz separation	10
9	Conservative advancement	11
9.1	Safe step	11
9.2	Remaining-interval miss	11
9.3	No unsafe minimum step	11
9.4	Contact-band refinement	11
9.5	Default binary64 numerical profile	11
10	Exact specialized profiles	13
10.1	Linear sphere-sphere	13
10.2	Linear sphere-capsule	13
10.3	Profile dispatch rule	13
11	Deterministic swept-AABB BVH	14
11.1	Swept bounds	14
11.2	Build and query	14
11.3	Capacity semantics	14
III	Certificates and Implementations	15
12	Certificate design	16
12.1	Status authority matrix	16
13	Python reference	18
13.1	Package map	18
13.2	Determinism	18
14	C++17 reference	19
15	CUDA continuation	20
15.1	Delivered artifacts	20
15.2	What remains open	20
IV	Verification and Evidence	21
16	Test program	22
16.1	Python unit suite	22
16.2	Independent formulations	22
16.3	C++ and host-GPU-layout tests	22
17	Benchmark corpus and results	23

17.1 Environment and scope	23
17.2 Static GJK	23
17.3 Linear and rotating CCD	23
17.4 Broad phase	23
18 Failure injection and residual risk	27
18.1 Budget exhaustion	27
18.2 Numerical residual risk	27
18.3 Motion-bound residual risk	27
18.4 False confidence risk	27
V Integration, Operations, and Retirement	28
19 Physics-engine integration	29
19.1 Recommended dispatch	29
19.2 Detection is not response	29
19.3 Step normalization	29
20 Public-safety and high-stakes boundary	30
21 Operations and maintenance	31
21.1 Telemetry	31
21.2 Incident response	31
21.3 Change control	31
22 Retirement	32
VI Separate Domain Adapters	33
23 A0–A5 qualification protocol	34
24 Adapter matrix	35
VII Limitations and Next Releases	36
25 Known limitations	37
26 Roadmap to v1.0	38
26.1 v0.7: linear primitives and deforming meshes	38
26.2 v0.8: contact dynamics	38
26.3 v0.9: acceleration and qualification	38
26.4 v1.0: stable profile	38
27 Conclusion	39

VIII	Technical Appendices	40
A	Property sketches	41
A.1	Safe advancement	41
A.2	Bounded memory	41
A.3	Contact-band lead	41
B	Selected public types	42
C	CUDA shared profile source	44
D	Machine-readable benchmark summary	46
E	Release file map	48
F	References	51
	Historical Annex: complete v0.5 dossier	53

List of Figures

5.1	State-threaded CCD pipeline. No parity bit or coordinate metaphor substitutes for the geometric proof obligations.	7
17.1	Coverage of the principal v0.6 validation corpora.	24
17.2	Decisive error and unresolved counts. A zero bar is an observed corpus result, not a universal proof.	24
17.3	GJK upper-bound error by analytic primitive family.	25
17.4	Conservative lead of contact-band entry relative to exact geometric TOI.	25
17.5	Deterministic swept-AABB BVH scaling in the Python reference.	26

List of Tables

1.1	The three NHDF layers and the v0.6 CCD realization.	2
3.1	Lifecycle gates and evidence.	4
6.1	Support shapes in the reference profile.	8
9.1	Default v0.6 numerical controls.	12
12.1	Principal certificate fields.	16
12.2	How callers must interpret statuses.	17
17.1	Authoritative benchmark summary.	23
24.1	Separated adapter status.	35

Part I

Provenance, Claims, and Lifecycle

Chapter 1

Roadmap continuation and release identity

1.1 Why v0.6 centers on support-mapped rigid convex CCD

Version 0.5 added a mesh-facing translational sphere/point-versus-triangle profile. That was the right step for graphics front ends, but a general engine also needs reusable distance and motion interfaces for rigid convex bodies. A support mapping supplies that reuse: each convex shape answers an extremal-point query in an arbitrary direction, allowing one narrow-phase distance algorithm to operate across multiple primitives.

The v0.6 architecture does not discard the NHDF vocabulary. It narrows each term to an engineering role. Local implicit or support geometry supplies valid shape constraints; kinematics provides a causal trajectory; bounded routing produces candidate pairs; projection produces a certificate rather than a metaphorical screen image; and feedback is the telemetry, replay, and change-control loop that improves the next release.

1.2 Layered identity

Table 1.1: The three NHDF layers and the v0.6 CCD realization.

Layer	Original role	v0.6 CCD realization
NHDF	Representation	Versioned scene, shapes, support mappings, poses, motion, AABBs, certificates, traces.
PCKF	Dynamics	Bounded forward-time conservative advancement using separation and point-speed bounds.
IHCP	Runtime	Deterministic CPU references, bounded BVH, schemas, test/benchmark pipeline, GPU port contract.
Adapter	Application	Physics-engine integration, wider-domain mapping, telemetry and safety policy.

1.3 Release identifiers

The software version is `0.6.0`. Scene and certificate schemas are separately versioned as `nhdf-ccd-scene-0.6.0` and `nhdf-ccd-certificate-0.6.0`. Benchmark records use `nhdf-ccd-benchmark-summary-0.6.0`. Schema identifiers are not changed by formatting-only report edits; semantic changes require a new profile or schema version.

Chapter 2

Source lineage and epistemic separation

2.1 Source set

The release package preserves byte copies and SHA-256 hashes for the mounted source artifacts and records one content-only source whose raw bytes were unavailable in this build:

1. the original 30-page operator-chain transcript [1];
2. the 26-page formal v0.1 specification [2];
3. the 40-page commute and CCD transcript [3];
4. the 70-page spherical–Cartesian and whole-substrate extension [4], mapped from the supplied parsed source while its raw byte stream remained unavailable to this build;
5. the 100-page v0.5 predecessor release [5], preserved as a byte copy and immutable historical annex.

2.2 What is source-derived

The following ideas are directly traceable to the source sequence: ordered closure, log-polar indexing, distributed local zero sets, a monotonic timeline, explicit branch pools, parity as a low-cost event, analytic sweep/projection language, self-reference, and the intent to apply the architecture to CCD. The source also contains broader optical, biological, medical, fabrication, and cosmological proposals.

2.3 What is release engineering

The following are corrections or additions made during formal validation: support-mapping types, GJK lower/upper bounds, conservative point-speed envelopes, analytic TOI oracles, explicit contact-band semantics, BVH capacity failure, deterministic trace digests, versioned schemas, test independence notes, GPU qualification gates, and the A0–A5 adapter maturity scale.

2.4 Claim ledger

A claim is labeled *implemented*, *observed*, *formal engineering argument*, *not implemented*, or *unsupported*. This prevents an executable CCD result from being rhetorically promoted into universal physics. The machine-readable ledger is included in `00_provenance/CLAIM_LEDGER.csv`.

Chapter 3

Cradle-to-grave lifecycle

Table 3.1: Lifecycle gates and evidence.

Gate	Name	Required evidence
G0	Provenance	Immutable source copies, source register, hashes, attribution and evidence boundary.
G1	Problem	Scope, requirements, exclusions, hazards, acceptance criteria.
G2	Formalization	Typed operators, GJK contract, motion bound, contact-band semantics, state machine, schemas.
G3	CPU reference	Python and independent C++17 implementations with finite resources.
G4	Verification	Unit, adversarial, analytic-oracle, deterministic replay, language smoke tests.
G5	Benchmark	Seeded CSV/JSON results, figures, environment, no hidden failures.
G6	Acceleration	CUDA data layout and kernel source; device-equivalence gate remains explicit.
G7	Integration	Caller obligations, fallback behavior, public-safety boundary.
G8	Operations	Telemetry, runbook, incident handling, safe degradation, change control.
G9	Maintenance	Versioning, migration, regression and deprecation policy.
G10	Retirement	Archive, evidence retention, migration and removal criteria.

Part II

Literal CCD Formalization

Chapter 4

Problem definition

4.1 Pairwise continuous collision query

Given bodies $A(t)$ and $B(t)$ for normalized time $t \in [0, 1]$, the geometric time of impact is

$$t^* = \inf\{t \in [0, 1] : A(t) \cap B(t) \neq \emptyset\}. \quad (4.1)$$

A digital implementation must additionally specify arithmetic, tolerances, trajectory class, supported shapes, iteration limits, and what happens when it cannot decide.

4.2 Why endpoint tests fail

A discrete endpoint test asks only whether $A(0)$ intersects $B(0)$ or $A(1)$ intersects $B(1)$. A fast object can pass completely through a thin obstacle between those samples. The bundled linear-sphere corpus contains 1034 such exact hits missed by endpoint-only checking. This is the literal tunnelling problem addressed by CCD.

4.3 Supported profile

The v0.6 general profile supports compact convex shapes with deterministic support mappings, quadratic translation, and constant world angular velocity. Its authoritative result is a contact-band entry or a certified miss. It does not yet implement general deforming-mesh or triangle-triangle exact CCD.

Chapter 5

Typed operator chain for CCD

5.1 State threading

The v0.1 chain is conceptually useful but not literally type-safe if an address, bit, branch tree, trajectory, and output are composed as though they were the same object. v0.6 therefore threads one enriched state:

$$\Sigma = (\mathcal{S}, \mathcal{B}, \mathcal{C}, t, P_A, P_B, \Delta, d_{lo}, d_{hi}, w_A, w_B, \chi, R, \tau), \quad (5.1)$$

where $\mathcal{S}$ is the scene, $\mathcal{B}$ the swept bounds, $\mathcal{C}$ the pair queue, P_i poses, Δ the simplex/workspace, d_{lo}, d_{hi} distance bounds, w_i witnesses, χ status, R resources, and τ telemetry.

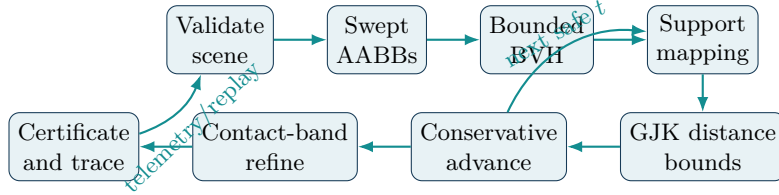


Figure 5.1: State-threaded CCD pipeline. No parity bit or coordinate metaphor substitutes for the geometric proof obligations.

5.2 Relation to parity and zero sets

A one-bit parity operator can detect an odd number of payload inversions, but it cannot determine collision, locate a corrupt bit, or correct an even-numbered error. Likewise, a global field $F \equiv 0$ has no distance gradient. In v0.6, payload integrity and geometry remain distinct. Local implicit constraints may define shapes, but the CCD profile uses actual support geometry and separation bounds.

Chapter 6

Convex support mappings

6.1 Definition

For compact convex $A \subset \mathbb{R}^3$,

$$s_A(d) = \arg \max_{a \in A} a \cdot d. \quad (6.1)$$

The Minkowski-difference support is

$$s_{A-B}(d) = s_A(d) - s_B(-d). \quad (6.2)$$

The origin lies in $A - B$ exactly when A and B intersect. This is the basis for GJK distance and intersection queries [6, 7].

6.2 Shape contracts

Table 6.1: Support shapes in the reference profile.

Shape	Support rule	Bounding radius
Sphere	center plus radius times normalized direction	$\ c\ + r$
Capsule	extremal endpoint plus radial sphere support	$\max(\ a\ , \ b\) + r$
Box	choose signed half-extent on each axis	$\ c\ + \ h\ $
Triangle	maximum dot product among three vertices	$\max_i \ v_i\ $
Convex hull	maximum dot product among finite vertices	$\max_i \ v_i\ $

The radius is measured around the declared motion origin, not automatically around the shape centroid. This matters for offset spheres and rotating bodies.

Chapter 7

GJK distance with auditable bounds

7.1 Simplex upper bound

At each iteration, the algorithm finds the point q in the convex hull of the current Minkowski simplex nearest the origin. Then

$$d(A, B) \leq \|q\| = d_{hi}. \quad (7.1)$$

The barycentric coefficients also combine source support points into closest witnesses w_A, w_B .

7.2 Support-plane lower bound

With unit direction $n = q/\|q\|$ and new support point $p = s_{A-B}(-q)$, a separating support plane supplies a lower estimate. The reference monotonically retains the best nonnegative lower bound and terminates when

$$d_{hi} - d_{lo} \leq \tau_{abs} + \tau_{rel} d_{hi}. \quad (7.2)$$

A safe time step uses the lower bound, not merely the closest-point upper estimate.

7.3 Finite-precision terminal states

Duplicate supports, rank-deficient simplex solves, upper-bound stalls, and iteration limits are labeled. A duplicate support is called converged only when its remaining gap fits a slightly relaxed machine envelope. Otherwise the result remains unresolved and conservative advancement may return a numerical failure or use the declared distance-error budget.

Why a scalar distance is not enough

A scalar without an error interval hides whether it is an overestimate or underestimate. Conservative time stepping requires a lower separation guarantee. v0.6 therefore exposes both bounds, the gap, convergence, support count, simplex size, witnesses, features, and termination reason.

Chapter 8

Motion model and conservative point speed

8.1 Translation

For body i ,

$$x_i(t) = x_{i0} + v_{i0}t + \frac{1}{2}a_it^2, \quad v_i(t) = v_{i0} + a_it. \quad (8.1)$$

On $[t, 1]$, the reference bounds the relative origin speed directly:

$$V_{lin} \leq \|v_A(t) - v_B(t)\| + \|a_A - a_B\|(1 - t). \quad (8.2)$$

This is tighter than summing two independent translation bounds while remaining conservative.

8.2 Rotation

The rotation is

$$R_i(t) = \text{Exp}([\omega_i]_{\times} t) R_{i0}. \quad (8.3)$$

If every shape point lies within r_i of the motion origin, rotational point speed is bounded by $\|\omega_i\|r_i$. The pair bound is

$$V = V_{lin} + \|\omega_A\|r_A + \|\omega_B\|r_B. \quad (8.4)$$

This treats rotation literally. It does not pretend that angular motion is already included in a log-polar bit shift.

8.3 Lipschitz separation

For any $h \geq 0$ within the modeled interval,

$$d(t + h) \geq d(t) - Vh. \quad (8.5)$$

This inequality is the safety basis for advancement and remaining-interval miss certification.

Chapter 9

Conservative advancement

9.1 Safe step

Let d_{lo} be the usable GJK lower bound, τ_c contact tolerance, ϵ_d distance error budget, and $0 < \eta \leq 1$ safety factor. The step is

$$h = \eta \frac{d_{lo} - \tau_c - \epsilon_d}{V} \quad (9.1)$$

when the numerator is positive. If it is nonpositive, the algorithm enters contact-band refinement rather than advancing through an uncertified interval.

9.2 Remaining-interval miss

A miss is certified before $t = 1$ when

$$d_{lo} - \epsilon_d - V(1 - t) - \tau_c > 0. \quad (9.2)$$

This proves the entire remaining interval stays outside the band under the declared motion bound.

9.3 No unsafe minimum step

It is tempting to force a minimum h for performance. That can leap over contact and invalidate CCD. v0.6 instead returns **INCONCLUSIVE** when safe progress is below the declared minimum and the current distance is not inside the numerical contact band.

9.4 Contact-band refinement

A separated lower sample and accepted upper sample are bisected until the time tolerance, refinement budget, or a numerical alarm. The retained interval has operational meaning even when the exact geometric root is later.

9.5 Default binary64 numerical profile

The reference defaults are a declared engineering profile rather than universal constants. Scene coordinates should be normalized to a documented working scale, and integrations must serialize any override with the certificate. The nominal default contact-band width is $\tau_c + \epsilon_d = 1.01 \times 10^{-6}$ scene units. A support-plane lower bound entering this band does not alone prove contact: the simplex upper bound must also enter the band (within the recorded closure slack), otherwise the query returns **INCONCLUSIVE**. This distinction prevents an uncertain lower bound from being promoted to a false hit.

Table 9.1: Default v0.6 numerical controls.

Control	Default	Meaning
Contact tolerance τ_c	$1.000\,000\,00 \times 10^{-6}$	Positive separation band accepted by the general profile.
Distance error budget ϵ_d	$1.000\,000\,00 \times 10^{-8}$	Maximum bound-gap budget accepted for a usable GJK sample.
GJK absolute/relative tolerance	$1.000\,000\,00 \times 10^{-11}$ /	Dual-gap and duplicate-support termination controls.
Time tolerance	$1.000\,000\,00 \times 10^{-10}$	Normalized-time refinement/stall threshold.
Closure slack	at most $6.400\,000\,00 \times 10^{-10}$	Finite-precision closure only: 64 times the GJK absolute tolerance, also guarded by machine-scale logic.

Listing 9.1: Semantics-first conservative advancement pseudocode.

```

1  t = 0
2  sample = gjk_distance(A(t), B(t))
3  if sample is unusable: NUMERICAL_FAILURE
4  if sample.upper <= contact_band: INITIAL_OVERLAP
5  while budget remains:
6      V = relative_point_speed_bound(A, B, [t, 1])
7      if sample.lower - eps_d - V*(1-t) - tau_c > 0: MISS
8      gap = sample.lower - tau_c - eps_d
9      if gap <= 0:
10         if sample.upper <= contact_band + closure_slack: HIT
11         else: INCONCLUSIVE
12         h = safety_factor * gap / V
13         if h < minimum_safe_step: INCONCLUSIVE
14         last_safe, t = t, min(1, t+h)
15         sample = gjk_distance(A(t), B(t))
16         if sample is unusable: NUMERICAL_FAILURE
17         if sample.upper <= contact_band: refine(last_safe, t); HIT
18  INCONCLUSIVE

```

Chapter 10

Exact specialized profiles

10.1 Linear sphere–sphere

With relative position p , velocity v , and combined radius r ,

$$\|p + tv\|^2 = r^2 \tag{10.1}$$

becomes a quadratic. The reference uses a cancellation-resistant root formulation and returns the earliest root in $[0, 1]$, initial overlap, or miss.

10.2 Linear sphere–capsule

A translating, nonrotating capsule is a segment swept by a sphere. The exact profile considers:

1. contact with each spherical end cap;
2. contact with the finite cylindrical interior, accepting roots whose axial coordinate lies on the segment;
3. initial segment distance.

The earliest candidate determines the witness and feature label. This exact profile is both a fast path and a validation oracle for the general support-mapped path.

10.3 Profile dispatch rule

A specialized solver may be used only when its motion and shape preconditions are met. A rotating capsule cannot silently enter the nonrotating sphere–capsule routine. Failed preconditions route to the general profile or **UNSUPPORTED** in a future stricter dispatch layer.

Chapter 11

Deterministic swept-AABB BVH

11.1 Swept bounds

For each translation axis, the quadratic coordinate is evaluated at endpoints and any derivative root inside $[0, 1]$. A sphere of radius equal to the shape's motion-origin bounding radius is then added. This encloses all orientations and translations but may be loose.

11.2 Build and query

The reference recursively splits object centroids on the axis of largest spread, resolves ties by body identifier, uses fixed leaf size, and traverses children in deterministic order. Candidate pairs are sorted canonically.

11.3 Capacity semantics

If output capacity is reached, the result is **RESOURCE_EXHAUSTED**; the partial pair list is explicitly non-authoritative. A caller must not process that prefix as though it were complete.

Part III

Certificates and Implementations

Chapter 12

Certificate design

Table 12.1: Principal certificate fields.

Field	Meaning
schema_version, profile	Exact schema and algorithm profile.
time_semantics	Human-readable authority of the time interval.
contact_tolerance, distance_error_budget	Numerical band defining general-profile contact.
status	Authoritative state; unresolved/resource states never mean miss.
toi_lower, toi_upper	Last separated and first accepted/refined time, when applicable.
distance_at_upper	Terminal GJK upper separation estimate.
witness_a, witness_b, normal_a_to_b	Terminal closest points and normal, potentially nonunique near degeneracy.
feature_a, feature_b	Deterministic support-feature labels.
conservative_speed_bound	Pairwise point-speed envelope used by the run.
ca_iterations, gjk_iterations_total	Work and budget observability.
termination	Exact branch that ended the query.
trace_sha256, trace	Canonical reference execution history and digest.

12.1 Status authority matrix

Table 12.2: How callers must interpret statuses.

Status	Collision decision	Required caller action
HIT	yes, within declared band/profile	schedule contact; retain profile semantics
INITIAL_OVERLAP	yes at start	use overlap/recovery path; witness may not be a manifold
MISS	no for declared interval/model	pair may be discarded for this step
INCONCLUSIVE	unknown	shorten step or use stronger backend; safe fallback
NUMERICAL_FAILURE	unknown	quarantine/retry; preserve diagnostics
INVALID_INPUT	undefined	reject scene/input
RESOURCE_EXHAUSTED	unknown	discard partial authority; resize or degrade

Chapter 13

Python reference

The Python package is intentionally readable. NumPy is the only runtime dependency. Shapes are immutable dataclasses; motion validation occurs at construction; GJK exposes a rich result; conservative advancement builds a certificate; the BVH uses deterministic ordering; and JSON I/O enforces the scene version.

13.1 Package map

math3d.py vector, quaternion, and pose conventions.
shapes.py support mappings and bounding radii.
gjk.py active-simplex solve, bounds, witnesses, trace.
motion.py quadratic translation, angular pose, swept envelopes.
ccd.py conservative advancement and certificates.
exact.py specialized analytic profiles.
bvh.py deterministic swept broad phase.
io.py, cli.py versioned scene loader and command-line entry.

13.2 Determinism

Floating-point determinism is scoped to the reference environment, algorithm order, and input. Canonical sort keys and a rounded trace encoding reduce incidental ordering changes. The SHA-256 trace is a regression digest, not an authenticity signature.

Chapter 14

C++17 reference

The C++17 implementation was written independently against the same contract. It includes vector/quaternion math, sphere/capsule/box/hull/triangle support mappings, active-set simplex distance, rigid motion, pair speed, and conservative advancement. Its public header exposes finite records and does not depend on Python.

The smoke executable checks known sphere distance, translational sphere CCD, sphere miss, and rotating capsule–box contact. C++ output fields are a subset of the richer Python certificate; full cross-language schema parity remains future work.

Chapter 15

CUDA continuation

15.1 Delivered artifacts

The package contains a portable POD layout, an exact sphere–sphere function marked for host/device use, a CUDA pair kernel, a fixed-capacity rigid-convex workspace contract, and a C++ host harness that executes the shared arithmetic.

15.2 What remains open

No `nvcc` compiler or CUDA device was available. The kernel is not device-compiled, occupancy is not measured, and CPU/GPU differential replay is not performed. The release therefore remains C1 for the full rigid-convex profile. The supplied GPU plan requires compile, deterministic-vector, randomized differential, overflow, and performance gates before C2.

No special-unit mythology

Tensor cores, ray-tracing cores, VRAM, or optical hardware do not automatically implement parity, topology, or CCD. Acceleration is valid only when an exposed operation has equivalent semantics and passes a differential test.

Part IV

Verification and Evidence

Chapter 16

Test program

16.1 Python unit suite

The release contains 73 pytest methods covering transforms, support shapes, GJK, exact profiles, translational and rotating CCD, quadratic acceleration, failure budgets, deterministic trace, BVH pair completeness, capacity failure, and JSON loading.

16.2 Independent formulations

The verification program avoids one single common code path:

- analytic primitive distances check GJK across sphere–sphere, sphere–capsule, capsule–capsule, and sphere–box families;
- an SLSQP barycentric convex program provides a diagnostic polytope cross-check;
- a quadratic solver checks linear sphere TOI;
- a trigonometric circle-intersection oracle checks an offset sphere rotating about the z axis;
- brute-force AABB pair enumeration checks the BVH at selected scales.

16.3 C++ and host-GPU-layout tests

The C++17 smoke executable and the CUDA shared host profile are compiled with warnings enabled. The latter checks hit, miss, initial overlap, and invalid input for the exact sphere profile, plus structure sizes.

Chapter 17

Benchmark corpus and results

17.1 Environment and scope

The recorded run used Python 3.13.5, NumPy 2.3.5, and SciPy 1.17.0 on the packaged environment. Timings are single-run observations and are not vendor-independent performance claims.

Table 17.1: Authoritative benchmark summary.

Corpus	Queries	False neg.	False pos./violations	Unresolved
Static GJK vs analytic primitives	2400	–	0 bound violations	0
Linear sphere CCD vs analytic TOI	1400	0	0	0
Rotating offset sphere vs analytic TOI	550	0	0	0
Swept BVH vs brute force	scales to 2000	–	0 pair disagreements	–

17.2 Static GJK

Across 2400 analytic primitive-distance cases, the maximum GJK upper-bound absolute error was $6.567\,280\,05 \times 10^{-11}$, and the maximum retained lower/upper gap was $7.068\,834\,41 \times 10^{-11}$. There were no tolerance-level lower- or upper-bound violations and no overlap-classification disagreements.

17.3 Linear and rotating CCD

The linear sphere corpus recovered 1034 exact collisions missed by endpoint-only tests. Its maximum observed conservative lead before exact geometric TOI was $1.091\,106\,32 \times 10^{-6}$; there was no late result. The rotating offset-sphere corpus had maximum lead $8.533\,020\,24 \times 10^{-7}$ and no late result. The lead is expected because the general profile detects a positive contact band.

17.4 Broad phase

The BVH was exercised at 100, 300, 1000, and 2000 bodies. At the largest scale it returned 50 candidate pairs in 0.512577846s in this Python reference. Pair sets matched brute force at checked scales.

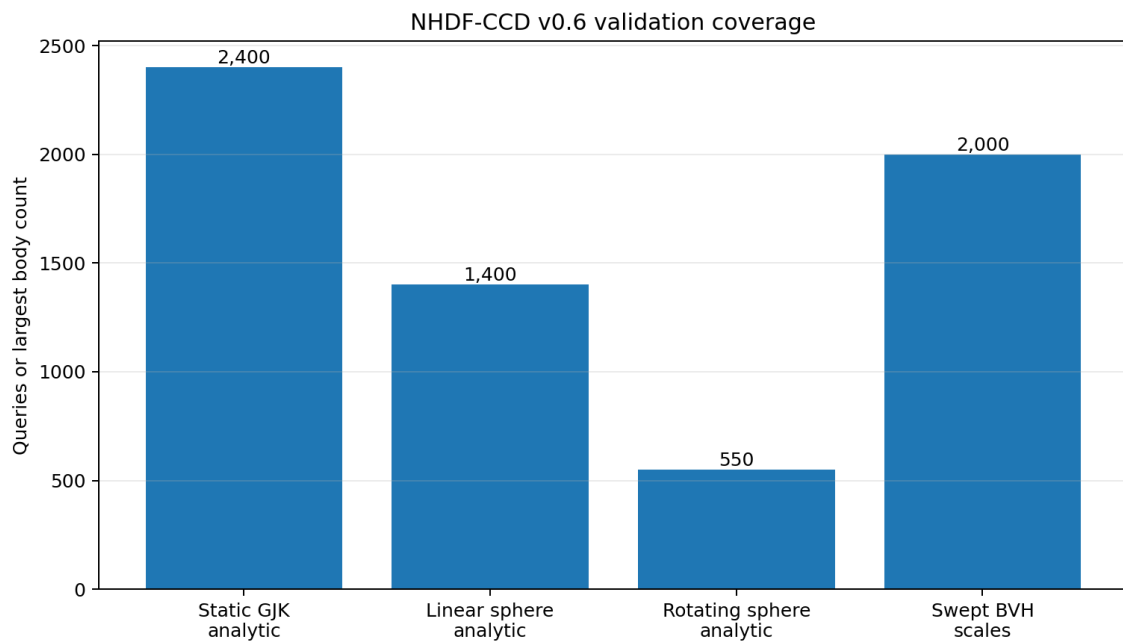


Figure 17.1: Coverage of the principal v0.6 validation corpora.

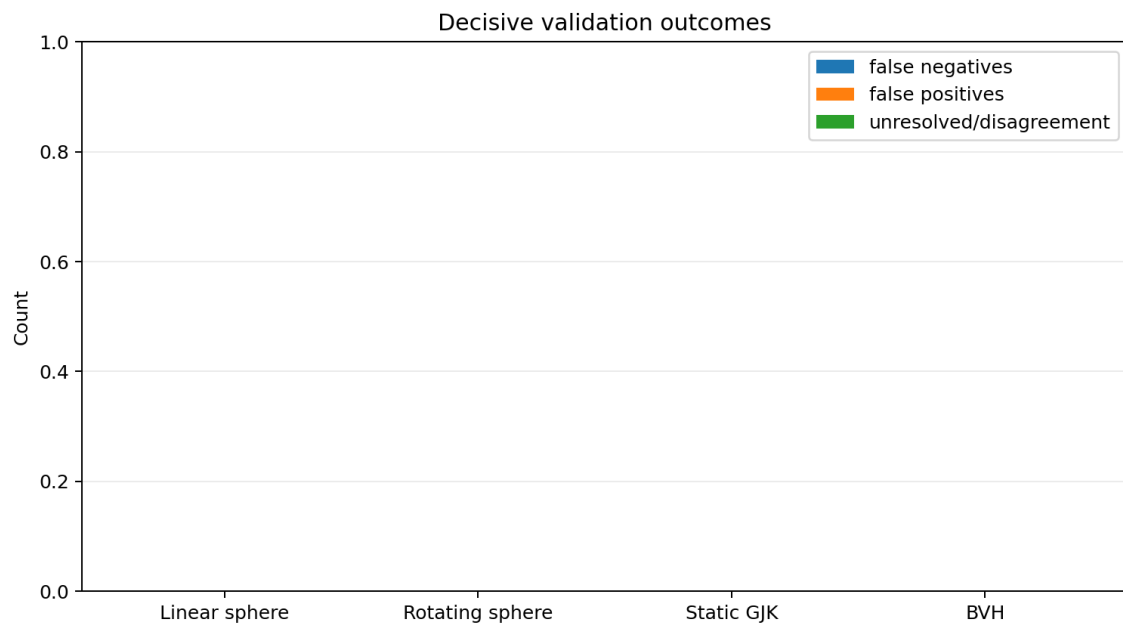


Figure 17.2: Decisive error and unresolved counts. A zero bar is an observed corpus result, not a universal proof.

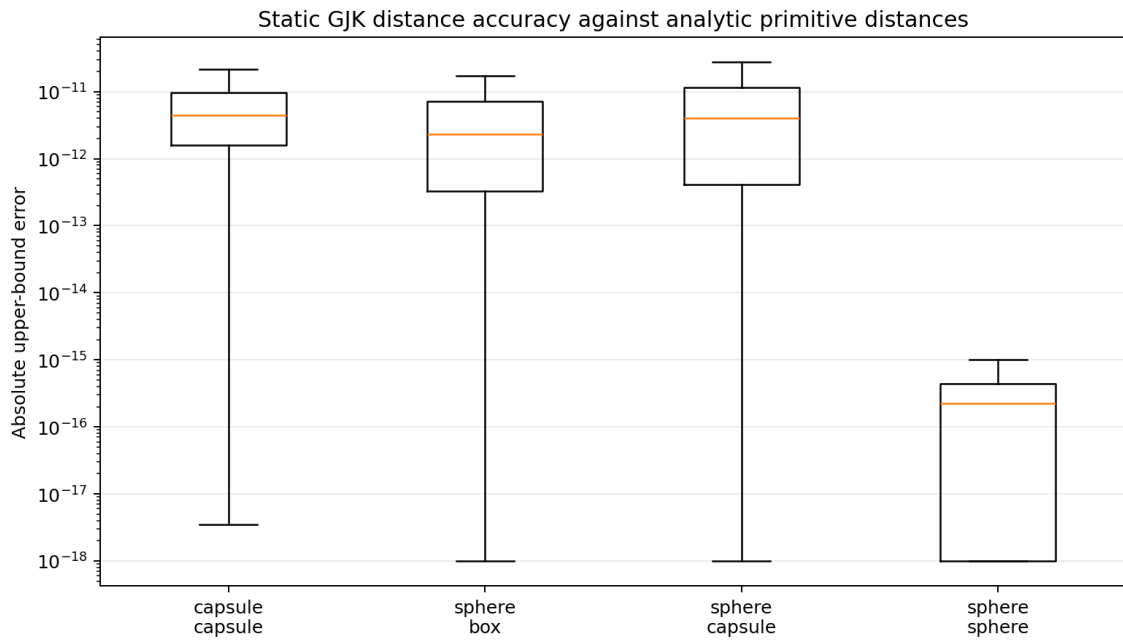


Figure 17.3: GJK upper-bound error by analytic primitive family.

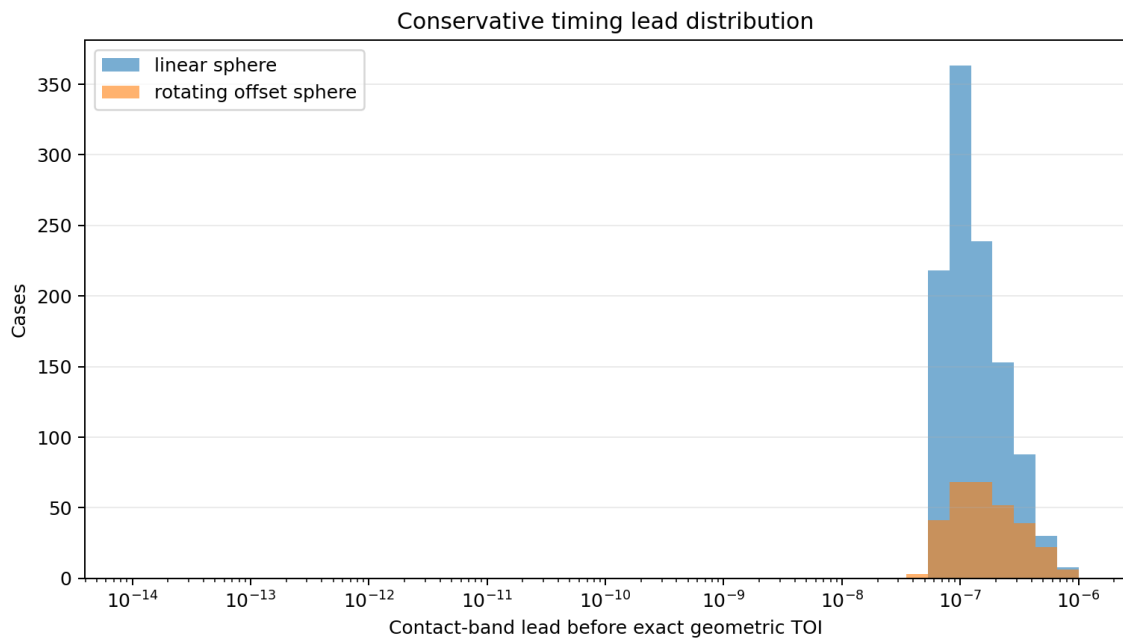


Figure 17.4: Conservative lead of contact-band entry relative to exact geometric TOI.

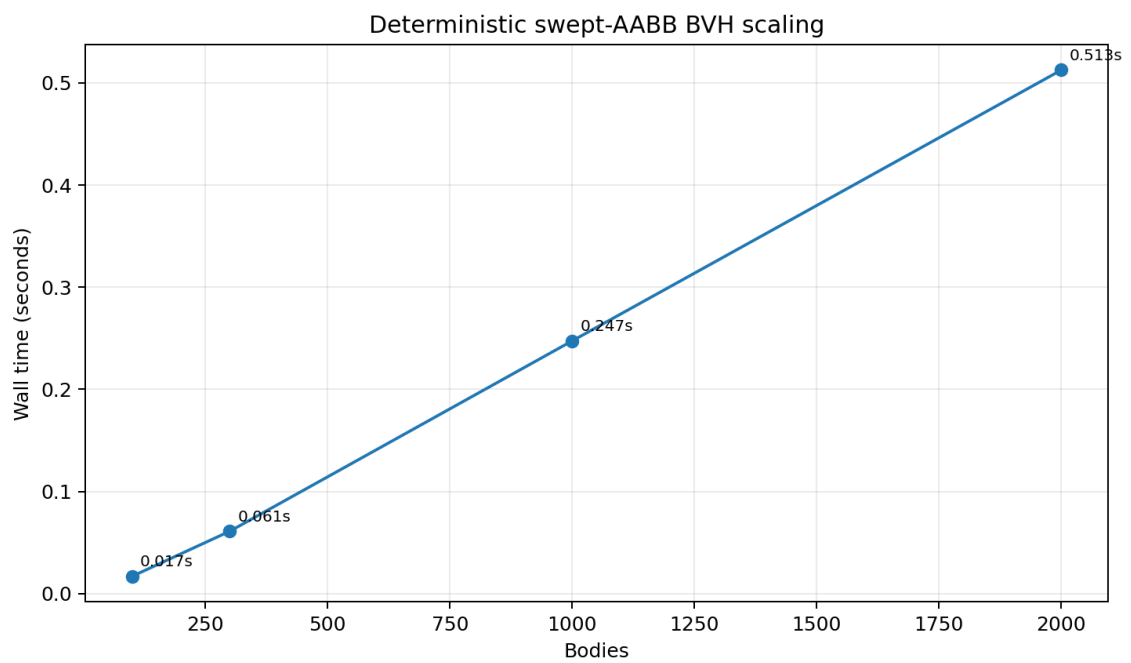


Figure 17.5: Deterministic swept-AABB BVH scaling in the Python reference.

Chapter 18

Failure injection and residual risk

18.1 Budget exhaustion

A one-iteration CA configuration is required to return **INCONCLUSIVE** on a known collision rather than **MISS**. Candidate capacity injection returns **RESOURCE_EXHAUSTED** and declares its prefix non-authoritative.

18.2 Numerical residual risk

GJK can be difficult near tangency, repeated support points, and lower-dimensional degeneracy. The reference exposes iteration-limit and stall states, but the current CA may accept a nonconverged GJK sample when its bound gap fits the declared distance-error budget. This is an explicit engineering trade, not exact arithmetic.

18.3 Motion-bound residual risk

The angular term requires a valid radius around the actual motion origin. Incorrect shape offsets can underestimate speed. The constructors and tests address common cases, but integration code owns its transform convention.

18.4 False confidence risk

Zero observed errors in a generated corpus can be mistaken for proof. The release repeatedly records the distribution, seed, tolerances, environment, and scope note to prevent that inference. Large public TOI datasets and exact inclusion methods remain important comparison targets [9, 10].

Part V

Integration, Operations, and Retirement

Chapter 19

Physics-engine integration

19.1 Recommended dispatch

A production engine should freeze motion for a generation, create conservative swept bounds, enumerate a bounded candidate list, dispatch exact specialized profiles where preconditions hold, use the general rigid-convex profile otherwise, group near-simultaneous contacts, and send authoritative results to an independently validated dynamics solver.

19.2 Detection is not response

The commute transcript proposed velocity sign flips and bit crushing as collision response. Literal rigid-body response requires contact manifolds, masses, inertia, restitution, friction, constraints, and solver stability. v0.6 intentionally stops at detection certificates. Response is planned for v0.8.

19.3 Step normalization

All motion is defined on $t \in [0, 1]$. An engine with physical step Δt must scale velocity and acceleration consistently or formulate poses directly as functions of normalized time. Mixing seconds and normalized units invalidates the speed proof.

Chapter 20

Public-safety and high-stakes boundary

CCD can contribute to obstacle prediction, simulation, or driver assistance. It cannot establish legal authority, identify lawful targets, determine intent, or justify force. High-stakes deployment requires independent sensing, privacy and civil-rights review, fail-safe actuation, human override, incident retention, and applicable standards. Personal identity or journey metadata is not a cryptographic key.

Chapter 21

Operations and maintenance

21.1 Telemetry

A conforming runtime records generation, pair counts, status histogram, maximum iteration counts, candidate capacity/utilization, wall time, build hash, and warnings. Deployment thresholds should be defined for unresolved rate, overflow, and latency.

21.2 Incident response

A suspected false miss is severity one: stop, preserve scene, binary, configuration, logs and hashes, and require independent review. Resource or determinism failures enter safe degradation. No incident is closed solely by increasing tolerance.

21.3 Change control

Support mapping, quaternion order, bounding radius, point-speed bound, tolerance, termination, queue capacity, or status semantics are safety-significant. Each change needs a record, regression vector, full differential benchmark, schema review, report rebuild, and migration note.

Chapter 22

Retirement

Version 0.6 should be retired only after a successor supplies a migration path or explicitly documents incompatibility. Source, report, tests, results, environment, schemas, hashes, and incident vectors remain immutable evidence. Historical benchmark files are never rewritten in place.

Part VI

Separate Domain Adapters

Chapter 23

A0–A5 qualification protocol

A common notation is not scientific unification. For domain d , a credible adapter requires state space X_d , accepted evolution Φ_d , NHDF encoder E_d , decoder D_d , uncertainty, invariants, and a declared operating domain. It must test

$$\sup_{x \in \mathcal{D}_d} \|D_d(\mathcal{N}_{\Delta t}(E_d(x))) - \Phi_d^{\Delta t}(x)\|_{W_d} \leq \epsilon_d. \quad (23.1)$$

The maturity ladder is A0 terminology, A1 formal map, A2 simulation, A3 component experiment, A4 system demonstration, and A5 independent replication.

Chapter 24

Adapter matrix

Table 24.1: Separated adapter status.

Domain	Current level	Proposed next experiment	Hard boundary
Graphics and game engines	A2	Sandbox engine comparison of misses, time, memory, determinism	Detection only; no production certification or response quality
Robotics and public safety	A0	Hardware-in-loop navigation with independent stop	No policing authority, targeting, or force decision
Measurement and control	A0	Instrument-specific calibration and uncertainty study	Coordinate reuse is not a universal sensor law
SAR and wave sensing	A0	Raw-data autofocus benchmark against accepted baselines	CCD success does not imply SAR improvement
GPU and edge AI	A0	Equal-quality memory/latency benchmark	No compression advantage from geometry alone
Optical and photonic	A0	Calibrated physical implementation of one operator pair	No zero-power or perfect correction claim
Quantum and Cross-Kerr	A0	Hamiltonian, channel, noise, resource and fidelity experiment	No biological gate or quantum advantage claim
Chemical networks	A0	Compare to mass-action/reaction-diffusion model	NHDF does not replace stoichiometry or thermodynamics
Biological and medical	A0	Non-therapeutic measurement/simulation under ethics approval	No diagnosis, treatment, regeneration, or dementia reversal
Fabrication	A0	Foundry or 2PP test coupon with metrology	Curvilinear coordinates do not bypass process physics
Cosmology and holography	A0	Specify two theories and a real bulk-boundary map	An SDF boundary alone is not holographic duality

Part VII

Limitations and Next Releases

Chapter 25

Known limitations

1. GJK and CA use binary64 rather than exact predicates.
2. General **HIT** is a contact-band event.
3. Angular bounds can be loose and increase iterations.
4. No EPA or robust penetration manifold is included.
5. Triangle is a convex support set, not a complete triangle–triangle/deforming profile.
6. Non-convex bodies require decomposition and external pair ownership.
7. The BVH is readable and deterministic, not production-parallel.
8. C++ output is not field-for-field identical to Python certificates.
9. CUDA device compile and equivalence remain open.
10. Wider substrate mappings remain hypotheses.

Chapter 26

Roadmap to v1.0

26.1 v0.7: linear primitives and deforming meshes

Add inclusion/interval edge–edge and triangle–triangle profiles, deforming vertex trajectories, exact/filtered predicates, and a public TOI dataset adapter.

26.2 v0.8: contact dynamics

Add manifold generation, simultaneous-contact grouping, friction/restitution interface, energy/momentum diagnostics, and solver-level tunnelling regression.

26.3 v0.9: acceleration and qualification

Complete CUDA rigid-convex device code, deterministic compaction, CPU/GPU differential replay, production BVH/refit, performance/thermal measurement, and an engine safety case.

26.4 v1.0: stable profile

Freeze schemas, publish a conformance suite, obtain independent replication, and define supported and unsupported scene classes plus retirement commitments.

Chapter 27

Conclusion

The scientifically useful center of NHDF is an ordered, bounded, causal architecture for constrained state, routing, motion, observation, and feedback. Version 0.6 makes that center more literal for computer-graphics CCD: convex shape semantics are explicit, distance uncertainty is visible, angular motion is bounded, safe advancement is derived, contact-band time is distinguished from exact impact, resources are finite, and failure remains failure.

The release does not prove all geometry or all substrates. It does provide a reproducible next step: 4350 authoritative narrow-phase validation cases, independent formulations, an executable Python/C++ reference, a bounded BVH, versioned certificates, and a precise GPU gate. The correct next scientific move is not to broaden the claim; it is to attack the difficult v0.7 degeneracies and reproduce the evidence independently.

Part VIII

Technical Appendices

Appendix A

Property sketches

A.1 Safe advancement

Assume separation is Lipschitz with constant V on $[t, t + h]$ and $d(t) \geq d_{lo} - \epsilon_d$. For

$$h \leq \frac{d_{lo} - \epsilon_d - \tau_c}{V}, \tag{A.1}$$

we have $d(t + h) \geq \tau_c$. Multiplying by $\eta \leq 1$ preserves the inequality. The proof fails if V is underestimated or the lower bound is invalid, which is why both are explicit contracts.

A.2 Bounded memory

If all candidate, trace, simplex, and iteration stores have finite capacities, and no rejected proposal creates hidden allocation, the resource bound is invariant by induction. Overflow must be an output state, not silent truncation.

A.3 Contact-band lead

For approximately constant normal closing speed $v_n > 0$, a band of width $\delta = \tau_c + \epsilon_d$ can trigger approximately δ/v_n before exact zero separation. The benchmark's positive lead and zero late values are consistent with this intended semantics.

Appendix B

Selected public types

Listing B.1: Python public result/configuration types.

```
1 """Public result and configuration types for NHDF-CCD v0.6."""
2 from __future__ import annotations
3
4 from dataclasses import asdict, dataclass, field
5 from enum import Enum
6 from typing import Any
7
8 import numpy as np
9
10 from .math3d import Array
11
12
13 class CCDStatus(str, Enum):
14     HIT = "HIT"
15     MISS = "MISS"
16     INITIAL_OVERLAP = "INITIAL_OVERLAP"
17     INCONCLUSIVE = "INCONCLUSIVE"
18     RESOURCE_EXHAUSTED = "RESOURCE_EXHAUSTED"
19     INVALID_INPUT = "INVALID_INPUT"
20     NUMERICAL_FAILURE = "NUMERICAL_FAILURE"
21
22
23 @dataclass(frozen=True, slots=True)
24 class CAConfig:
25     contact_tolerance: float = 1e-6
26     distance_error_budget: float = 1e-8
27     time_tolerance: float = 1e-10
28     safety_factor: float = 0.9
29     min_safe_step: float = 1e-14
30     max_iterations: int = 128
31     refinement_iterations: int = 48
32     gjk_max_iterations: int = 64
33     gjk_absolute_tolerance: float = 1e-11
34     gjk_relative_tolerance: float = 1e-11
35     trace_limit: int = 256
36
37     def __post_init__(self) -> None:
38         if not (0.0 < self.safety_factor <= 1.0):
39             raise ValueError("safety_factor must lie in (0,1]")
40         if self.contact_tolerance < 0.0 or self.distance_error_budget < 0.0:
41             raise ValueError("tolerances must be non-negative")
42         if self.time_tolerance <= 0.0 or self.min_safe_step <= 0.0:
43             raise ValueError("time tolerances must be positive")
44         if self.max_iterations < 1 or self.refinement_iterations < 0 or self.gjk_max_iterations < 1:
45             raise ValueError("iteration limits must be valid")
46
47
48 @dataclass(frozen=True, slots=True)
49 class TraceStep:
50     iteration: int
51     time: float
52     distance: float
53     speed_bound: float
54     safe_step: float
55     gjk_iterations: int
56     gjk_termination: str
57
58
59 @dataclass(frozen=True, slots=True)
60 class ContactCertificate:
61     schema_version: str
62     profile: str
63     time_semantics: str
```

```

64     contact_tolerance: float
65     distance_error_budget: float
66     status: CCDStatus
67     body_a: str
68     body_b: str
69     toi_lower: float | None
70     toi_upper: float | None
71     distance_at_upper: float | None
72     witness_a: Array | None
73     witness_b: Array | None
74     normal_a_to_b: Array | None
75     feature_a: str | None
76     feature_b: str | None
77     conservative_speed_bound: float | None
78     ca_iterations: int
79     gjk_iterations_total: int
80     termination: str
81     trace_sha256: str
82     trace: tuple[TraceStep, ...] = field(default_factory=tuple)
83     warnings: tuple[str, ...] = field(default_factory=tuple)
84
85     def to_dict(self, include_trace: bool = True) -> dict[str, Any]:
86         def convert(value: Any) -> Any:
87             if isinstance(value, np.ndarray):
88                 return [float(x) for x in value]
89             if isinstance(value, Enum):
90                 return value.value
91             if isinstance(value, tuple):
92                 return [convert(x) for x in value]
93             if hasattr(value, "__dataclass_fields__"):
94                 return {k: convert(v) for k, v in asdict(value).items()}
95             return value
96
97         data = {name: convert(getattr(self, name)) for name in self.__dataclass_fields__}
98         if not include_trace:
99             data.pop("trace", None)
100         return data
101
102
103 @dataclass(frozen=True, slots=True)
104 class BroadPhaseResult:
105     status: CCDStatus
106     pairs: tuple[tuple[str, str], ...]
107     tested_aabbs: int
108     capacity: int
109     termination: str
110
111     def to_dict(self) -> dict[str, Any]:
112         return {
113             "status": self.status.value,
114             "pairs": [list(p) for p in self.pairs],
115             "tested_aabbs": self.tested_aabbs,
116             "capacity": self.capacity,
117             "termination": self.termination,
118         }

```

Appendix C

CUDA shared profile source

Listing C.1: Host/device exact sphere profile shared by the CUDA kernel and host harness.

```
1 #pragma once
2 #include <cmath>
3 #include <stdint>
4
5 #ifdef __CUDACC__
6 #define NHDF_HD __host__ __device__
7 #else
8 #define NHDF_HD
9 #endif
10
11 namespace nhdf_cuda_v06 {
12
13 enum Status : std::uint32_t { MISS=0, HIT=1, INITIAL_OVERLAP=2, INVALID_INPUT=3, NUMERICAL_FAILURE=4, UNSUPPORTED=5 };
14 struct Vec3 { double x,y,z; };
15 struct SpherePair {
16     Vec3 center_a0, velocity_a;
17     double radius_a;
18     Vec3 center_b0, velocity_b;
19     double radius_b;
20 };
21 struct PairResult { std::uint32_t status; double toi; double residual; };
22
23 NHDF_HD inline Vec3 sub(Vec3 a, Vec3 b){ return {a.x-b.x,a.y-b.y,a.z-b.z}; }
24 NHDF_HD inline double dot(Vec3 a, Vec3 b){ return a.x*b.x+a.y*b.y+a.z*b.z; }
25 NHDF_HD inline bool finite_scalar(double x){
26 #ifdef __CUDA_ARCH__
27     return isfinite(x);
28 #else
29     return std::isfinite(x);
30 #endif
31 }
32 NHDF_HD inline bool finite3(Vec3 a){ return finite_scalar(a.x)&&finite_scalar(a.y)&&finite_scalar(a.z); }
33
34 NHDF_HD inline PairResult sphere_sphere_linear(const SpherePair& p){
35     if(!finite3(p.center_a0)||!finite3(p.center_b0)||!finite3(p.velocity_a)||!finite3(p.velocity_b)||
36         !finite_scalar(p.radius_a)||!finite_scalar(p.radius_b)||p.radius_a<0.0||p.radius_b<0.0)
37         return {INVALID_INPUT,-1.0,0.0};
38     const Vec3 x=sub(p.center_a0,p.center_b0), v=sub(p.velocity_a,p.velocity_b);
39     const double r=p.radius_a+p.radius_b;
40     const double c=dot(x,x)-r*r;
41     if(c<=0.0) return {INITIAL_OVERLAP,0.0,c};
42     const double a=dot(v,v), b=2.0*dot(x,v);
43     if(a<=1e-30) return {MISS,-1.0,c};
44     const double disc=b*b-4.0*a*c;
45     if(disc<0.0) return {MISS,-1.0,disc};
46     const double root=sqrt(disc<0.0?0.0:disc);
47     const double q=-0.5*(b+::copysign(root,b));
48     double t0=q/a;
49     double t1=(::fabs(q)>1e-30)?c/q:1e300;
50     double t=1e300;
51     if(t0>=0.0&&t0<=1.0)t=t0;
52     if(t1>=0.0&&t1<=1.0&&t1<t)t=t1;
53     return t<1e200 ? PairResult{HIT,t,disc} : PairResult{MISS,-1.0,disc};
54 }
55
56 } // namespace nhdf_cuda_v06
```

Listing C.2: CUDA pair kernel. Device compilation remains unverified.

```
1 #include "nhdf_cuda_common.cuh"
2 using namespace nhdf_cuda_v06;
3
```

```
4 extern "C" __global__ void nhdf_sphere_pair_ccd_v06(  
5     const SpherePair* __restrict__ pairs,  
6     PairResult* __restrict__ results,  
7     std::uint32_t count)  
8 {  
9     const std::uint32_t index=blockIdx.x*blockDim.x+threadIdx.x;  
10    if(index<count) results[index]=sphere_sphere_linear(pairs[index]);  
11 }
```

Appendix D

Machine-readable benchmark summary

Listing D.1: Seeded v0.6 benchmark summary JSON.

```
1 {
2   "authoritative_checks": {
3     "linear_sphere_vs_analytic": {
4       "elapsed_seconds": 17.87441551700067,
5       "endpoint_only_missed_hits": 1034,
6       "exact_hits": 1100,
7       "exact_misses": 300,
8       "false_negatives": 0,
9       "false_positives": 0,
10      "max_contact_band_lead": 1.0911063195173654e-06,
11      "max_late_after_exact": 0.0,
12      "max_total_gjk_iterations_per_query": 43,
13      "mean_ca_iterations": 16.09,
14      "queries": 1400,
15      "queries_per_second": 78.32423939504122,
16      "unresolved": 0
17    },
18    "orbiting_sphere_vs_analytic": {
19      "analytic_hits_or_initial_overlaps": 350,
20      "analytic_misses": 200,
21      "construction_or_oracle_disagreements": 0,
22      "elapsed_seconds": 5.535645764000037,
23      "false_negatives": 0,
24      "false_positives": 0,
25      "max_contact_band_lead": 8.533020241507927e-07,
26      "max_late_after_exact": 0.0,
27      "mean_ca_iterations": 11.287272727272727,
28      "queries": 550,
29      "queries_per_second": 99.35606855063139,
30      "unresolved": 0
31    },
32    "static_gjk_vs_analytic_primitives": {
33      "elapsed_seconds": 14.645905380999466,
34      "families": [
35        "sphere_sphere",
36        "sphere_capsule",
37        "capsule_capsule",
38        "sphere_box"
39      ],
40      "gjk_unresolved": 0,
41      "lower_bound_violations_over_tolerance": 0,
42      "max_gjk_bound_gap": 7.068834406709357e-11,
43      "max_lower_bound_violation": 1.7763568394002505e-15,
44      "max_upper_absolute_error": 6.567280053104696e-11,
45      "max_upper_bound_violation": 1.7763568394002505e-15,
46      "mean_gjk_iterations": 6.854583333333333,
47      "overlap_classification_disagreements": 0,
48      "queries": 2400,
49      "queries_per_family": 600,
50      "queries_per_second": 163.8683261680487,
51      "upper_bound_violations_over_tolerance": 0
52    },
53    "swept_bvh_vs_bruteforce": {
54      "brute_force_checked_scales": [
55        100,
56        300
57      ],
58      "largest_body_count": 2000,
59      "largest_scale_candidate_pairs": 50,
60      "largest_scale_seconds": 0.5125778460005677,
61      "pair_set_disagreements": 0,
62      "scales": [
63        100,
```

```

64     300,
65     1000,
66     2000
67   ]
68 }
69 },
70 "diagnostic_crosschecks": {
71   "polytope_qp": {
72     "elapsed_seconds": 0.4971050389995071,
73     "interpretation": "diagnostic only; the numerical QP is not a release oracle when it returns above a known feasible GJK witness",
74     "max_gjk_upper_minus_qp": 9.827091279840872e-16,
75     "max_qp_minus_gjk_upper": 9.668315801032463e-08,
76     "qp_failures": 0,
77     "qp_results_above_known_feasible_gjk_witness": 0,
78     "queries": 100
79   }
80 },
81 "environment": {
82   "cpu_count": 5,
83   "numpy": "2.3.5",
84   "platform": "linux",
85   "python": "3.13.5",
86   "scipy": "1.17.0"
87 },
88 "generated_unix_time": 1788272852.5989337,
89 "release": "0.6.0",
90 "schema_version": "nhdf-ccd-benchmark-summary-0.6.0",
91 "scope_note": "Observed results on generated binary64 cases are not a proof over all real inputs.",
92 "seed_decimal": 1313358918,
93 "seed_hex": "0x4e484446",
94 "stage_seeds": {
95   "diagnostic convex QP": 1313358919,
96   "linear sphere CCD": 1313358920,
97   "orbiting sphere CCD": 1313358921,
98   "static GJK vs analytic primitives": 1313358918,
99   "swept BVH": 1313358922
100 }
101 }

```


Appendix E

Release file map

The full source and result files are in the accompanying ZIP. Generated build directories and interpreter caches are excluded from the release manifest.

Listing E.1: Release tree before final packaging.

```
1 00_provenance/CLAIM_LEDGER.csv (1399 bytes)
2 00_provenance/CLAIM_LEDGER.json (2100 bytes)
3 00_provenance/PROVENANCE_POLICY.md (985 bytes)
4 00_provenance/SOURCE_HASHES.csv (906 bytes)
5 00_provenance/SOURCE_REGISTER.csv (1250 bytes)
6 00_provenance/SOURCE_REGISTER.md (1732 bytes)
7 00_provenance/source_material/S1_original_operator_chain.pdf (382884 bytes)
8 00_provenance/source_material/S2_NHDF_Formal_Specification_v0.1.pdf (372636 bytes)
9 00_provenance/source_material/S3_commute_design_transcript.pdf (470176 bytes)
10 00_provenance/source_material/S5_NHDF_CCD_v0.5_previous_release.pdf (1756026 bytes)
11 00_provenance/source_material/S5_predecessor_release_v0.5.pdf (1756026 bytes)
12 00_provenance/source_material/previous_releases/NHDF_CCD_Cradle_to_Grave_v0.4.pdf (773984 bytes)
13 00_provenance/source_material/previous_releases/NHDF_CCD_Cradle_to_Grave_v0.4.zip (2896294 bytes)
14 00_provenance/source_material/previous_releases/NHDF_CCD_Cradle_to_Grave_v0.5.pdf (1756026 bytes)
15 01_problem_definition/ACCEPTANCE_CRITERIA.md (1566 bytes)
16 01_problem_definition/CLAIM_LEDGER.md (1229 bytes)
17 01_problem_definition/HAZARD_ANALYSIS.md (1159 bytes)
18 01_problem_definition/REQUIREMENTS.md (1549 bytes)
19 01_problem_definition/SCOPE_AND_EXCLUSIONS.md (843 bytes)
20 01_problem_definition/SCOPE_AND_REQUIREMENTS.md (2009 bytes)
21 01_problem_definition/TRACEABILITY.csv (1118 bytes)
22 02_formal_specification/CCD_ALGORITHM.md (1568 bytes)
23 02_formal_specification/CERTIFICATE_SEMANTICS.md (1320 bytes)
24 02_formal_specification/GJK_CONTRACT.md (1366 bytes)
25 02_formal_specification/OPERATOR_CONTRACT.md (1442 bytes)
26 02_formal_specification/RIGID_CONVEX_CCD_SPECIFICATION.md (2941 bytes)
27 02_formal_specification/STATE_MACHINE.md (813 bytes)
28 02_formal_specification/adaptor_qualification.schema.json (832 bytes)
29 02_formal_specification/broadphase.schema.json (705 bytes)
30 02_formal_specification/certificate.schema.json (2870 bytes)
31 02_formal_specification/scene.schema.json (3086 bytes)
32 02_formal_specification/schemas/adaptor.schema.json (980 bytes)
33 02_formal_specification/schemas/certificate.schema.json (3996 bytes)
34 02_formal_specification/schemas/scene.schema.json (4437 bytes)
35 02_formal_specification/schemas/telemetry.schema.json (1405 bytes)
36 02_formal_specification/state_machine.json (1421 bytes)
37 03_reference/cpp/CMakeLists.txt (516 bytes)
38 03_reference/cpp/README.md (581 bytes)
39 03_reference/cpp/include/nhdf_ccd_v06/ccd.hpp (3307 bytes)
40 03_reference/cpp/src/ccd.cpp (14739 bytes)
41 03_reference/cpp/tests/smoke.cpp (1424 bytes)
42 03_reference/python/README.md (652 bytes)
43 03_reference/python/examples/rotating_capsule_box.json (743 bytes)
44 03_reference/python/examples/rotating_capsule_box_result.json (7084 bytes)
45 03_reference/python/examples/run_demo.py (321 bytes)
46 03_reference/python/pyproject.toml (382 bytes)
47 03_reference/python/requirements-dev.txt (63 bytes)
48 03_reference/python/src/nhdf_ccd_v06/__init__.py (918 bytes)
49 03_reference/python/src/nhdf_ccd_v06/bvh.py (4481 bytes)
50 03_reference/python/src/nhdf_ccd_v06/ccd.py (12675 bytes)
51 03_reference/python/src/nhdf_ccd_v06/cli.py (1048 bytes)
52 03_reference/python/src/nhdf_ccd_v06/exact.py (4564 bytes)
53 03_reference/python/src/nhdf_ccd_v06/gjk.py (10910 bytes)
54 03_reference/python/src/nhdf_ccd_v06/io.py (2323 bytes)
55 03_reference/python/src/nhdf_ccd_v06/math3d.py (3617 bytes)
56 03_reference/python/src/nhdf_ccd_v06/motion.py (3464 bytes)
57 03_reference/python/src/nhdf_ccd_v06/oracles.py (3078 bytes)
58 03_reference/python/src/nhdf_ccd_v06/shapes.py (6754 bytes)
59 03_reference/python/src/nhdf_ccd_v06/types.py (3655 bytes)
60 04_verification/ADVERSARIAL_CASE_CATALOG.md (1402 bytes)
```

```

61 04_verification/ADVERSARIAL_CATALOG.md (822 bytes)
62 04_verification/INDEPENDENCE_NOTES.md (701 bytes)
63 04_verification/TEST_PLAN.md (1270 bytes)
64 04_verification/benchmark_rerun_full.txt (244 bytes)
65 04_verification/build_release_final.txt (1802 bytes)
66 04_verification/build_release_manifest_final.txt (189 bytes)
67 04_verification/build_release_rerun.txt (43740 bytes)
68 04_verification/build_release_rerun2.txt (85636 bytes)
69 04_verification/cpp_build.txt (321 bytes)
70 04_verification/cpp_configure.txt (404 bytes)
71 04_verification/cpp_full_run.txt (1057 bytes)
72 04_verification/cpp_test_run.txt (1057 bytes)
73 04_verification/gpu_host_build.txt (185 bytes)
74 04_verification/gpu_host_configure.txt (394 bytes)
75 04_verification/gpu_host_rerun.txt (579 bytes)
76 04_verification/gpu_host_test.txt (319 bytes)
77 04_verification/gpu_static_contract.txt (32 bytes)
78 04_verification/gpu_validation.txt (579 bytes)
79 04_verification/latex_build_final.txt (86125 bytes)
80 04_verification/pdf_fonts_check.txt (21 bytes)
81 04_verification/pdf_inspect.txt (3162 bytes)
82 04_verification/pdf_preflight.txt (157 bytes)
83 04_verification/pdf_render_smoke.txt (39 bytes)
84 04_verification/python_test_run.json (207 bytes)
85 04_verification/python_test_run.txt (179 bytes)
86 04_verification/release_validate_final.txt (158 bytes)
87 04_verification/release_validate_final_stdout.txt (158 bytes)
88 04_verification/render_all_pages_final.txt (62 bytes)
89 04_verification/schema_validation.txt (32 bytes)
90 04_verification/tests/conftest.py (156 bytes)
91 04_verification/tests/test_bvh_io.py (2080 bytes)
92 04_verification/tests/test_bvh_io_schema.py (2848 bytes)
93 04_verification/tests/test_conservative_advancement.py (4436 bytes)
94 04_verification/tests/test_contracts_and_cli.py (4723 bytes)
95 04_verification/tests/test_exact_ca.py (3304 bytes)
96 04_verification/tests/test_exact_profiles.py (1327 bytes)
97 04_verification/tests/test_gjk.py (2607 bytes)
98 04_verification/tests/test_math_shapes.py (2221 bytes)
99 04_verification/tests/test_math_shapes_gjk.py (2298 bytes)
100 04_verification/tests/test_randomized.py (1535 bytes)
101 04_verification/tests/test_rotation_acceleration.py (2937 bytes)
102 04_verification/vectors/core_vectors.json (2087 bytes)
103 05_benchmarks/figures/bvh_scaling.png (65005 bytes)
104 05_benchmarks/figures/ca_trace.png (70808 bytes)
105 05_benchmarks/figures/contact_band_lead.png (53928 bytes)
106 05_benchmarks/figures/gjk_accuracy.png (56443 bytes)
107 05_benchmarks/figures/gjk_bound_closure.png (68176 bytes)
108 05_benchmarks/figures/validation_coverage.png (59742 bytes)
109 05_benchmarks/figures/validation_outcomes.png (49853 bytes)
110 05_benchmarks/make_figures.py (5889 bytes)
111 05_benchmarks/results/benchmark_decision.json (3847 bytes)
112 05_benchmarks/results/benchmark_run.txt (4285 bytes)
113 05_benchmarks/results/benchmark_summary.csv (2530 bytes)
114 05_benchmarks/results/benchmark_summary.json (3258 bytes)
115 05_benchmarks/results/bvh_scaling.csv (419 bytes)
116 05_benchmarks/results/gjk_bound_trace.json (756 bytes)
117 05_benchmarks/results/linear_sphere_cases.csv (198097 bytes)
118 05_benchmarks/results/orbiting_sphere_cases.csv (82251 bytes)
119 05_benchmarks/results/reference_certificates.json (25809 bytes)
120 05_benchmarks/results/stage_assemble_run.txt (3847 bytes)
121 05_benchmarks/results/stage_bvh_run.txt (604 bytes)
122 05_benchmarks/results/stage_linear_run.txt (785 bytes)
123 05_benchmarks/results/stage_orbit_run.txt (783 bytes)
124 05_benchmarks/results/stage_qp_run.txt (734 bytes)
125 05_benchmarks/results/stage_static_run.txt (1076 bytes)
126 05_benchmarks/results/stages/bvh.json (555 bytes)
127 05_benchmarks/results/stages/linear.json (725 bytes)
128 05_benchmarks/results/stages/orbit.json (722 bytes)
129 05_benchmarks/results/stages/qp.json (675 bytes)
130 05_benchmarks/results/stages/static.json (1000 bytes)
131 05_benchmarks/results/static_gjk_analytic_cases.csv (414645 bytes)
132 05_benchmarks/results/static_qp_crosscheck_cases.csv (13032 bytes)
133 05_benchmarks/run_benchmarks.py (34407 bytes)
134 05_benchmarks/run_benchmarks_staged.py (6592 bytes)
135 05_benchmarks/scripts/run_benchmarks.py (25561 bytes)
136 06_gpu/GPU_EQUIVALENCE_PLAN.md (1205 bytes)
137 06_gpu/README.md (1157 bytes)
138 06_gpu/cuda/convex_core.cuh (8986 bytes)
139 06_gpu/cuda/nhdf_ccd_v06.cu (526 bytes)
140 06_gpu/cuda/nhdf_cuda_common.cuh (1932 bytes)
141 06_gpu/cuda/rigid_convex_port_contract.cuh (719 bytes)
142 06_gpu/cuda/sphere_pair_kernel.cu (362 bytes)
143 06_gpu/harness/CMakeLists.txt (367 bytes)
144 06_gpu/harness/host_core_smoke.cpp (492 bytes)
145 06_gpu/harness/run_cuda_differential.py (610 bytes)
146 06_gpu/harness/run_host_core_smoke.sh (198 bytes)
147 06_gpu/harness/static_contract_check.py (368 bytes)
148 06_gpu/harness/test_host_profile.cpp (897 bytes)
149 07_integration/ENGINE_API.md (1010 bytes)

```

```

150 07_integration/ENGINE_INTEGRATION.md (1666 bytes)
151 07_integration/INTEGRATION_CHECKLIST.md (770 bytes)
152 07_integration/MIGRATION_v0.5_to_v0.6.md (1127 bytes)
153 07_integration/PUBLIC_SAFETY_BOUNDARY.md (486 bytes)
154 07_integration/PUBLIC_SAFETY_GOVERNANCE.md (753 bytes)
155 08_operations/CHANGE_CONTROL.md (740 bytes)
156 08_operations/INCIDENT_RESPONSE.md (868 bytes)
157 08_operations/MAINTENANCE.md (722 bytes)
158 08_operations/RUNBOOK.md (1173 bytes)
159 08_operations/telemetry_example.json (386 bytes)
160 09_retirement/DEPRECATION_POLICY.md (357 bytes)
161 09_retirement/RETIREMENT_PLAN.md (736 bytes)
162 10_domain_adapters/01_graphics_physics_engines.md (447 bytes)
163 10_domain_adapters/02_robotics_public_safety.md (410 bytes)
164 10_domain_adapters/03_measurement_sensors_actuators.md (431 bytes)
165 10_domain_adapters/04_sar_wave_sensing.md (402 bytes)
166 10_domain_adapters/05_gpu_edge_ai.md (457 bytes)
167 10_domain_adapters/06_optical_photonic.md (385 bytes)
168 10_domain_adapters/07_quantum_cross_kerr.md (417 bytes)
169 10_domain_adapters/08_chemical.md (390 bytes)
170 10_domain_adapters/09_biology_medical.md (439 bytes)
171 10_domain_adapters/10_fabrication_3d_photonics.md (428 bytes)
172 10_domain_adapters/11_cosmology.md (368 bytes)
173 10_domain_adapters/ADAPTER_CONTRACT.md (783 bytes)
174 10_domain_adapters/ADAPTER_QUALIFICATION_PROTOCOL.md (859 bytes)
175 10_domain_adapters/adapter_matrix.csv (3564 bytes)
176 10_domain_adapters/adapter_matrix.json (3548 bytes)
177 10_domain_adapters/records/AD-AI.json (560 bytes)
178 10_domain_adapters/records/AD-BIO.json (607 bytes)
179 10_domain_adapters/records/AD-CHEM.json (586 bytes)
180 10_domain_adapters/records/AD-COS.json (538 bytes)
181 10_domain_adapters/records/AD-FAB.json (601 bytes)
182 10_domain_adapters/records/AD-GFX.json (609 bytes)
183 10_domain_adapters/records/AD-MET.json (604 bytes)
184 10_domain_adapters/records/AD-OPT.json (587 bytes)
185 10_domain_adapters/records/AD-QNT.json (609 bytes)
186 10_domain_adapters/records/AD-ROB.json (601 bytes)
187 10_domain_adapters/records/AD-SAR.json (589 bytes)
188 11_report/NHDF_CCD_Cradle_to_Grave_v0.6.tex (49235 bytes)
189 11_report/figures/bvh_scaling.png (68113 bytes)
190 11_report/figures/ca_trace.png (71750 bytes)
191 11_report/figures/contact_band_lead.png (53920 bytes)
192 11_report/figures/gjk_accuracy.png (56443 bytes)
193 11_report/figures/gjk_bound_closure.png (68176 bytes)
194 11_report/figures/validation_coverage.png (59742 bytes)
195 11_report/figures/validation_outcomes.png (49853 bytes)
196 11_report/generated_metrics.tex (1009 bytes)
197 11_report/pdf_inspect_final.txt (3148 bytes)
198 11_report/pdf_preflight_final.txt (143 bytes)
199 11_report/report.tex (49235 bytes)
200 11_report/report.tex.pre_final (47638 bytes)
201 12_release/BUILD_ENVIRONMENT.md (374 bytes)
202 12_release/KNOWN_LIMITATIONS.md (830 bytes)
203 12_release/MIGRATION_v0.5_to_v0.6.md (571 bytes)
204 12_release/RELEASE_NOTES.md (797 bytes)
205 12_release/ROADMAP_v0.7_to_v1.0.md (846 bytes)
206 12_release/SBOM-lite.json (526 bytes)
207 CHANGELOG.md (1455 bytes)
208 CRADLE_TO_GRAVE_ROADMAP.md (2280 bytes)
209 CRADLE_TO_GRAVE_ROADMAP.yaml (876 bytes)
210 DISTRIBUTION_NOTICE.md (644 bytes)
211 ENVIRONMENT.json (298 bytes)
212 LICENSE.txt (1070 bytes)
213 NOTICE.md (862 bytes)
214 README.md (2463 bytes)
215 RELEASE_NOTES.md (2281 bytes)
216 RISK_REGISTER.csv (1589 bytes)
217 SECURITY.md (628 bytes)
218 TRACEABILITY_MATRIX.csv (971 bytes)
219 pyproject.toml (730 bytes)
220 scripts/build_release.py (7130 bytes)
221 scripts/release_validate.py (9874 bytes)
222 scripts/run_all.sh (482 bytes)
223 scripts/run_cpp_tests.sh (277 bytes)
224 scripts/run_gpu_host_harness.sh (526 bytes)
225 scripts/run_python_tests.sh (157 bytes)
226 scripts/validate_schemas.py (1304 bytes)
227 tools/make_sha256_manifest.py (530 bytes)

```

Appendix F

References

Bibliography

- [1] Supplied source S1. *Operator-chain prompt transcript*, 30 pages, 2026.
- [2] Tom Klootwijk. *Non-Euclidean Holographic Data Fields: A Formal Operator Specification*, Version 0.1, 31 August 2026.
- [3] Supplied source S3. *Commute design transcript with log-polar Jacobian and CCD discussion*, 40 pages, 2026.
- [4] Supplied source S4. *Spherical–Cartesian and whole-substrate extension*, 70 pages, 2026.
- [5] Tom Klootwijk. *NHDF-CCD Cradle-to-Grave*, Version 0.5, 2026.
- [6] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi. “A fast procedure for computing the distance between complex objects in three-dimensional space.” *IEEE Journal of Robotics and Automation*, 1988.
- [7] M. Montanari, N. Petrinic, and E. Barbieri. “Improving the GJK Algorithm for Faster and More Reliable Distance Queries Between Convex Objects.” *ACM Transactions on Graphics*, 2017.
- [8] S. Redon, A. Kheddar, and S. Coquillart. “Fast Continuous Collision Detection between Rigid Bodies.” *Computer Graphics Forum*, 2002.
- [9] B. Wang et al. “A Large Scale Benchmark and an Inclusion-Based Algorithm for Continuous Collision Detection.” *ACM Transactions on Graphics*, 2021. Project material: <https://continuous-collision-detection.github.io/>.
- [10] D. Belgrod et al. “Time of Impact Dataset for Continuous Collision Detection and a Scalable Conservative Algorithm.” arXiv:2112.06300.
- [11] B. Wang et al. “Fast and Exact Root Parity for Continuous Collision Detection.” *Computer Graphics Forum*, 2022.
- [12] M. Montanari. *openGJK: Fast and reliable implementation of the Gilbert–Johnson–Keerthi algorithm*, SoftwareX repository and paper, 2018.

Historical Annex: NHDF-CCD v0.5

Immutable predecessor

The following 100 pages are the complete v0.5 predecessor PDF included without textual alteration. They preserve the translational mesh-facing roadmap and the earlier v0.4 annex. v0.6 semantics and corrections in the preceding pages take precedence for the new profile.

NHDF—CCD

Cradle-to-Grave Formalization and Reference Release

Version 0.5

Translational sphere/point versus triangle and triangle-mesh continuous collision detection

Release center: literal engine/graphics Continuous Collision Detection.

Continuation: v0.4 is retained as an immutable historical annex.

External materials: optical, quantum, chemical, biological, fabrication, AI, sensing and cosmological material is mapped through independent evidence gates; no validation is inherited from CCD.

Author attribution supplied by requester: Tom Klootwijk

Declared identifier NL200678942; declared date of birth 10 July 1990

Release date: 1 September 2026

Release notice

This dossier is an engineering research artifact. The identity data on the title page is self-declared provenance, was not independently verified, and is not a cryptographic credential or evidence of technical correctness. The implementation is an analytic-filtered binary64 reference, not an exact-real proof. Its authoritative collision results are limited to the motion and geometry profile stated in this document.

The medical adapter is explicitly non-therapeutic. The quantum, optical, chemical, biological and cosmological maps are research hypotheses or formal analogies at the maturity levels recorded in Section 18. They do not establish clinical efficacy, quantum advantage, a biological photonic computer, or an ontological “entire substrate.”

Release gate	State	Evidence
Python reference tests	PASS	24 declared test methods
C++17 independent smoke	PASS	five feature/status cases
Constructed analytic corpus	PASS	16000 queries; 0 wrong
Independent numerical oracle	PASS	2500 queries; 0 false negatives/positives
Tunnelling corpus	PASS	5000 endpoint-only missed hits recovered
CUDA execution	DESIGN	ABI and fail-closed skeleton; not compiled
Rotational narrow phase	DEFERRED	conservative speed bound only
Whole-substrate theory	NON-CLAIM	adapters require independent evidence

Contents

Release notice	1
1 Executive release statement	5
1.1 What v0.5 adds over v0.4	5
1.2 Claim ledger	6
2 Cradle-to-grave lifecycle	6
2.1 Gate definitions	6
3 Provenance and epistemic separation	7
4 Typed state and operator chain	8
5 CCD problem definition	8
5.1 Normalized motion	8
5.2 Why endpoint checking is insufficient	9
5.3 Supported geometry	9

6	Geometric feature decomposition	9
6.1	Face candidate	10
6.2	Edge candidate	10
6.3	Vertex candidate	10
6.4	Full-triangle residual filter	10
7	Reference algorithm	10
7.1	Narrow phase pseudocode	10
7.2	Deterministic tie policy	11
7.3	Quadratic stability	11
7.4	Degenerate geometry	11
8	Triangle-mesh profile	12
8.1	Conservative broad phase	12
8.2	Capacity semantics	12
8.3	Earliest contact group	12
9	Certificate and status semantics	12
9.1	Contact invariants	13
9.2	Miss invariant	13
9.3	Parity boundary	13
10	Numerical policy and scale	13
11	Implementation architecture	13
11.1	Python reference	13
11.2	C++17 reference	15
11.3	File/API boundary	16
12	Verification program	16
12.1	Test classes	16
12.2	Constructed analytic corpora	16
12.3	Independent randomized differential corpus	17
12.4	Tunnelling corpus	17
13	Benchmark observations	18
14	Rotational motion: bounded, not solved	19

15 GPU/CUDA continuation	19
16 Engine integration and response boundary	19
16.1 Step sequence	19
16.2 What CCD does not do	20
16.3 Safety-relevant fallback	20
17 Operations, hazards and incident response	20
18 Independent mapping of the wider materials	21
18.1 Adapter protocol	21
18.2 Adapter register	21
18.3 Graphics and game engines	22
18.4 Robotics and public safety	22
18.5 Measurement, sensors and actuators	22
18.6 SAR and wave sensing	22
18.7 GPU and edge AI	22
18.8 Optical and photonic systems	23
18.9 Quantum and Cross-Kerr hypotheses	23
18.10 Chemical reaction networks	23
18.11 Biological, biophotonic and medical modeling	23
18.12 Fabrication	23
18.13 Cosmology and the holographic principle	23
19 Roadmap from v0.6 to v1.0	23
19.1 v0.6: rotational and convex foundation	23
19.2 v0.7: triangle-triangle and deformation	24
19.3 v0.8: dynamics integration	24
19.4 v0.9: qualification	24
19.5 v1.0: stable profiles	24
20 Reproducibility and release integrity	24
21 Conclusion	24
A Requirements traceability summary	26
B Certificate example	26

C Failure decision table	26
D Source and package map	27
E References	27
Immutable historical annex notice	29

1 Executive release statement

Version 0.5 continues the roadmap by moving the strongest NHDF research nucleus—bounded, causal, typed geometric event isolation—into a mesh-facing CCD profile. It adds a fully executable sphere/point versus triangle solver, a deterministic triangle-mesh query path, checked certificates, an independently written C++17 implementation, a numerical differential oracle, synthetic benchmark corpora, engine handoff rules, and lifecycle evidence from source provenance through retirement.

The release deliberately narrows its strongest claim:

Within normalized time $t \in [0, 1]$, for a sphere or point undergoing uniform translation relative to a static triangle, the reference generates analytic face/edge/vertex candidate times, filters them by direct distance to the complete triangle, and returns the earliest checked contact or an explicit non-authoritative failure. A mesh query applies conservative swept-AABB enumeration and does not certify a miss after truncation.

This claim is testable. It is distinct from the larger proposition that one symbolic chain is the literal substrate of physics, computation, chemistry, biology or cosmology. Shared notation can support adapters; it cannot substitute for domain equations or evidence.

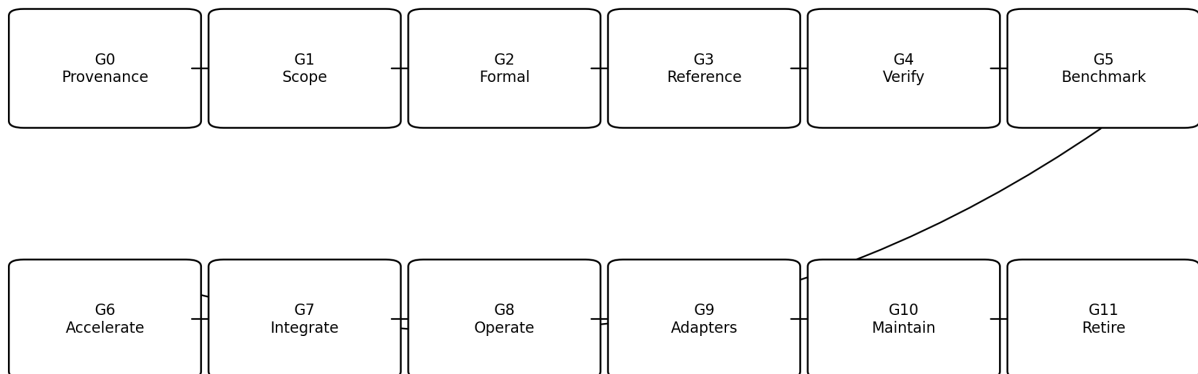
1.1 What v0.5 adds over v0.4

Area	v0.4 baseline	v0.5 continuation
Narrow phase	primitive reference profiles and conservative-advancement framework	analytic sphere/point-triangle feature decomposition with direct residual filtering
Mesh path	architectural roadmap	swept-AABB candidate phase, deterministic triangle order and earliest manifold grouping
Motion	linear primitives; future generality	uniform triangle translation through relative motion; rotation explicitly bounded but unsupported
Certificate	TOI/witness/failure concepts	versioned schema, feature IDs, time enclosure, residual, trace and capacity counts
Cross-language	Python/C++ seed components	independently written C++17 feature solver and compiled smoke test
Validation	primitive/tunnelling corpus	constructed feature corpora plus independent convex-distance oracle and mesh scaling
GPU	semantics skeleton	concrete batch ABI, stable-reduction plan and fail-closed finite-input kernel
External domains	mapped extensions	qualification ladder A0–A5 and one failure boundary per adapter
Lifecycle	eleven release gates	migration, immutable baseline annex, SBOM, operations, decommission and atomic release procedure

1.2 Claim ledger

Classification	Claim	State
Implemented	Translational sphere/point-triangle and uniformly translating triangle-mesh reference	PASS
Observed	Bundled tests and seeded corpora meet their declared gates in the recorded environment	PASS
Designed	CUDA port preserving statuses, residuals and deterministic reduction	DESIGN
Planned	Rotation, triangle-triangle, deformation, exact predicates, production BVH and solver integration	DEFERRED
Rejected as inference	A common coordinate/parity vocabulary proves one physical substrate	NON-CLAIM
Rejected as inference	Biomedical notation establishes diagnosis, repair or dementia treatment	NON-CLAIM

2 Cradle-to-grave lifecycle



Immutable history and machine-readable evidence accompany each transition.

Figure 1: The release is treated as a lifecycle, not only an algorithm. Each gate leaves machine-readable evidence and an explicit failure boundary.

2.1 Gate definitions

Gate	Name	Exit evidence
G0	Provenance	source register, byte copies where available, SHA-256 hashes, attribution policy
G1	Scope	supported/excluded profiles, requirements, acceptance criteria

Gate	Name	Exit evidence
G2	Formalization	state-threaded operator chain, geometric equations, schemas, numerical policy
G3	Reference	readable Python and independent C++17 source
G4	Verification	unit, constructed, randomized differential, vector and build tests
G5	Benchmark	seeded data, machine-readable results, figures and interpretation limits
G6	Acceleration	memory layout, status-preserving CUDA contract and equivalence gates
G7	Integration	coordinate/time conversion, substep order and contact-solver boundary
G8	Operations	telemetry, incident response, fail-closed fallback and roll-back
G9	Adapters	separate maturity/evidence record for every non-CCD domain
G10	Maintenance	semantic versioning, regression and defect policy
G11	Retirement	immutable archive, migration, support status and evidence retention
G12	History	complete v0.4 package/PDF retained without rewriting

The lifecycle is recursive in the practical sense: a future v0.6 begins a new G0/G1 cycle, consumes the accepted v0.5 evidence, and preserves v0.5 as historical input. It is not infinitely recursive at runtime; resources and generations are explicit.

3 Provenance and epistemic separation

The supplied source material contains several kinds of content: operator definitions, implementation ideas, commuting observations, speculative cross-domain maps, author metadata and dedication. Version 0.5 separates them into four evidence classes:

1. **Source assertion:** a proposition appears in a supplied document.
2. **Formal consequence:** a proposition follows from stated definitions under explicit assumptions.
3. **Implementation observation:** code produced a recorded result in the release environment.
4. **External empirical claim:** a proposition requires physical, biological, clinical or independent evidence outside this package.

Only the third class validates the bundled CCD implementation, and only within its profile. For example, the commute transcript's intuition that high-speed motion can tunnel through discrete sampling is confirmed by the dedicated corpus. The stronger idea that one lookup and one bit solve general CCD is not inherited: broad phase, feature roots, residual checks, degeneracies, capacity and failure states remain necessary.

The source PDFs and v0.4 baseline are stored under `00_provenance` and `12_legacy`. The source manifest is regenerated during release construction. No unavailable byte stream is silently reconstructed.

4 Typed state and operator chain

A central correction from the earlier canonical composition is to thread one enriched state through every operator. Define

$$\Sigma_n = (\mathcal{S}, \mathcal{F}, I, Q, C, H, \Gamma, \Theta, \chi)_n, \quad (1)$$

where $\mathcal{S}$ is scene data, $\mathcal{F}$ the coordinate/time frame, I the normalized interval, Q the query, C candidates, H checked contacts, Γ the certificate, Θ telemetry and χ status.

Each operator has one type:

$$O_j : \Sigma \rightarrow \Sigma. \quad (2)$$

The CCD specialization is

$$\Sigma_{n+1} = O_{\text{telemetry}} \circ O_{\text{certify}} \circ O_{\text{group}} \circ O_{\text{residual}} \circ O_{\text{narrow}} \circ O_{\text{enumerate}} \circ O_{\text{sweep}} \circ O_{\text{validate}} \circ O_{\text{normalize}}(\Sigma_n). \quad (3)$$



Every stage transforms one typed state and may set an explicit non-authoritative failure.

Figure 2: State-threaded CCD operator chain. A stage may refine state or preserve a non-authoritative failure; it may not erase failure into a miss.

Log-polar addressing, parity events or bounded routing can be inserted as optional addressing/telemetry policies only when their types are explicit. None is a collision certificate. The geometric truth condition remains distance and time under the declared motion.

5 CCD problem definition

5.1 Normalized motion

Let the sphere center and uniformly translating triangle be

$$p_s(t) = p_0 + tv_s, \quad (4)$$

$$T(t) = \{a + tv_T, b + tv_T, c + tv_T\}, \quad t \in [0, 1]. \quad (5)$$

Using relative displacement $v = v_s - v_T$, the query reduces to a fixed triangle:

$$f(t) = d(p_0 + tv, T(0)) - r. \quad (6)$$

The time of impact is

$$t_* = \inf\{t \in [0, 1] : f(t) \leq 0\}. \quad (7)$$

If $f(0) \leq \varepsilon_g$, the status is `INITIAL_OVERLAP`. If the set is empty and every candidate has been processed, the status is `MISS`.

Physical engine velocity u over a step Δt must be converted to the normalized displacement $v = u\Delta t$. Mixing seconds, ticks and normalized time without this conversion invalidates the result.

5.2 Why endpoint checking is insufficient

Endpoint sampling tests only $f(0)$ and $f(1)$. A continuous path may satisfy

$$f(0) > 0, \quad \min_{t \in (0,1)} f(t) \leq 0, \quad f(1) > 0. \quad (8)$$

This is tunnelling. The bundled corpus constructs 5000 such paths; all are detected by CCD and 5000 are missed by endpoint-only checking.

5.3 Supported geometry

The triangle can be regular or degenerate. A zero-area triangle is treated as the union of its segments and vertices. Radius $r = 0$ produces point-triangle crossing. Triangles are two-sided. Arbitrary rotation, per-vertex deformation and topology change are outside v0.5.

6 Geometric feature decomposition

The Minkowski offset of a triangle by a radius- r sphere is partitioned into feature regions. Earliest contact with its boundary can occur on a face sheet, an edge cylinder or a vertex sphere.

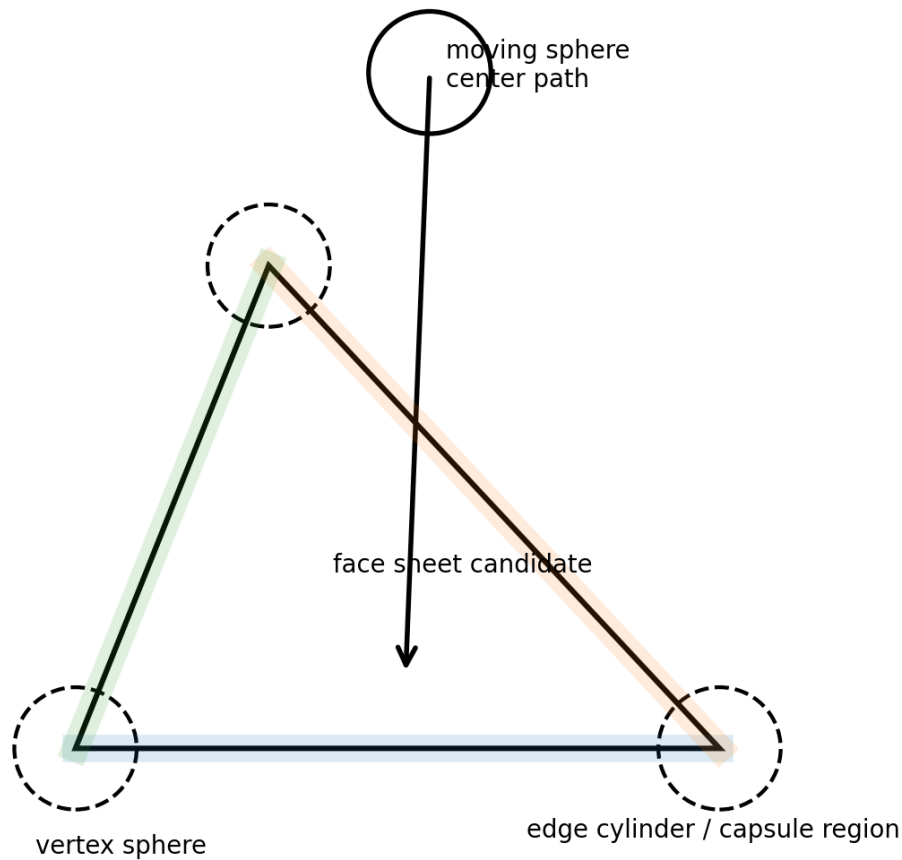


Figure 3: Schematic feature decomposition. The implementation generates analytic candidates for all regions and then checks distance to the complete triangle to reject extension roots.

6.1 Face candidate

Let

$$n = \frac{(b - a) \times (c - a)}{\|(b - a) \times (c - a)\|} \quad (9)$$

for a nondegenerate triangle. The signed center-plane distance is

$$d(t) = (p_0 - a) \cdot n + t(v \cdot n). \quad (10)$$

Candidates satisfy $d(t) = \pm r$:

$$t_{\pm} = \frac{\pm r - (p_0 - a) \cdot n}{v \cdot n}. \quad (11)$$

A candidate is retained only when the projected center lies inside or on the triangle within the barycentric tolerance.

6.2 Edge candidate

For an edge $e(s) = a + s(b - a)$, $s \in [0, 1]$, write the center motion relative to the infinite line as perpendicular components $m_{\perp}$ and $v_{\perp}$. Cylinder contact satisfies

$$\|m_{\perp} + tv_{\perp}\|^2 = r^2, \quad (12)$$

which is quadratic in t . A root is an edge-interior candidate only when the projected segment parameter is in $[0, 1]$ within tolerance.

6.3 Vertex candidate

For vertex q , contact satisfies

$$\|p_0 - q + tv\|^2 = r^2. \quad (13)$$

The reference evaluates the quadratic using a stable q -form where possible, clips roots to the normalized interval, and sorts all candidates by time, feature rank and index.

6.4 Full-triangle residual filter

Feature equations include infinite planes, lines or overlapping boundaries. Therefore every candidate is re-evaluated against the complete triangle:

$$\rho(t) = d(p_0 + tv, T) - r. \quad (14)$$

A candidate is accepted only if $|\rho(t)|$ is within the profile's residual threshold. Its witness and Voronoi feature are replaced by the complete closest-point evaluation. This is the critical defense against roots belonging to extensions rather than the finite triangle.

7 Reference algorithm

7.1 Narrow phase pseudocode

Listing 1: Normative algorithm shape; implementation details are in the accompanying source.

```

1 def sweep_sphere_triangle(p0, sphere_v, radius, triangle, triangle_v, profile):
2     validate_or_return_invalid()
3     v = sphere_v - triangle_v
4     closest0 = closest_point_triangle(p0, triangle)
5     if distance(p0, closest0) - radius <= profile.geometry_epsilon:
6         return initial_overlap_certificate()
7
8     candidates = []
9     candidates += face_plane_roots(p0, v, triangle, radius)
10    for edge in triangle.edges:
11        candidates += restricted_edge_cylinder_roots(p0, v, edge, radius)
12    for vertex in triangle.vertices:
13        candidates += vertex_sphere_roots(p0, v, vertex, radius)
14
15    candidates.sort(stable_feature_key)
16    checked = []
17    for c in candidates:
18        closest = closest_point_triangle(p0 + c.t * v, triangle)
19        residual = distance(p0 + c.t * v, closest) - radius
20        if abs(residual) <= residual_tolerance:
21            checked.append(contact(c.t, closest, residual))
22
23    if not checked:
24        return miss_certificate() # only after all candidates complete
25    return earliest_contact_group(checked, profile.manifold_time_epsilon)

```

7.2 Deterministic tie policy

Candidate ordering is

$$(t, \text{rank}(\text{feature}), \text{feature index, method}), \quad (15)$$

with face before edge before vertex only as a deterministic boundary convention. Mesh contacts add triangle ID before feature rank. The order does not change physical time; it makes serialization and replay stable.

7.3 Quadratic stability

For

$$at^2 + 2bt + c = 0, \quad (16)$$

the usual $(-b \pm \sqrt{b^2 - ac})/a$ can lose precision through cancellation. The reference computes one root through

$$q = -b - \text{sign}(b)\sqrt{b^2 - ac} \quad (17)$$

and the other as c/q when q is sufficiently nonzero. This improves, but does not certify, floating robustness.

7.4 Degenerate geometry

If squared triangle normal magnitude is at most ε_d , face roots are omitted. Closest-point and sweep operations examine the three segments, each of which may reduce to a vertex. This preserves an explicit lower-dimensional interpretation rather than dividing by near-zero area.

8 Triangle-mesh profile

8.1 Conservative broad phase

The swept sphere AABB is the coordinatewise envelope of p_0 and $p_0 + v_s$, expanded by r . A translating triangle's swept AABB envelopes its vertices at both endpoints. A candidate is generated when the boxes overlap, including geometric tolerance.

The serial reference intentionally uses a simple scan. It establishes semantics, not production asymptotics. A production BVH may replace enumeration only if it is conservative and reproduces the same candidate set or a superset.

8.2 Capacity semantics

Let N_C be the candidate count and B_C the configured budget. If $N_C > B_C$, v0.5 returns `RESOURCE_EXHAUSTED` before narrow phase. It does not process an arbitrary prefix and call the absence of a hit a miss. A best-so-far contact, if future implementations expose one, is telemetry only.

8.3 Earliest contact group

All candidate triangles are processed. Contacts within ε_M of the minimum TOI form the mesh contact group:

$$H_* = \{h \in H : t_h \leq \min_{g \in H} t_g + \varepsilon_M\}. \quad (18)$$

This supports shared edges and coplanar triangle tiles without pretending to solve the subsequent impulse manifold.

9 Certificate and status semantics

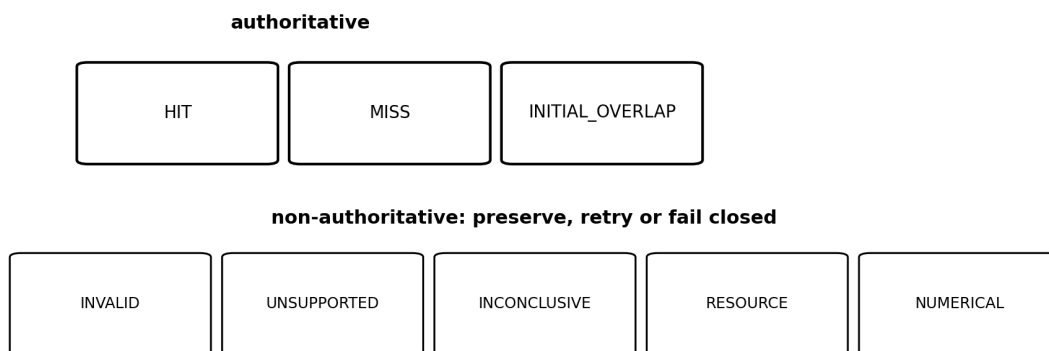


Figure 4: Authoritative results are deliberately few. Non-authoritative outcomes remain visible to the caller.

A contact record includes TOI and enclosure, center, witness, normal, separation residual, feature and triangle identifiers, method and residual-check flag. The query certificate includes solver/profile, status, candidate/processed counts, reason, deterministic trace and diagnostics.

9.1 Contact invariants

For an accepted contact,

$$0 \leq t_L \leq t \leq t_U \leq 1, \quad (19)$$

$$|||p_c - p_w|| - r| \leq \varepsilon_\rho, \quad (20)$$

$$\|n\| \approx 1 \quad (21)$$

except that a declared fallback normal is used when center and witness coincide for a zero-radius or degenerate event.

9.2 Miss invariant

A **MISS** is authoritative only if validation, conservative candidate enumeration and every candidate narrow phase completed. Any unknown status breaks that proof obligation and propagates as non-authoritative.

9.3 Parity boundary

A parity bit may summarize a trace word or detect odd bit changes against a stored reference. It cannot certify collision, distinguish all corruption, or replace the geometric residual. Topological orientation events and payload-integrity parity remain separate diagnostic channels.

10 Numerical policy and scale

The default tolerances are recorded in every certificate. They are not universal physical constants. An engine adapter must scale scene coordinates or choose a profile appropriate to expected magnitudes and condition numbers.

Parameter	Default	Function
ε_g	10^{-9}	initial overlap and geometric residual
ε_t	10^{-10}	root clipping and TOI enclosure
ε_d	10^{-24}	squared degeneracy threshold
ε_b	10^{-10}	barycentric/segment boundary
ε_M	10^{-8}	simultaneous contact grouping
B_C	10^6	candidate capacity

Extreme coordinates, nearly tangent paths and ill-conditioned triangles can exceed binary64 reliability. A caller may normalize coordinates, subdivide time, retry in higher precision or use an interval/exact backend. It may not reinterpret numerical failure as a clear path.

11 Implementation architecture

11.1 Python reference

The Python package uses immutable dataclasses, no external numerical array dependency, explicit enums and JSON-serializable certificates. Its purpose is readability, deterministic tests and a reference for other backends.

Listing 2: Public data model excerpt.

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass, field, asdict
4 from enum import Enum
5 from math import sqrt, isfinite
6 from typing import Any, Dict, Iterable, List, Optional, Sequence, Tuple
7
8
9 @dataclass(frozen=True, slots=True)
10 class Vec3:
11     x: float
12     y: float
13     z: float
14
15     def __add__(self, other: "Vec3") -> "Vec3":
16         return Vec3(self.x + other.x, self.y + other.y, self.z + other.z)
17
18     def __sub__(self, other: "Vec3") -> "Vec3":
19         return Vec3(self.x - other.x, self.y - other.y, self.z - other.z)
20
21     def __mul__(self, scalar: float) -> "Vec3":
22         return Vec3(self.x * scalar, self.y * scalar, self.z * scalar)
23
24     __rmul__ = __mul__
25
26     def __truediv__(self, scalar: float) -> "Vec3":
27         if scalar == 0.0:
28             raise ZeroDivisionError("Vec3 division by zero")
29         return Vec3(self.x / scalar, self.y / scalar, self.z / scalar)
30
31     def dot(self, other: "Vec3") -> float:
32         return self.x * other.x + self.y * other.y + self.z * other.z
33
34     def cross(self, other: "Vec3") -> "Vec3":
35         return Vec3(
36             self.y * other.z - self.z * other.y,
37             self.z * other.x - self.x * other.z,
38             self.x * other.y - self.y * other.x,
39         )
40
41     def norm2(self) -> float:
42         return self.dot(self)
43
44     def norm(self) -> float:
45         return sqrt(max(0.0, self.norm2()))
46
47     def normalized(self, eps: float = 1e-30) -> "Vec3":
48         n = self.norm()
49         if n <= eps:
50             return Vec3(0.0, 0.0, 0.0)
51         return self / n
52
53     def finite(self) -> bool:
54         return isfinite(self.x) and isfinite(self.y) and isfinite(self.z)
55
56     def as_list(self) -> List[float]:
57         return [self.x, self.y, self.z]
58
59     @classmethod
60     def from_seq(cls, values: Sequence[float]) -> "Vec3":
61         if len(values) != 3:
62             raise ValueError("Vec3 requires exactly three components")
63         return cls(float(values[0]), float(values[1]), float(values[2]))
64

```

```

65
66 @dataclass(frozen=True, slots=True)
67 class Triangle:
68     a: Vec3
69     b: Vec3
70     c: Vec3

```

11.2 C++17 reference

The C++ implementation was written independently against the same contract rather than generated from Python. The release compiles it with C++17, optimization and warning flags, then executes face, edge, vertex, miss and initial-overlap smoke cases. Passing a small smoke set is not full cross-language equivalence; deterministic JSON vectors are included for expansion.

Listing 3: C++17 public header excerpt.

```

1  #pragma once
2
3  #include <array>
4  #include <cstdint>
5  #include <cstdint>
6  #include <optional>
7  #include <string>
8  #include <vector>
9
10 namespace nhdf::ccd::v05 {
11
12     struct Vec3 {
13         double x{0.0}, y{0.0}, z{0.0};
14     };
15
16     Vec3 operator+(Vec3 a, Vec3 b) noexcept;
17     Vec3 operator-(Vec3 a, Vec3 b) noexcept;
18     Vec3 operator*(Vec3 a, double s) noexcept;
19     Vec3 operator/(Vec3 a, double s);
20     double dot(Vec3 a, Vec3 b) noexcept;
21     Vec3 cross(Vec3 a, Vec3 b) noexcept;
22     double norm2(Vec3 a) noexcept;
23     double norm(Vec3 a) noexcept;
24     Vec3 normalized(Vec3 a, double eps = 1e-30) noexcept;
25
26     struct Triangle {
27         Vec3 a, b, c;
28         std::int64_t id{-1};
29     };
30
31     enum class Status {
32         Hit,
33         Miss,
34         InitialOverlap,
35         InvalidInput,
36         NumericalFailure,
37     };
38
39     enum class Feature {
40         Face,
41         Edge,
42         Vertex,
43         Degenerate,
44         Unknown,
45     };
46
47     struct Profile {

```

```

48     double geometry_epsilon{1e-9};
49     double time_epsilon{1e-10};
50     double degeneracy_epsilon{1e-24};
51     double barycentric_epsilon{1e-10};
52     double manifold_time_epsilon{1e-8};
53 };
54
55 struct Contact {
56     double toi{0.0};
57     double toi_lower{0.0};
58     double toi_upper{0.0};
59     Vec3 center{};
60     Vec3 witness{};
61     Vec3 normal{};
62     double separation{0.0};
63     Feature feature{Feature::Unknown};
64     int feature_index{-1};
65     std::int64_t triangle_id{-1};
66     std::string method{};
67     bool residual_checked{false};
68 };
69
70 struct Certificate {

```

11.3 File/API boundary

A scene schema describes one sphere, one mesh translation and one or more triangles. A one-triangle scene invokes the narrow phase directly; a multi-triangle scene invokes mesh broad/narrow phase. The CLI writes a complete certificate and never prints only a bare time.

12 Verification program

12.1 Test classes

The 24 declared Python test methods cover:

- face, edge, vertex, tangent, endpoint and point-radius-zero events;
- misses, zero velocity, initial overlap and invalid values;
- uniform triangle translation and degenerate triangles;
- mesh earliest selection, no-candidate miss, simultaneous triangle group and capacity failure;
- deterministic serialization and angular speed bound;
- 1,500 constructed random face/edge/vertex cases;
- 1,000 randomized comparisons with an independent convex-distance oracle.

The independent oracle does not use feature roots. Distance to a closed convex triangle along an affine trajectory is minimized numerically on $[0, 1]$ by golden-section search; when the minimum penetrates, bisection isolates the earliest zero. This is independent enough to catch many implementation mistakes, but it is still numerical and not a formal proof.

12.2 Constructed analytic corpora

The benchmark constructs known-contact families:

- a center moving normally toward a triangle interior, with expected $t = \text{gap}/\text{speed}$;
- a center moving toward an edge from outside the face region;

- a center moving radially toward a vertex from outside both adjacent edges;
- separated trajectories far outside the triangle.

Across 16000 queries, the release gate observed 0 wrong results. The maximum absolute TOI error against the constructed formula was $1.354\text{e-}14$ normalized step units.

12.3 Independent randomized differential corpus

Across 2500 arbitrary line/triangle/radius queries, the numerical oracle classified 197 hits. The v0.5 reference recorded 0 false negatives, 0 false positives and maximum absolute TOI disagreement $1.943\text{e-}15$. These figures are observations for the checked seed and implementation, not a proof for all binary64 inputs.

12.4 Tunnelling corpus

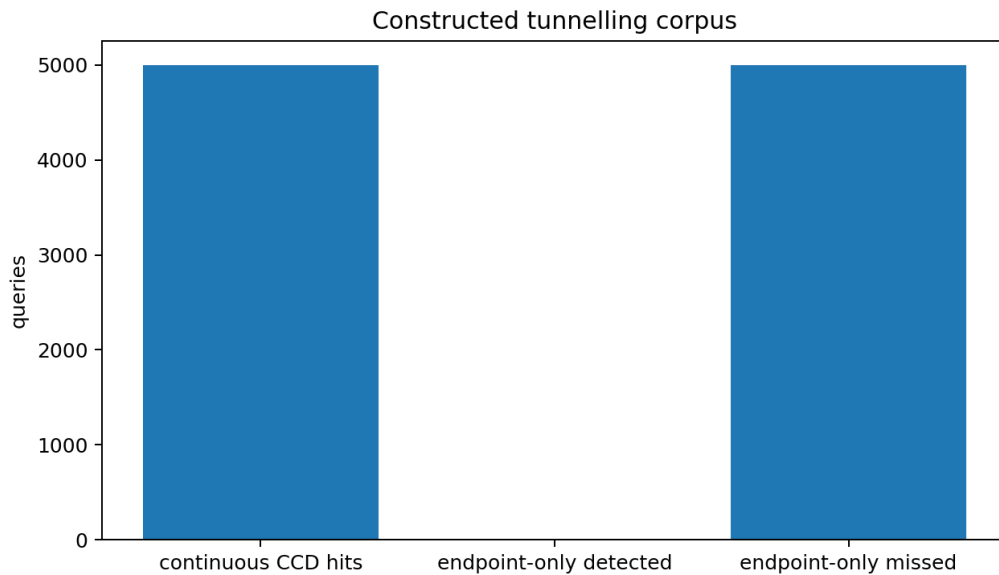


Figure 5: Every constructed path crosses the triangle between separated endpoints. Endpoint-only detection misses the continuous event; the CCD solver recovers it.

The corpus is intentionally adversarial to discrete endpoint checking. It validates the need for continuous event isolation, not general engine completeness.

13 Benchmark observations

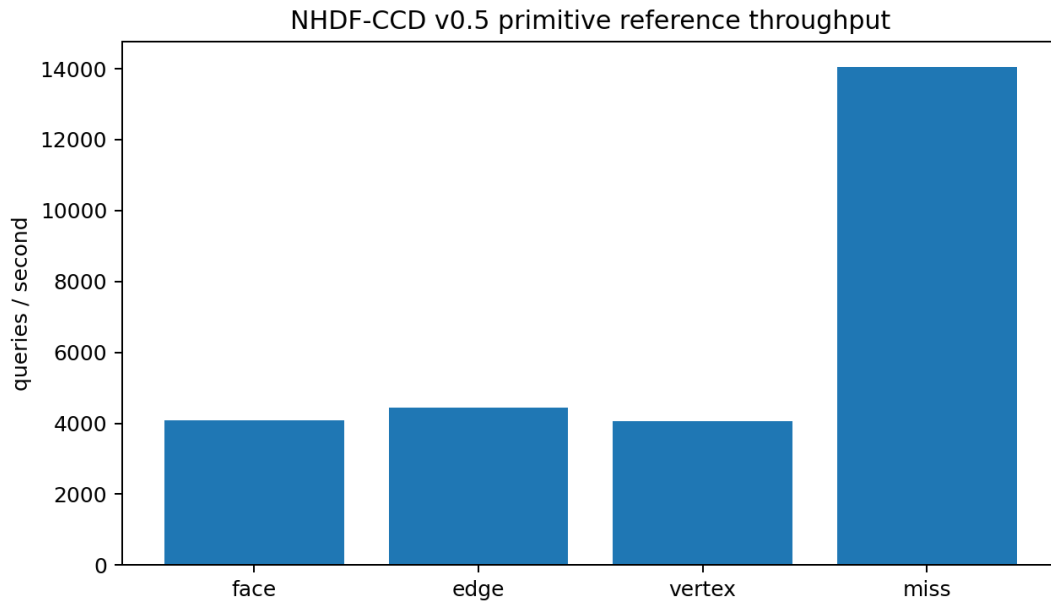


Figure 6: Single-process Python reference throughput by synthetic feature family. Values are environment-specific and intended for regression, not product marketing.

The feature benchmark measures source-readable Python and includes allocation/certificate construction. Performance is expected to vary with interpreter, processor, power state and candidate distribution. Equal-budget comparison with established engine libraries remains a v0.9 qualification task.

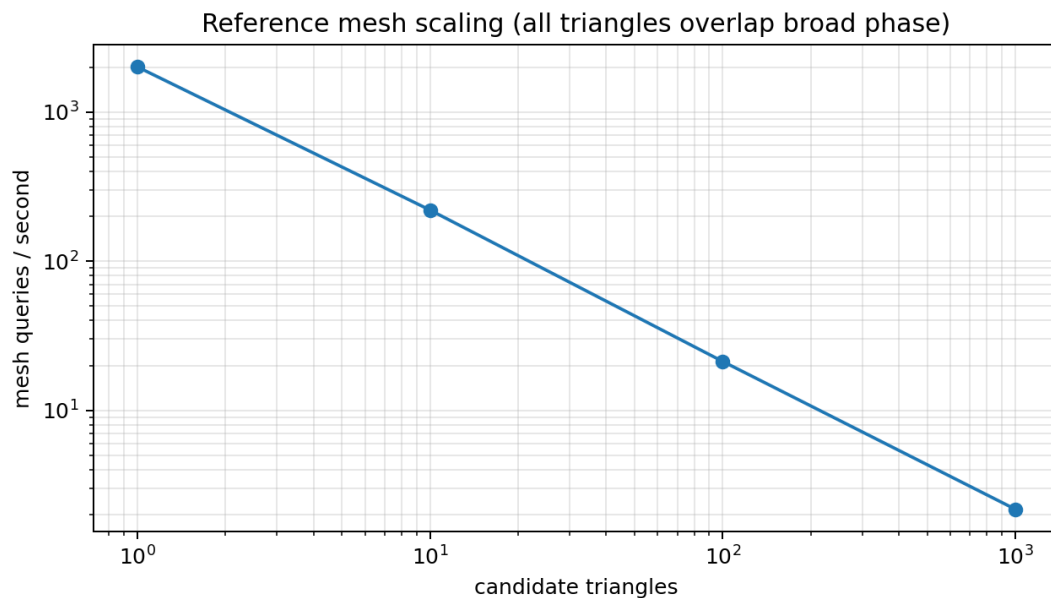


Figure 7: Worst-case serial mesh scaling when every repeated triangle survives broad phase. The line motivates BVH and parallel work but does not change reference semantics.

The all-overlap mesh benchmark intentionally defeats culling. A production broad phase should scale better for spatially separated scenes, but must preserve conservative candidates, capacity telemetry

and deterministic reduction mode.

14 Rotational motion: bounded, not solved

For a rigid body point x at distance at most R from its reference origin, instantaneous rotational speed is bounded by $|\omega|R$. A pairwise point-closing-speed bound is

$$\mu = \|v_{\text{rel}}\| + |\omega_A|R_A + |\omega_B|R_B. \quad (22)$$

This is useful for conservative advancement step selection or broad-phase expansion when R_A, R_B are true enclosing radii. It does not provide the changing triangle orientation, feature trajectories or exact TOI. v0.5 therefore exposes the bound as a separate function and requires the engine adapter to reject/reroute unsupported rotational narrow-phase queries.

Version 0.6 is planned to add support mappings and conservative advancement using this bound. Triangle-triangle rotation and deforming events require further vertex-face/edge-edge root isolation and robust predicates.

15 GPU/CUDA continuation

The accompanying CUDA source fixes a status-preserving ABI. It classifies invalid inputs and returns `INCONCLUSIVE` for finite inputs until the complete feature solver is ported. This is intentionally safer than a placeholder `MISS`.

A conforming GPU path requires:

1. conservative pair generation into a bounded buffer;
2. one-thread-per-pair candidate generation or an equivalent deterministic partition;
3. direct full-triangle residual evaluation;
4. stable segmented reduction by query and contact key;
5. explicit pair/contact overflow statuses;
6. sampled and adversarial CPU differential replay;
7. measured tolerance envelopes for any mixed-precision mode.

No CUDA compiler or GPU execution result is claimed by this release. Tensor or matrix units do not automatically compute parity, topology, collision certificates or geometric validity; each operation must be implemented and tested through the exposed hardware programming model.

16 Engine integration and response boundary

16.1 Step sequence

A literal engine integration should:

1. snapshot world transforms and velocities;
2. convert to the declared coordinate frame and normalized displacement;
3. route only supported translational sphere/triangle queries to v0.5;
4. enumerate and solve CCD with failure preservation;
5. group earliest contacts;
6. advance bodies only to a conservative accepted time;
7. pass contact geometry to a separate dynamics/contact solver;
8. consume remaining time under a substep and iteration budget;

9. record telemetry and unresolved outcomes.

16.2 What CCD does not do

Time-of-impact detection does not select impulses, restitution, friction, resting constraints, penetration correction or contact persistence. Those are dynamics problems. Applying a future contact normal before advancing to contact or ignoring simultaneous contacts can destabilize an otherwise correct CCD query.

16.3 Safety-relevant fallback

For robotics or public-safety systems, the preferred policy is

$$\text{retry precision} \rightarrow \text{subdivide} \rightarrow \text{independent solver} \rightarrow \text{conservative stop.} \quad (23)$$

This package is not a certified safety function and is not authorization for surveillance, identification, policing or autonomous force decisions.

17 Operations, hazards and incident response

Operational validity depends on observing what the solver actually does. Minimum telemetry includes profile, scale, timestep, candidate/processed counts, status, TOI interval, manifold size, feature distribution, residual tail, retries, overflows, stage timings and deterministic trace hash.

Hazard	Effect	Principal defense	Open boundary
Broad-phase under-bound	false authoritative miss	endpoint-envelope swept AABB; differential/tunnelling tests	rotation and deformation unsupported
Extension root	false hit	full-triangle closest-point residual	near-degenerate binary64 conditioning
Silent capacity truncation	missed earliest contact	fail-closed RESOURCE_EXHAUSTED	production parallel buffer design
Time/unit mismatch	wrong TOI	explicit adapter normalization contract	integrator responsibility
Nondeterministic reduction	divergent replay	stable keys and deterministic mode	GPU unverified
Extreme scale	precision loss	scale monitoring, normalization, higher precision	exact backend deferred
Adapter overclaim	scientific/clinical misuse	A0–A5 evidence ledger and non-claims	governance and independent review

A possible false MISS is a release-blocking incident. The exact scene, source/build hash, profile and environment are preserved; a minimized regression is added before any code change; sibling feature paths are reviewed; the complete release is rebuilt atomically rather than patched in place.

18 Independent mapping of the wider materials

The wider NHDF material is retained, but each domain must climb its own evidence ladder.

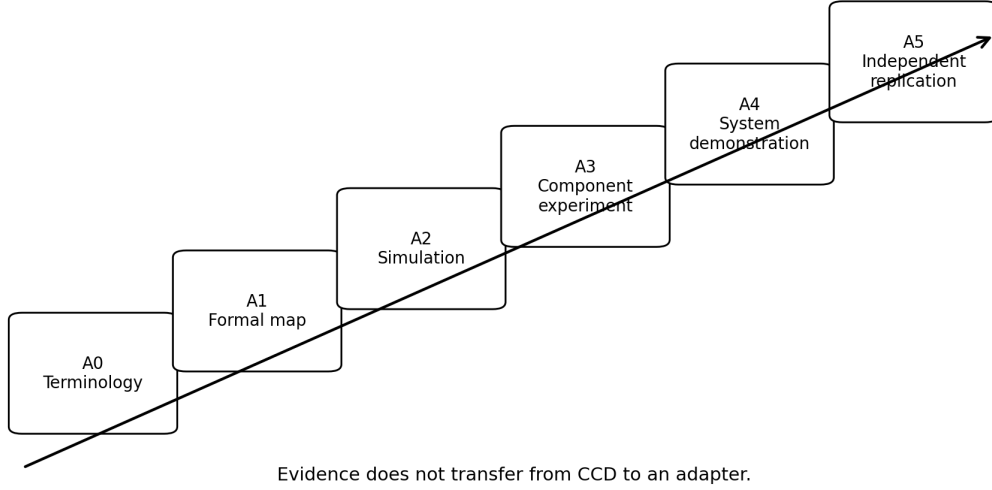


Figure 8: Adapter qualification. A formal map is not a component experiment, and a CCD test cannot move a biomedical or quantum adapter upward.

18.1 Adapter protocol

For domain d , define state X_d , accepted evolution $\Phi_d^{\Delta t}$, encoder E_d , decoder D_d , mapped operator $\mathcal{N}$, metric W_d and operating set $\mathcal{D}_d$. A falsifiable claim is

$$\sup_{x \in \mathcal{D}_d} \left\| D_d(\mathcal{N}_{\Delta t}(E_d(x))) - \Phi_d^{\Delta t}(x) \right\|_{W_d} \leq \epsilon_d. \quad (24)$$

Units, causality, conservation, positivity, stability and resource bounds remain domain obligations.

18.2 Adapter register

ID	Domain	Maturity	Qualification boundary
A-01	Graphics/game engines	A2	Direct target; needs engine corpus and independent backend; no rotation/deformation/response claim
A-02	Robotics/public safety	A1	One monitor input only; needs uncertainty envelopes and safety qualification
A-03	Measurement/control	A1	Requires units, calibration, uncertainty, traceability and bandwidth
A-04	SAR/wave sensing	A1	Requires electromagnetic forward model and calibrated detection baseline
A-05	GPU/edge AI	A1	Requires executable model/kernel, quality, memory, latency and energy results

ID	Domain	Maturity	Qualification boundary
A-06	Optical/phononics	A1	Requires a calibrated component transfer function including I/O and drift
A-07	Quantum/Cross-Kerr	A0	Requires Hamiltonian/channel, noise, measurement and resource scaling
A-08	Chemical networks	A1	Requires stoichiometry, rates, positivity and thermodynamic accounting
A-09	Biological/medical	A0	Non-therapeutic; requires mechanism, preclinical evidence, replication and trials
A-10	Fabrication	A1	Fabricated component is not a complete processor; needs metrology/yield/function
A-11	Cosmology/holography	A0	Shared boundary vocabulary is not a bulk-boundary duality

18.3 Graphics and game engines

This is the strongest adapter because the core problem is literally engine CCD. The next qualification step is an exported scene corpus containing scale extremes, shared features, fast projectiles, moving meshes and known failures from a mainstream engine. Compare against an independent implementation under equal tolerances; record misses, false contacts, TOI interval error, candidate count, memory and cost.

18.4 Robotics and public safety

A CCD certificate can constrain a motion planner, but geometry and motion estimates carry uncertainty. A valid adapter expands obstacles/trajectories by calibrated uncertainty, preserves unresolved statuses, and is independently monitored. It is not a decision system about people and cannot be used as a sole basis for rights-affecting action.

18.5 Measurement, sensors and actuators

Continuous event isolation can timestamp a calibrated threshold crossing. It does not eliminate sensor physics. Each quantity needs units, calibration, uncertainty, hysteresis, drift and response model. An SDF-like residual is meaningful only after a fixed, independently defined measurement surface exists.

18.6 SAR and wave sensing

Log-polar indexing can support rotation/scale registration, while swept envelopes can prune motion hypotheses. The adapter must still implement electromagnetic propagation, sampling, phase, noise and calibration. A binary parity output has no physical meaning until the transducer and reference are defined.

18.7 GPU and edge AI

The architecture can become a bounded sparse routing or codec experiment. It must be compared with ordinary quantization, pruning, sparse kernels and smaller models at equal quality. Analytical memory estimates are plans, not hardware results. v0.5's CUDA work concerns CCD semantics only.

18.8 Optical and photonic systems

Optical propagation may approximate selected linear transforms. A first experiment should implement one typed operator, then measure transfer error, calibration drift, bandwidth, dynamic range, latency and total energy including conversion. Nonlinear logic, memory and feedback are not supplied by a diagram.

18.9 Quantum and Cross-Kerr hypotheses

A parity measurement or conditional phase is a specific channel governed by a Hamiltonian, coupling, interaction time, loss and measurement. A symbolic phase shift is not a device. Nothing in the CCD results supports quantum advantage or a biological Cross-Kerr processor.

18.10 Chemical reaction networks

A chemical adapter must preserve stoichiometry, rates, positivity and thermodynamic balances. Reaction-state regions may be monitored by event isolation or a network may implement finite-state logic, but parity requires an explicit chemical encoding/transducer.

18.11 Biological, biophotonic and medical modeling

The material may motivate simulations or signal-processing features. It does not demonstrate that myelin, tubulin, biophotons or vascular branching implement NHDF computation. It does not establish dementia reversal, neural repair, tissue regeneration or any treatment. No clinical use is warranted by this package.

18.12 Fabrication

Swept-volume certificates can assist design-rule checks for curvilinear or 3-D structures. Qualification needs process constraints, coupons, metrology, yield and optical/electrical function. Manufacturable waveguides do not by themselves create sources, detectors, memory, nonlinear control, packaging or a complete processor.

18.13 Cosmology and the holographic principle

A geometric boundary, projection or display hologram is not the holographic principle. Advancing beyond A0 requires two specified theories, a dictionary of observables, matching dynamics/entropy and nontrivial predictions. The current map intentionally prevents a category error.

19 Roadmap from v0.6 to v1.0

19.1 v0.6: rotational and convex foundation

Add support mappings, conservative advancement using translational/angular speed bounds, sphere-capsule/capsule-triangle profiles, a reference BVH and a complete CUDA port. Qualification requires CPU/GPU differential tests, overflow injection and deterministic reduction.

19.2 v0.7: triangle-triangle and deformation

Add vertex-face and edge-edge event isolation for linearly moving vertices, interval root enclosures, filtered/exact predicates, degeneracy taxonomy and deforming triangle support. Self-collision requires duplicate/shared-feature filtering and topological adjacency rules.

19.3 v0.8: dynamics integration

Add reference substepping, persistent contact identities, solver handoff examples, friction/resting-state boundary tests, fault injection and full deterministic replay through one engine adapter.

19.4 v0.9: qualification

Publish a large public corpus, independent implementation comparison, established-library baselines and an externally reviewed safety boundary. Performance claims begin only here, after correctness and equal-budget methodology.

19.5 v1.0: stable profiles

Freeze the supported profile APIs and certificate schema, document cross-backend tolerance, complete independent review and publish all known defects. Version 1.0 would still be an engineering CCD release, not an entire-substrate theory.

20 Reproducibility and release integrity

The accompanying ZIP contains source, tests, synthetic data generators, machine-readable results, figures, schemas, report source, build scripts, logs, source hashes, a software bill of materials and a release SHA-256 manifest. A clean validation sequence is:

```
1 cd nhdf_ccd_v0_5
2 PYTHONPATH=03_reference/python python3 04_verification/run_tests.py
3 bash 04_verification/run_cpp_smoke.sh
4 PYTHONPATH=03_reference/python python3 04_verification/generate_vectors.py
5 PYTHONPATH=03_reference/python python3 05_benchmarks/run_benchmark.py
6 python3 13_report/generate_report_assets.py
7 python3 tools/release_validate.py
```

Generated measurements are tied to the recorded environment. Rebuilding on another machine should preserve statuses and tolerance-bounded results but may change throughput and exact floating serialization. The immutable ZIP is the delivery unit; modifying any contained file invalidates its manifest.

21 Conclusion

NHDF-CCD v0.5 makes a concrete turn from broad symbolism to a narrower executable claim. The useful nucleus is a typed, bounded, causal event-isolation pipeline with conservative candidate generation, analytic feature roots, direct residual checks, explicit failures and reproducible certificates.

For literal graphics/engine CCD, the release now covers translating spheres/points against translating triangle meshes and demonstrates face, edge, vertex, degeneracy, simultaneous-contact, capacity

and tunnelling behavior. It does not yet solve rotating/deforming triangle–triangle CCD or contact dynamics.

The wider materials remain present but are separated into qualification programs. Their scientific standing depends on domain equations and evidence, not on reuse of the CCD vocabulary. This separation is not a retreat from generality; it is the mechanism that makes future generality testable.

A Requirements traceability summary

ID	Requirement	Primary evidence
R-001-002	normalized interval and invalid-input rejection	endpoint, negative radius and NaN tests
R-003-005	face, edge and vertex contact	unit and 1,500 constructed random cases
R-006-008	overlap, relative translation and degeneracy	dedicated unit cases
R-009-011	mesh broad phase, fail-closed capacity and deterministic group	mesh tests and scaling corpus
R-012	versioned witness/residual certificate	JSON schema and vector file
R-013	independent differential comparison	convex-distance oracle corpus
R-014	continuous tunnelling detection	5,000-path benchmark
R-015	cross-language seed	C++17 compile/run log
R-016-017	angular bound and CUDA semantics	unit test plus design-only source
R-018-020	exactness boundary, adapter separation and retirement	policy documents and report

The complete row-level matrix is 01_scope/REQUIREMENTS.csv.

B Certificate example

Listing 4: Abbreviated face-contact certificate.

```

1 {
2   "version": "0.5",
3   "status": "HIT",
4   "authoritative": true,
5   "solver": "analytic-filtered-sphere-triangle",
6   "contacts": [{
7     "toi": 0.5,
8     "toi_interval": [0.4999999999, 0.5000000001],
9     "center": [0.25, 0.25, 0.5],
10    "witness": [0.25, 0.25, 0.0],
11    "normal": [0.0, 0.0, 1.0],
12    "separation": 0.0,
13    "feature": "FACE",
14    "triangle_id": 0,
15    "method": "face-plane-linear",
16    "residual_checked": true
17  }],
18   "candidate_count": 2,
19   "processed_count": 2
20 }
```

The actual deterministic vector file contains all generated fields and multiple feature/status cases.

C Failure decision table

Status	Authority	Required caller behavior
HIT	authoritative within profile	advance conservatively to accepted time; pass contact to response solver
MISS	authoritative within profile	continue step only if all adapter assumptions hold
INITIAL_OVERLAP	authoritative state classification	use overlap/resting/repair policy; do not treat as future TOI
INVALID_INPUT	non-authoritative	correct input or stop; inspect finite/unit/profile contract
UNSUPPORTED	non-authoritative	route to a solver supporting the geometry/motion
INCONCLUSIVE	non-authoritative	retry, subdivide or use independent backend
RESOURCE_EXHAUSTED	non-authoritative	increase capacity, partition safely or stop
NUMERICAL_FAILURE	non-authoritative	rescale, increase precision or use certified backend

D Source and package map

Path	Purpose
00_provenance	supplied sources, attribution and hashes
01_scope	requirements and acceptance boundary
02_formal	typed contract, numerical policy and schemas
03_reference	Python, C++17 and CUDA source
04_verification	tests, deterministic vectors and logs
05_benchmarks	seeded scripts, CSV/JSON results and limitations
06_acceleration	CUDA equivalence and data-layout plan
07_integration	public API and engine handoff
08_operations	telemetry, incident and hazard controls
09_adapters	A0–A5 protocol and eleven separate domain maps
10_maintenance	version/change control and roadmap
11_retirement	end-of-life and archive procedure
12_legacy	immutable v0.4 materials
13_report	this report source and figures
tools	validation, preflight and packaging scripts

E References

References

- [1] T. Moller and B. Trumbore, “Fast, Minimum Storage Ray–Triangle Intersection,” *Journal of Graphics Tools*, 2(1), 1997.

- [2] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2005.
- [3] S. Redon, A. Kheddar and S. Coquillart, “Fast Continuous Collision Detection between Rigid Bodies,” *Computer Graphics Forum*, 21(3), 2002.
- [4] R. Bridson, R. Fedkiw and J. Anderson, “Robust Treatment of Collisions, Contact and Friction for Cloth Animation,” *ACM Transactions on Graphics*, 21(3), 2002.
- [5] D. Harmon, E. Vouga, B. Tamstorf and E. Grinspun, “Robust Treatment of Simultaneous Collisions,” *ACM Transactions on Graphics*, 27(3), 2008.
- [6] J. C. Hart, “Sphere Tracing: A Geometric Method for the Antialiased Ray Tracing of Implicit Surfaces,” *The Visual Computer*, 12, 1996.
- [7] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, 18, 1997.
- [8] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE 754-2019.
- [9] Supplied source S1, initial parity/log-polar/NHDF operator prompt-chain PDF, preserved in the release provenance directory.
- [10] Supplied source S2, *NHDF Formal Specification*, preserved in the release provenance directory.
- [11] Supplied source S3, commute/design transcript, preserved in the release provenance directory.
- [12] Supplied source S4, spherical-Cartesian whole-substrate extension, preserved in the release provenance directory.
- [13] *NHDF-CCD Cradle-to-Grave v0.4*, immutable baseline annex included after the v0.5 report.

Immutable historical annex notice

The final delivered PDF appends the complete v0.4 dossier after this page. The annex is historical evidence, not silently updated text. Where v0.4 and v0.5 conflict, the v0.5 supported profile, typed certificate and failure semantics govern the current release.

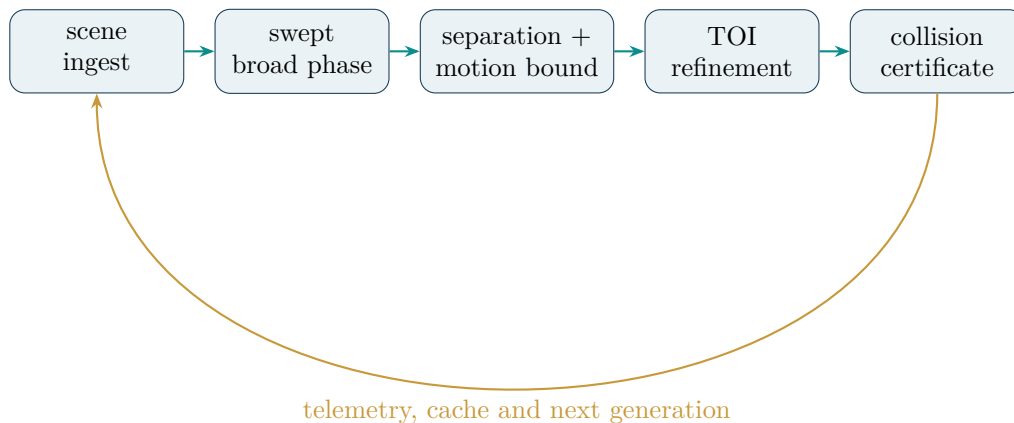
End of v0.5 continuation report.

The v0.4 baseline follows in the merged delivery PDF.

Non-Euclidean Holographic Data Fields

CCD-Centered Formalization

Cradle-to-Grave Validation Dossier, Version 0.4



Release position

This document turns the NHDF roadmap toward one falsifiable center: general-purpose continuous collision detection as an architecture, with an executable primitive reference and explicit pathways to robust mesh, convex, SDF, CPU, GPU, and application implementations. Optical, quantum, chemical, biological, medical, SAR, fabrication, and cosmological material is retained only through separate adapter contracts and evidence gates.

Tom Klootwijk

1 September 2026

Document status: research specification and executable validation scaffold. Not safety-certified, medically validated, cryptographically authoritative, or a claim that physical reality is an NHDF substrate.

Document control

Field	Value
Title	NHDF-CCD: CCD-Centered Formalization and Cradle-to-Grave Validation Dossier
Version	0.4.0
Author attribution	Tom Klootwijk
Date	1 September 2026
Primary center	Continuous Collision Detection (CCD)
Executable status	Python CPU reference; C++17 exact-sphere slice; CUDA source skeleton
Bundled verification	22 Python tests, one C++ smoke test, four reference vectors, 5,000-query generated differential benchmark
Evidence class	Conceptual architecture plus bounded executable scaffold; no production or safety certification
Privacy rule	Civil identifiers and date of birth are not included in executable state or used as cryptographic material

Abstract

The supplied NHDF work develops an ordered vocabulary of log-polar addressing, local zero-level constraints, parity events, bounded branching, forward kinematics, analytic sweeps, projection, and feedback. The earlier formal specification made the essential non-degenerate correction that a global field satisfying $F \equiv 0$ is unusable and that each valid cell must instead lie on a nontrivial local constraint. The commute transcript then connected an SDF lookup and kinematic motion to Continuous Collision Detection, but overstated that connection as a complete one-clock solution. The whole-substrate extension mapped the vocabulary into sensors, actuators, chemistry, biology, medicine, photonics, quantum optics, fabrication, SAR, and cosmology.

Version 0.4 centers the research program on CCD because it supplies a precise mathematical question, exact and conservative reference methods, public benchmark datasets, explicit failure states, and a direct route from CPU semantics to GPU and application integration. A pairwise separation function $d_{ij}(t)$ defines the contact zero set. Exact primitive solvers, conservative advancement, interval inclusion, swept broad-phase bounds, and bounded queues produce a certificate with one of seven states: `HIT`, `NO_HIT`, `INITIAL_OVERLAP`, `INCONCLUSIVE`, `UNSUPPORTED`, `INVALID_INPUT`, or `CAPACITY_EXCEEDED`. The NHDF log-polar, parity, golden-ratio, and topology constructs remain traceable as scheduling and experimental operators, but none may certify or suppress a collision unless an independent equivalence proof is established.

The accompanying package implements exact linear sphere–sphere, sphere–plane, and swept-AABB queries; conditional conservative advancement for declared separation oracles; point/sphere versus a true or conservatively bounded SDF under translation; a deterministic broad phase; interval-refinement ablations; schemas; telemetry; tests; benchmarks; a C++ slice; and a GPU plan. The remaining substrate materials are mapped through a uniform adapter contract that preserves domain-specific state, evolution laws, units, invariants, uncertainty, validation metrics, and stop conditions.

Executive determination

What this release establishes

NHDF can be developed as a bounded, causal operator architecture around CCD. The scientifically defensible nucleus is the ordered closure of local constraints, kinematic prediction, conservative swept bounds, bounded refinement, observable certificates, and feedback telemetry. Version 0.4 makes that nucleus executable for a deliberately small primitive profile and gives every later implementation a testable contract.

What this release does not establish

The release does not solve every possible CCD query, does not implement triangle-mesh or rotating-body robustness, does not include contact response, and does not demonstrate a GPU or physical-substrate advantage. It does not validate medical treatment, biological Cross-Kerr computation, universal golden-ratio branching, storm immunity, zero-latency optics, or cosmological holography. Those claims are either rejected or retained as separately gated hypotheses.

Immediate next gate

The next scientific milestone is not a larger vocabulary. It is robust differential validation against exact time-of-impact corpora and independent implementations for vertex–face and edge–edge triangle queries, followed by CPU/GPU equivalence. Only after that gate should an application claim be made.

Contents

Document control	i
Abstract	ii
Executive determination	iii
I Evidence boundary and architectural turn	1
1 Purpose, scope, and release philosophy	2
1.1 Why CCD becomes the center	2
1.2 Cradle-to-grave means lifecycle, not only algorithm	2
1.3 Authorship and privacy	3
2 Source register and claim disposition	4
2.1 Supplied materials	4
2.2 Four disposition classes	4
2.3 The decisive corrections	5
2.3.1 From global zero to pairwise contact zero	5
2.3.2 From parity as physics to parity as bounded information	5
2.3.3 From automatic logarithmic time dilation to coordinate-aware scheduling	5
2.3.4 From a sign-bit response to a separate dynamics problem	5
3 Research context: what robust CCD actually requires	7
3.1 Broad phase and narrow phase	7
3.2 Families of methods	7
3.3 The reference does not compete with those methods yet	7
II CCD formal core	8
4 Problem statement and outcome semantics	9
4.1 Scene and motion	9
4.2 Seven outcomes	9

4.3	Collision certificate	10
5	Typed, state-threaded operator chain	11
5.1	Why the older composition needed repair	11
5.2	Common state	11
5.3	Operator obligations	12
6	Exact primitive backends	13
6.1	Linear sphere–sphere	13
6.2	Linear sphere–plane	13
6.3	Swept axis-aligned boxes	13
6.4	Why exact primitives matter in a general architecture	13
7	Conservative advancement as the SDF bridge	14
7.1	The useful part of the source intuition	14
7.2	No-skip theorem	14
7.3	SDF specialization	14
7.4	Termination and brackets	15
7.5	Pseudocode	15
8	Broad phase and bounded work	16
8.1	Conservative swept bounds	16
8.2	Capacity is part of semantics	16
8.3	Bounded interval refinement and the BST mapping	16
8.4	Branching ablation rule	16
9	Log-polar, phase, parity, and topology in the CCD profile	17
9.1	Log-polar hint	17
9.2	Second phase difference	17
9.3	Parity firewall	17
9.4	Klein-bottle topology	17
10	Contact output and the boundary with response	18
10.1	Witnesses and normals	18
10.2	Simultaneous contacts	18
10.3	Detection is not response	18
III	Executable reference and verification	20
11	Reference implementation architecture	21
11.1	Package structure	21

11.2	Design principles embodied in code	21
11.3	Exact sphere implementation excerpt	21
11.4	Conservative implementation excerpt	23
11.5	C++ and GPU slices	24
12	Test plan and included results	25
12.1	Test layers	25
12.2	Generated differential benchmark	25
12.3	Interpretation	26
13	Robustness and adversarial validation plan	27
13.1	Mandatory geometry cases	27
13.2	Mandatory software and resource cases	27
13.3	Differential program	27
14	Falsifiable NHDF hypotheses inside CCD	28
14.1	Core hypotheses	28
14.2	Kill criteria	28
IV	Hardware, integration, and lifecycle	29
15	GPU roadmap	30
15.1	Semantics first	30
15.2	Proposed pipeline	30
15.3	Floating-point policy	30
15.4	NHDF scheduling experiments	30
15.5	Acceptance metrics	31
16	Application integration and response boundary	32
16.1	Step protocol	32
16.2	Rollback and safe stop	32
16.3	Public-safety and autonomous-agent boundary	32
17	Cradle-to-grave gates	33
17.1	Gate G0 – provenance and claim boundary	33
17.2	Gate G1 – requirements and hazards	33
17.3	Gate G2 – mathematical contract	33
17.4	Gate G3 – deterministic CPU reference	33
17.5	Gate G4 – robust differential validation	34
17.6	Gate G5 – accelerated backend	34

17.7 Gate G6 – application integration	34
17.8 Gate G7 – deployment and operations	34
17.9 Gate G8 – domain adapters	34
17.10 Gate G9 – maintenance	34
17.11 Gate G10 – retirement	34
18 Operations, incident response, and change control	35
18.1 Runtime telemetry contract	35
18.2 Alarm conditions	35
18.3 Incident package	35
18.4 Change categories	35
V Separated domain adapters	37
19 Uniform adapter contract	38
19.1 Definition	38
19.2 Relationship classes	38
20 Graphics, robotics, measurement, and public safety	40
20.1 Graphics and game engines	40
20.2 Robotics	40
20.3 Measurement, sensors, and actuators	40
20.4 Public safety	40
21 SAR, GPU/AI, and wave sensing	42
21.1 SAR and ionospheric disturbance	42
21.2 GPU CCD	42
21.3 Edge AI	42
22 Optical, photonic, quantum, and Cross-Kerr adapters	43
22.1 Optical operator equivalence	43
22.2 Photonic waveguides and direct writing	43
22.3 Quantum profile	43
22.4 Cross-Kerr boundary	43
23 Chemical, biological, and medical adapters	45
23.1 Chemical reaction networks	45
23.2 Biological modeling	45
23.3 Biophotonics	45
23.4 Medical stop line	46

24 Fabrication and 3D photonics	47
24.1 What CCD contributes literally	47
24.2 What coordinate choice does not remove	47
24.3 Credible cradle-to-device sequence	47
25 Cosmology and the limit of the substrate claim	48
25.1 Holographic word versus holographic principle	48
25.2 Permitted use	48
26 Integrated adapter roadmap	49
26.1 Order of work	49
26.2 Why this preserves the entire project	49
VI Release roadmap, risks, and appendices	50
27 Version roadmap to 1.0	51
27.1 Definition of done for 1.0	51
28 Conclusions	52
A Normative minimum specification	53
B Symbol table	54
C Package manifest overview	55
D Reproduction commands	56
References	57

List of Figures

5.1	State-threaded CCD chain. Dashed NHDF hints are deliberately outside the correctness authority.	11
7.1	Reference conservative advancement toward a unit-sphere SDF. Sample spacing contracts near contact; no free “time dilation” is claimed.	15
10.1	A sphere can cross another sphere while both frame endpoints remain separated. CCD isolates the first contact instead of responding to an already penetrated final state.	19
12.1	The benchmark verifies the included sphere profile and demonstrates the endpoint-tunneling failure. It does not validate mesh or GPU completeness.	26
17.1	Lifecycle gates. Flow runs left-to-right on the first row and right-to-left on the second. An adapter may enter only after the core evidence it depends on exists; retirement remains a designed stage.	33

List of Tables

2.1 Traceability from supplied sources to the v0.4 disposition. 5

4.1 Normative certificate outcomes. 9

12.1 Bundled generated benchmark summary. Timings are machine-specific CPython
measurements and not product claims. 25

14.1 CCD-centered falsifiable hypotheses. 28

18.1 Risk register summary. 36

19.1 Separated domain adapters and evidence gates. 38

27.1 Proposed release sequence. 51

B.1 Principal symbols. 54

C.1 Top-level ZIP layout. 55

Part I

Evidence boundary and architectural turn

Purpose, scope, and release philosophy

1.1 Why CCD becomes the center

A useful research center must expose a clear input, a clear output, a correctness criterion, failure conditions, baselines, and a way to reproduce results. CCD satisfies all six. Given continuous object trajectories over a time interval, the engine must determine whether the objects first touch, identify or bound the earliest time, and avoid the tunneling failure of endpoint-only testing. Robust CCD research also provides exact or analytic datasets, inclusion-based algorithms, open implementations, and known degeneracies against which a new architecture can be tested (B. Wang, Ferguson, Schneider, et al., [2021](#); Belgrod et al., [2023](#)).

The turn toward CCD does not discard the earlier NHDF work. It changes the burden of proof. Terms such as “local zero set,” “forward timeline,” “bounded branch tree,” and “analytic sweep” receive direct geometric meanings. Terms such as parity, golden-ratio scaling, and non-orientable chart flips are kept as optional scheduling or diagnostic variables until an ablation establishes their value. This preserves source traceability without allowing attractive vocabulary to substitute for a collision certificate.

1.2 Cradle-to-grave means lifecycle, not only algorithm

The package treats a collision engine as a lifecycle artifact:

1. establish provenance and claim boundaries;
2. define supported geometry, motion, units, tolerances, and hazards;
3. formalize the mathematics and proof obligations;
4. build a deterministic reference implementation;
5. validate against exact and adversarial cases;
6. port only verified semantics to accelerated hardware;
7. integrate detection with a separate response solver and application policy;
8. deploy with telemetry, incident replay, and explicit safe stops;
9. maintain through controlled changes and regression evidence;
10. retire unsafe or obsolete backends without silent degradation.

Every stage in the ZIP has its own numbered directory. The PDF is therefore not the only product; it is the narrative and formal index to executable and operational artifacts.

1.3 Authorship and privacy

The work is attributed to Tom Klootwijk. Personal civil identifiers and dates of birth do not strengthen a mathematical proof, do not authenticate a runtime, and are not suitable cryptographic keys. The package intentionally omits them from executable state. Release authenticity should instead use repository history, reproducible builds, conventional digital signatures, and published hashes.

Source register and claim disposition

2.1 Supplied materials

The release is grounded in four supplied sources:

- S1 – original operator-chain transcript.** A 30-page progression from parity, lower-case phi, log-polar LUT, kinematics, cone sweep, projections, distributed apex, BST/L-system branching, SDF/Klein-bottle language, and later GPU, optical, quantum, and SAR proposals (Klootwijk, 2026c).
- S2 – Formal Specification v0.1.** A 26-page formalization that separates NHDF representation, PCKF dynamics, and IHCP runtime; replaces the degenerate global zero field with cell-local constraints; distinguishes data and topology parity; bounds recursion; and defines a causal update (Klootwijk, 2026b).
- S3 – commute design transcript.** A 40-page design log that identifies the missing log-polar scale/Jacobian issue and then treats SDF plus kinematics as a claimed CCD solution, including collision-response bit operations and register-level metaphors (Klootwijk, 2026a).
- S4 – spherical-Cartesian whole-substrate extension.** A 70-page extension into measurement, sensors, actuators, chemistry, biology, medicine, photonics, Cross-Kerr logic, microvasculature, lithography, direct writing, SAR, and cosmology (Klootwijk, 2026d).

2.2 Four disposition classes

Class	Meaning	Example
Preserved	The source idea already has a valid typed interpretation.	Monotonic forward time; bounded allocation; local zero-set residual.
Reinterpreted	The source insight is useful but its original mechanism or scope was too broad.	Parity becomes a hint or diagnostic rather than a universal collision governor.
Rejected	The statement conflicts with mathematics, physics, or the evidence available.	One SDF read and one sign-bit flip completely solve general CCD and collision response.
Adapter hypothesis	The idea may define a research program only after domain laws and validation are supplied.	Optical transfer, chemical event fields, biological waveguides, medical applications.

2.3 The decisive corrections

2.3.1 From global zero to pairwise contact zero

A global $F(x) \equiv 0$ has no distance or useful gradient. CCD instead uses a family of nontrivial pairwise separation functions,

$$d_{ij}(t) = \text{sep}(A_i(t), B_j(t)), \quad \mathcal{M}_{ij} = \{t \in [0, 1] : d_{ij}(t) = 0\}. \quad (2.1)$$

Positive separation means the pair is disjoint under the backend’s convention; zero denotes contact; negative values denote overlap when a signed measure exists. This is the CCD-specific realization of distributed local zero-set semantics.

2.3.2 From parity as physics to parity as bounded information

A bitwise parity flag detects an odd count of bit inversions under a reference comparison and misses every even count. In a 2022 CCD paper, “root parity” is a precise mathematical construction for cubic roots, yet the authors explicitly note that parity cannot distinguish zero roots from the rare two-root case (B. Wang, Ferguson, Jiang, et al., 2022). This is particularly instructive: parity can be valuable in CCD, but only with a clearly defined object and a known ambiguity. Version 0.4 therefore stores parity-derived scheduling metadata but never treats it as a complete collision predicate.

2.3.3 From automatic logarithmic time dilation to coordinate-aware scheduling

Log-polar coordinates can compact scale and convert rotation/scale relationships into shifts, but they do not increase physical sampling rate for free. The Jacobian and metric must be respected, and the underlying separation must remain coordinate invariant. Version 0.4 uses log-polar bins to group or prioritize work; it does not let a coordinate transform change whether two bodies collide.

2.3.4 From a sign-bit response to a separate dynamics problem

Collision detection answers when and where contact occurs. Response must use mass, inertia, normal, friction, restitution, constraints, compliance, and material failure. A sign inversion can be a toy elastic response in one dimension, not a general physical solver. The architecture therefore ends CCD at a certificate and hands that certificate to a separate response layer.

Table 2.1: Traceability from supplied sources to the v0.4 disposition.

Source	Source idea	Disposition	v0.4 destination
S1 pp.1-6	Parity-conditioned self-referential SDF engine	Reinterpreted	Local pair separation + bounded causal state; parity is diagnostic
S1 pp.8-11	Operator precedence chain	Preserved with typing	State-threaded CCD chain and explicit address/candidate resolution
S1 pp.14-15	Parity oscillation under disturbance	Preserved as failure signal	Debounce/telemetry/safe stop; no correction claim
S1 pp.16-18	SDF zero in every LUT cell	Corrected	Family of nontrivial local constraints; in CCD, $d_{ij}(t)=0$
S1 pp.19-22	Optical/quantum substrate	Separated	Operator-equivalence adapter only
S1 pp.22-30	GPU/Tensor/VRAM claims	Corrected	Normal exposed operations, bounded queues, CPU equivalence first

Source	Source idea	Disposition	v0.4 destination
S2 Sec.2-5	Non-degenerate local zero sets	Adopted	Foundation for pairwise contact manifolds
S2 Sec.6-8	Forward timeline, phase, bounded branching	Adopted with limitation	Causal TOI and bounded refinement; phase/golden are heuristic
S2 Sec.12	GPU reference schedule	Extended	Broad/narrow phase, compaction, certificate emission
S3 pp.2-4	Missing log-polar Jacobian	Preserved as coordinate warning	Coordinates may schedule work; geometry remains Cartesian/invariant
S3 pp.14-17	SDF+kinematics completely solves CCD in one clock	Rejected	General CCD needs broad phase, robust narrow phase, TOI isolation, degeneracy handling, certificates
S3 pp.29-30	Compare step magnitude with SDF	Partially adopted	Conditional conservative advancement with valid distance and speed bounds
S3 pp.31-36	Autonomous/public-safety use	Restricted	Collision avoidance only, with governance; no identity or force decisions
S4 pp.12-18	Universal sensors/actuators substrate	Separated	Measurement/actuation adapter with units and calibrated transfer laws
S4 pp.22-30	Chemical, biological, macro, SAR, and cosmic maps	Separated	Independent adapters; no ontological unification
S4 pp.33-48	Medical, microvascular, biophotonic, cross-Kerr claims	Rejected as therapeutic mechanism	Simulation/research hypotheses only
S4 pp.49-70	Lithography, GDSII, 3D photonics	Separated	Fabrication adapter with PDK/DR-C/metrology gates

Research context: what robust CCD actually requires

3.1 Broad phase and narrow phase

Modern CCD systems separate a conservative broad phase from an accurate narrow phase. A false-negative broad phase is fatal because the narrow phase never sees the missed pair. Exact time-of-impact datasets have shown that implementation details can invalidate methods that appear correct on paper, and that broad and narrow phases must be evaluated together (Belgrod et al., 2023). The v0.4 scaffold therefore treats candidate capacity and unsupported backends as visible states rather than silently dropping work.

3.2 Families of methods

The roadmap is backend-plural rather than method-monolithic:

- **Closed-form primitive solvers** for spheres, planes, capsules, and translational boxes where stable roots or slabs are available.
- **Conservative advancement** for convex or implicit geometry when a trustworthy distance and motion bound exist (Mirtich, 1996; Zhang et al., 2006).
- **Interval and inclusion methods** for robust polynomial or parametric-surface queries (B. Wang, Ferguson, Schneider, et al., 2021; Chen et al., 2024).
- **Feature-based mesh CCD** using vertex-face and edge-edge tests, exact predicates, degeneracy handling, and adjacency filters.
- **Contact-integrated methods** such as IPC and its codimensional/rigid variants, which depend on collision queries but also solve a larger time-stepping problem (Li et al., 2021; Ferguson et al., 2021).
- **SDF-specific methods.** Recent work treats triangle-SDF contact directly and, in 2026, SDF-SDF time of impact through alternating spatial-temporal optimization (Pelletier-Guénette et al., 2025; Nielsen and Erleben, 2026).
- **Convex cone/ray casting and hardware acceleration.** C5D and GPU ray-tracing research illustrate active directions without implying that one representation dominates every workload (Yuan et al., 2025; Sui et al., 2024).

3.3 The reference does not compete with those methods yet

The bundled Python implementation is intentionally modest. It validates the state machine, certificate semantics, conservative-advance proof obligation, failure preservation, deterministic traces, and lifecycle tooling. It is not presented as a replacement for Tight Inclusion, IPC Toolkit, or a production physics engine. Those projects supply the correct next comparison points; IPC Toolkit itself exposes broad- and narrow-phase CCD for linear and nonlinear trajectories and explicitly states that it is not a complete physics solver (IPC-Sim Project, 2026).

Part II

CCD formal core

Problem statement and outcome semantics

4.1 Scene and motion

Let a scene contain bodies $B_i = (S_i, M_i)$, where S_i is a geometric shape and M_i maps normalized step time $t \in [0, 1]$ to a configuration. The normalized interval avoids binding the mathematics to a particular frame duration; physical time is recovered by $\tau = \tau_n + t\Delta\tau_n$.

For a pair (i, j) , a narrow-phase backend provides or derives:

1. a separation or inclusion object $d_{ij}(t)$;
2. witness features or points when meaningful;
3. a contact normal or normal set when meaningful;
4. a conservative bound on change over an interval;
5. a procedure for isolating the earliest contact.

The first time of impact is

$$t_{\text{impact}} = \inf\{t \in [0, 1] : d_{ij}(t) \leq 0\}, \quad (4.1)$$

with conventions adjusted for a minimum separation or thickness. If the pair overlaps at $t = 0$, the query is not reported as an ordinary future hit.

4.2 Seven outcomes

Table 4.1: Normative certificate outcomes.

Status	Meaning	Caller permission and obligation
HIT	Contact is found within the interval. Exact backends return a point; iterative backends return a conservative bracket.	Advance no later than the certified upper TOI, group simultaneous contacts, and invoke response.
NO_HIT	The supported backend certifies positive separation throughout the interval.	The caller may advance through the step for that pair.
INITIAL_OVERLAP	Separation is negative at the start.	Roll back, depenetrate, or invoke a dedicated overlap solver. Do not pretend the overlap began later.
INCONCLUSIVE	A possible contact could not be safely classified within numerical or resource bounds.	Reduce the step, increase precision/budget, or switch backend.

Status	Meaning	Caller permission and obligation
UNSUPPORTED	No declared backend covers the shape/motion pair.	Route to another backend or reject the scene.
INVALID_INPUT	A value, tolerance, bound, or oracle contract is invalid.	Quarantine the query and fail safe.
CAPACITY_EXCEEDED	A bounded pool or queue could not contain required work.	Preserve the required count and reconfigure; never reinterpret as no-hit.

4.3 Collision certificate

A certificate is the minimum auditable output:

$$\mathcal{C}_{ij} = (s, [t^-, t^+], \mathcal{W}, n, b, k, \mathcal{T}, h), \quad (4.2)$$

where s is status, $[t^-, t^+]$ is the TOI bracket, $\mathcal{W}$ contains witnesses, n is a normal, b names the backend, k is iteration count, $\mathcal{T}$ is bounded telemetry, and h is a deterministic digest. The digest is for replay integrity, not a replacement for a signed release or evidence chain.

Typed, state-threaded operator chain

5.1 Why the older composition needed repair

The v0.1 chain correctly ordered log-polar encoding, local boundary projection, parity, routing, kinematics, sweep, projection, and update. Its written function signatures, however, did not literally compose: an address is not a candidate cell, a bit is not a branch tree, and a tree is not a trajectory. Version 0.4 threads one enriched state through each stage.

5.2 Common state

$$\Sigma = (\mathcal{S}, I_t, \mathcal{B}, \mathcal{P}, \mathcal{O}, \mathcal{Q}, \mathcal{C}, \mathcal{H}, \mathcal{F}), \quad (5.1)$$

where $\mathcal{S}$ is the scene, I_t the active time interval, $\mathcal{B}$ swept bounds, $\mathcal{P}$ candidate pairs, $\mathcal{O}$ separation oracles, $\mathcal{Q}$ bounded refinement queues, $\mathcal{C}$ certificates, $\mathcal{H}$ telemetry/history, and $\mathcal{F}$ failures.

Every operator has type $O_k : \Sigma \rightarrow \Sigma$. The normative chain is

$$\Sigma' = U \circ \Pi_{\text{cert}} \circ R_{\text{refine}} \circ A_{\text{CA}} \circ B_{\text{bound}} \circ C_{\text{pair}} \circ E_{\text{scene}}(\Sigma) \quad (5.2)$$

with optional source-traceable scheduling metadata H_{NHDF} that may modify priority but not classification.

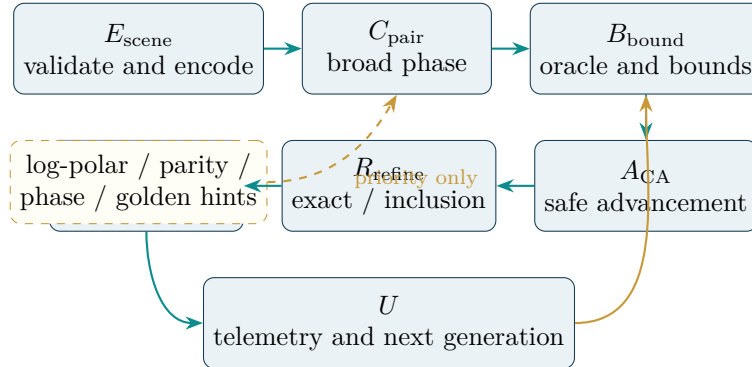


Figure 5.1: State-threaded CCD chain. Dashed NHDF hints are deliberately outside the correctness authority.

5.3 Operator obligations

Operator	Must produce	Must never do
E_{scene}	Valid units, finite values, normalized time, declared shape and motion profiles.	Silently clamp invalid values into apparently valid geometry.
C_{pair}	Conservative candidates and explicit overflow.	Prune on an unverified learned, parity, or log-polar heuristic.
B_{bound}	A backend contract: distance/inclusion object, witnesses, and valid motion/error bounds.	Call an approximate field a signed distance without a conservative error envelope.
A_{CA}	Certified no-skip time advances or an explicit stall/failure.	Force a minimum step larger than the safe bound.
R_{refine}	Earliest-contact isolation under exact, interval, or conservative semantics.	Convert budget exhaustion into no-hit.
Π_{cert}	Complete status, bracket, backend, tolerance, trace, and digest.	Hide initial overlap or unsupported geometry.
U	Deterministic state/telemetry update.	Change a geometric result after certification.

Exact primitive backends

6.1 Linear sphere–sphere

Let $p = c_B(0) - c_A(0)$, $v = v_B - v_A$, and $R = r_A + r_B$. Contact solves

$$\|p + tv\|^2 = R^2, \quad (6.1)$$

which yields $at^2 + bt + c = 0$ with

$$a = v \cdot v, \quad b = 2p \cdot v, \quad c = p \cdot p - R^2. \quad (6.2)$$

The earliest root in $[0, 1]$ is the TOI. The implementation checks initial overlap before root solving and uses a stable quadratic formulation. This exact backend is the main ground truth for the bundled randomized benchmark.

6.2 Linear sphere–plane

For a one-sided plane $n \cdot x = o$ and a sphere center $c(t)$,

$$d(t) = n \cdot c(t) - o - r. \quad (6.3)$$

Under linear relative translation, d is linear. A negative initial value is overlap; a nonnegative derivative while $d(0) > 0$ is a no-hit certificate; otherwise $t = -d(0)/\dot{d}$.

6.3 Swept axis-aligned boxes

For each axis, relative box centers and summed half-extents produce a one-dimensional overlap interval. The boxes collide if and only if the three slab intervals overlap in time. The maximum entry time is the TOI and identifies an entry normal. The backend is exact only while the boxes remain axis-aligned and motion is translational linear.

6.4 Why exact primitives matter in a general architecture

A general engine should not force every query through one iterative method. Closed-form or slab solutions are cheaper, easier to validate, and excellent differential oracles. Version 0.4 therefore dispatches by typed pair and motion profile before using conservative advancement.

Conservative advancement as the SDF bridge

7.1 The useful part of the source intuition

The commute transcript’s central CCD intuition is recognizable: if the current distance is trustworthy and the maximum rate of closure is bounded, the engine may advance by a fraction of distance divided by speed without passing through contact. This is conservative advancement, not a new one-clock law (Mirtich, 1996; Zhang et al., 2006).

7.2 No-skip theorem

Assume $d : [a, b] \rightarrow \mathbb{R}$ is continuous and L -Lipschitz,

$$|d(t) - d(s)| \leq L|t - s|, \quad (7.1)$$

where $d(t) > 0$ denotes separation. At time t_k , choose

$$h_k = \eta \frac{d(t_k)}{L_k}, \quad 0 < \eta \leq 1, \quad (7.2)$$

with L_k valid on the interval to be covered. For any $0 \leq s < h_k$,

$$d(t_k + s) \geq d(t_k) - L_k s > d(t_k)(1 - \eta) \geq 0. \quad (7.3)$$

Thus the open interval is collision-free. Repeated safe advances cannot skip a first contact, provided the distance is not overestimated and the speed bound is not underestimated.

The two contracts that cannot be waived

A sampled texture, neural implicit field, approximate Euclidean distance transform, or interpolated SDF may overestimate true separation. A velocity bound that ignores rotation, deformation, acceleration, or calibration error may underestimate closure. Either violation destroys the no-skip guarantee. In that case the engine must add a conservative bias/error bound or return `UNSUPPORTED/INCONCLUSIVE`.

7.3 SDF specialization

For a true signed distance F with spatial Lipschitz constant 1 and a moving point $x(t)$ with $\|\dot{x}\| \leq V$, the composed separation $d(t) = F(x(t)) - r$ is V -Lipschitz. If the field moves, a conservative relative-speed bound is used. If the field’s Lipschitz bound is K , then $L \leq KV_{\text{rel}}$. The executable reference supports point/sphere versus an implicit field under translation through exactly this contract.

7.4 Termination and brackets

The reference stops when distance is within a declared tolerance and returns the previous and current sample as a conservative contact bracket. If the safe step falls below temporal resolution before the distance tolerance is reached, the result is **INCONCLUSIVE**; it does not force a larger step. If the current distance minus the speed bound times remaining time stays positive, the algorithm returns a no-hit certificate for the rest of the interval.

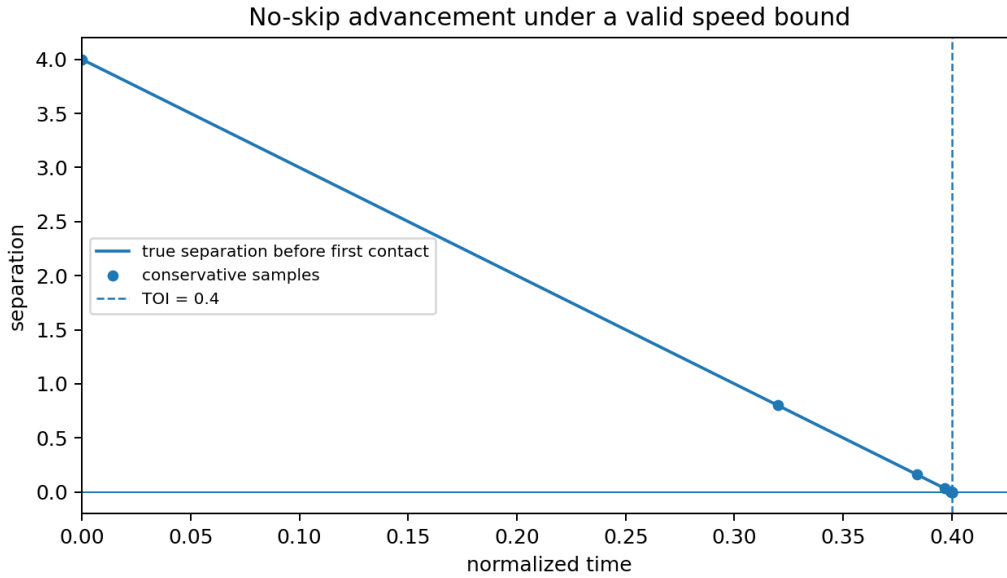


Figure 7.1: Reference conservative advancement toward a unit-sphere SDF. Sample spacing contracts near contact; no free “time dilation” is claimed.

7.5 Pseudocode

Listing 7.1: Semantics of conservative advancement in v0.4.

```

1 sample = oracle.evaluate(t)
2 if sample.distance < -distance_tolerance:
3     return INITIAL_OVERLAP
4 if sample.distance <= distance_tolerance:
5     return HIT at t
6
7 while budget remains:
8     L = oracle.closure_speed_bound(t, t_end)
9     if L <= speed_epsilon:
10        return NO_HIT
11    if sample.distance - L * (t_end - t) > 0:
12        return NO_HIT with remaining-interval certificate
13    h = safety_factor * sample.distance / L
14    if h <= time_tolerance:
15        return INCONCLUSIVE
16    previous_t = t
17    t = t + h
18    sample = oracle.evaluate(t)
19    if sample.distance <= distance_tolerance:
20        return HIT bracket [previous_t, t]
21 return INCONCLUSIVE

```

Broad phase and bounded work

8.1 Conservative swept bounds

For a body with translational motion, each coordinate is bounded over the step, including a quadratic extremum where velocity crosses zero. Shape extents expand that center interval into a swept AABB. A sweep-and-prune pass sorts by minimum x , maintains an active list, and tests overlap on all axes. Infinite planes are conservatively paired with bounded bodies.

8.2 Capacity is part of semantics

A finite machine has finite candidates, nodes, trace records, and output capacity. Let N_c be generated candidates and $N_c^{\max}$ the configured capacity. The invariant is not “overflow cannot happen”; it is

$$N_c \leq N_c^{\max} \quad \text{or the result is CAPACITY_EXCEEDED.} \quad (8.1)$$

A silent truncation would be indistinguishable from a false-negative broad phase.

8.3 Bounded interval refinement and the BST mapping

The source BST/L-system becomes a bounded earliest-first refinement tree. For interval $I = [a, b]$, sample the midpoint m and use a valid Lipschitz bound L_I :

$$\underline{d}(I) = d(m) - \frac{L_I}{2}(b - a). \quad (8.2)$$

If $\underline{d}(I) > 0$, the interval is collision-free. Otherwise it is sampled or subdivided until contact, certification, time tolerance, or node capacity. The split may be midpoint or a golden-ratio point as an experiment. The lower bound, not the aesthetic of the tree, supplies correctness.

8.4 Branching ablation rule

The golden-ratio policy **MUST** be compared with midpoint, adaptive, and backend-specific policies at equal node budget. If it does not improve a predeclared metric, it remains a source-traceable curiosity rather than a justified operator.

Log-polar, phase, parity, and topology in the CCD profile

9.1 Log-polar hint

The reference computes scheduling metadata from nonnegative separation and relative approach direction,

$$\rho = \log(1 + d), \quad \theta = \text{atan2}(v_y, v_x), \quad (9.1)$$

quantizes (ρ, θ, t) , packs a word, and records its bit parity. A priority such as $(d + \epsilon)^{-1}$ can place near contacts earlier in a work queue. The hint can improve coherence or load balance; it cannot delete a pair or certify no-hit.

9.2 Second phase difference

For a directional phase sequence ϕ_n , the source's $\Delta\Delta\phi$ becomes

$$\Delta^2\phi_n = \text{wrap}(\phi_n - 2\phi_{n-1} + \phi_{n-2}). \quad (9.2)$$

It may predict rapidly changing approach direction and request a stronger backend or smaller global step. It is not a bound unless a separate derivation relates it to motion.

9.3 Parity firewall

Normative parity firewall

Enabling or disabling NHDF parity/log-polar hints must not change the supported collision classification or certificate validity. It may change query order, iteration budget escalation, diagnostics, or performance. The bundled test suite checks classification invariance for the SDF example.

9.4 Klein-bottle topology

A Klein-bottle quotient can model non-orientable chart transitions in a specialized geometry system, but ordinary Euclidean collision detection does not require it. If a simulation explicitly uses a non-orientable manifold, chart metadata must be handled by the geometry backend. An orientation flip is not equivalent to a payload error, collision, or response impulse.

Contact output and the boundary with response

10.1 Witnesses and normals

A hit certificate may carry points w_A, w_B and a normal n . For sphere–sphere contact, n follows the center difference and the witnesses lie at radii along the normal. For implicit fields, the gradient supplies a local normal only if it is well-conditioned and consistent with the field’s sign convention. Degenerate or multiple contacts may require a normal set or contact manifold rather than one vector.

10.2 Simultaneous contacts

Let t_* be the earliest upper TOI across pairs. A response layer should group all contacts satisfying

$$t_{ij}^- \leq t_* + \tau_{\text{group}} \quad (10.1)$$

under a declared tolerance. Processing one pair and advancing past another simultaneous contact can create order-dependent penetrations.

10.3 Detection is not response

A frictionless point-mass reflection may use

$$v^+ = v^- - (1 + e)(v^- \cdot n)n, \quad (10.2)$$

for restitution e , but rigid bodies require angular momentum and effective mass; friction needs tangential constraints; resting contact and stacking need persistent constraints or potentials; deformable bodies need constitutive models. CCD supplies the event and geometry, not these laws.

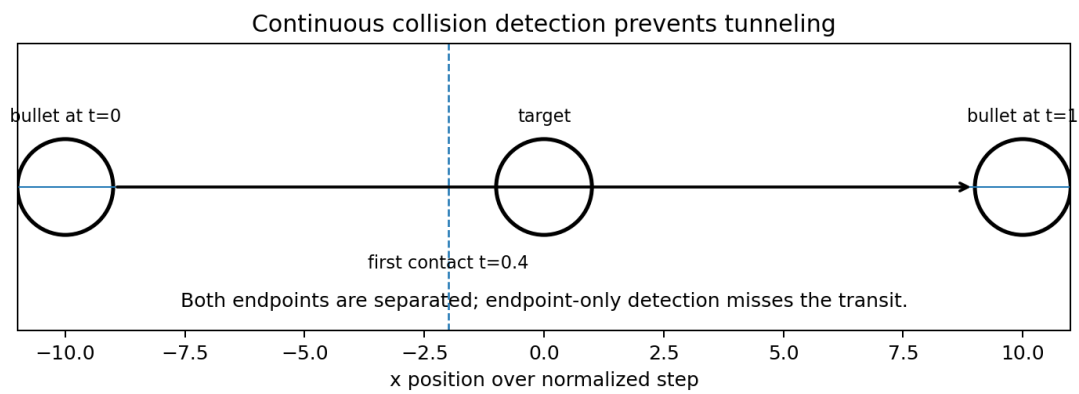


Figure 10.1: A sphere can cross another sphere while both frame endpoints remain separated. CCD isolates the first contact instead of responding to an already penetrated final state.

Part III

Executable reference and verification

Reference implementation architecture

11.1 Package structure

Module	Responsibility
<code>vector.py</code>	Minimal deterministic three-dimensional vector operations.
<code>motion.py</code>	Static, linear, and quadratic translational motion with speed and coordinate bounds.
<code>shapes.py</code>	Sphere, axis-aligned box, plane, point, and implicit-SDF types.
<code>oracles.py</code>	Separation samples, witnesses, normals, and closure-speed bounds.
<code>exact.py</code>	Exact linear primitive backends.
<code>conservative.py</code>	Contract-driven conservative advancement.
<code>interval.py</code>	Bounded midpoint/golden interval-refinement ablation.
<code>broadphase.py</code>	Swept bounds and deterministic sweep-and-prune.
<code>nhdf.py</code>	Log-polar, parity, and phase scheduling metadata outside the safety path.
<code>engine.py</code>	Typed dispatch and scene-level status reduction.
<code>types.py</code>	Configuration, status, samples, certificates, scene results, and trace hashes.
<code>io.py/cli.py</code>	Serialized scene loading and command-line execution.

11.2 Design principles embodied in code

1. **Dispatch before iteration.** Exact profiles are selected before generic advancement.
2. **Contracts are visible.** SDF Lipschitz and conservative-bias values are explicit fields.
3. **No hidden allocation.** Candidate capacity and trace capacity are bounded.
4. **No hidden fallback.** Unsupported pairs remain `UNSUPPORTED`.
5. **Deterministic replay.** A canonical JSON trace is hashed with SHA-256.
6. **Hints cannot decide.** NHDF metadata is attached after sampling and does not change the geometry path.

11.3 Exact sphere implementation excerpt

Listing 11.1: Exact linear sphere–sphere backend from the bundled Python reference.

```

1
2 def exact_sphere_sphere(body_a: Body, body_b: Body, config: CCDConfig) -> CollisionCertificate |
   None:
3     if not (isinstance(body_a.shape, Sphere) and isinstance(body_b.shape, Sphere)):
4         return None
5     va = _linear_velocity(body_a)

```

```

6     vb = _linear_velocity(body_b)
7     if va is None or vb is None:
8         return None
9     oracle = SphereSphereOracle(body_a, body_b)
10    s0 = oracle.evaluate(0.0)
11    pair = (body_a.body_id, body_b.body_id)
12    if s0.distance < -config.distance_tolerance:
13        return CollisionCertificate(CCDStatus.INITIAL_OVERLAP, pair, "exact_sphere_sphere_linear",
14                                   0.0, 0.0, s0, reason="negative initial separation").finalize()
15    if s0.distance <= config.distance_tolerance:
16        return CollisionCertificate(CCDStatus.HIT, pair, "exact_sphere_sphere_linear", 0.0, 0.0,
17                                   s0, reason="initial contact within tolerance").finalize()
18
19    p = body_b.motion.position(0.0) - body_a.motion.position(0.0)
20    v = vb - va
21    radius = body_a.shape.radius + body_b.shape.radius
22    a = v.dot(v)
23    b = 2.0 * p.dot(v)
24    c = p.dot(p) - radius * radius
25    if a <= config.speed_epsilon * config.speed_epsilon:
26        return CollisionCertificate(CCDStatus.NO_HIT, pair, "exact_sphere_sphere_linear", reason=
27                                   "zero relative speed and positive separation", sample=s0).finalize()
28    disc = b * b - 4.0 * a * c
29    if disc < 0.0:
30        return CollisionCertificate(CCDStatus.NO_HIT, pair, "exact_sphere_sphere_linear", reason=
31                                   "quadratic has no real roots", sample=s0).finalize()
32    sqrt_disc = math.sqrt(max(0.0, disc))
33    q = -0.5 * (b + math.copysign(sqrt_disc, b))
34    roots: list[float]
35    if abs(q) <= 1e-30:
36        roots = [-b / (2.0 * a)]
37    else:
38        roots = [q / a, c / q]
39    valid = sorted(t for t in roots if -config.time_tolerance <= t <= 1.0 + config.time_tolerance)
40    if not valid:
41        return CollisionCertificate(CCDStatus.NO_HIT, pair, "exact_sphere_sphere_linear", reason=
42                                   "roots lie outside the step", sample=s0).finalize()
43    toi = min(max(valid[0], 0.0), 1.0)
44    sample = oracle.evaluate(toi)
45    return CollisionCertificate(CCDStatus.HIT, pair, "exact_sphere_sphere_linear", toi, toi,
46                               sample, iterations=1, reason="earliest quadratic root").finalize()
47
48 def exact_sphere_plane(body_a: Body, body_b: Body, config: CCDConfig) -> CollisionCertificate |
49     None:
50     reverse = False
51     sphere_body, plane_body = body_a, body_b
52     if isinstance(body_a.shape, Plane) and isinstance(body_b.shape, Sphere):
53         reverse = True
54         sphere_body, plane_body = body_b, body_a
55     if not (isinstance(sphere_body.shape, Sphere) and isinstance(plane_body.shape, Plane)):
56         return None
57     vs = _linear_velocity(sphere_body)
58     vp = _linear_velocity(plane_body)
59     if vs is None or vp is None:
60         return None
61     oracle = SpherePlaneOracle(sphere_body, plane_body)
62     s0 = oracle.evaluate(0.0)
63     pair = (body_a.body_id, body_b.body_id)

```

11.4 Conservative implementation excerpt

Listing 11.2: Beginning of the conservative-advancement implementation, including explicit stall and invalid-bound behavior.

```

1
2
3 def conservative_advancement(
4     oracle: SeparationOracle,
5     config: CCDConfig,
6     *,
7     t0: float = 0.0,
8     t1: float = 1.0,
9 ) -> CollisionCertificate:
10     """Certified time advancement under an oracle-supplied Lipschitz speed bound.
11
12     Safety is conditional on two contracts:
13     1. 'evaluate(t).distance' does not overstate true separation; and
14     2. 'closure_speed_bound(a,b)' upper-bounds the absolute rate of change
15        of that separation over the interval.
16     """
17     config.validate()
18     pair = (oracle.body_a.body_id, oracle.body_b.body_id)
19     backend = "conservative_advancement"
20     trace: list[dict[str, float | int]] = []
21     t = t0
22     previous_t = t0
23     sample = oracle.evaluate(t)
24     if not math.isfinite(sample.distance):
25         return CollisionCertificate(CCDStatus.INVALID_INPUT, pair, backend, reason="non-finite
26             initial separation").finalize()
27     if sample.distance < -config.distance_tolerance:
28         return CollisionCertificate(CCDStatus.INITIAL_OVERLAP, pair, backend, t, t, sample,
29             reason="negative initial separation").finalize()
30     if sample.distance <= config.distance_tolerance:
31         return CollisionCertificate(CCDStatus.HIT, pair, backend, t, t, sample, reason="initial
32             contact within tolerance").finalize()
33
34     metadata: dict[str, object] = {}
35     for iteration in range(1, config.max_iterations + 1):
36         speed_bound = float(oracle.closure_speed_bound(t, t1))
37         if not math.isfinite(speed_bound) or speed_bound < 0.0:
38             return CollisionCertificate(CCDStatus.INVALID_INPUT, pair, backend, iterations=
39                 iteration, reason="invalid closure-speed bound", trace=trace).finalize()
40         if config.use_nhdf_hints:
41             metadata["nhdf_hint"] = nhdf_hint_for_sample(sample, oracle, t).to_dict()
42         if speed_bound <= config.speed_epsilon:
43             return CollisionCertificate(
44                 CCDStatus.NO_HIT,
45                 pair,
46                 backend,
47                 sample=sample,
48                 iterations=iteration,
49                 reason="positive separation with zero closure-speed bound",
50                 trace=trace,
51                 metadata=metadata,
52             ).finalize()
53
54     remaining = t1 - t
55     safe_step = config.safety_factor * sample.distance / speed_bound
56     no_hit_margin = sample.distance - speed_bound * remaining
57     trace_item = {
58         "iteration": iteration,

```



```

55         "t": t,
56         "distance": sample.distance,
57         "speed_bound": speed_bound,
58         "safe_step": safe_step,
59         "remaining": remaining,
60         "no_hit_margin": no_hit_margin,
61     }
62     if len(trace) < config.max_trace_steps:
63         trace.append(trace_item)
64
65     if no_hit_margin > 0.0:
66         return CollisionCertificate(
67             CCDStatus.NO_HIT,
68             pair,
69             backend,
70             sample=sample,
71             iterations=iteration,
72             reason="Lipschitz no-hit certificate covers the remaining interval",
73             trace=trace,
74             metadata=metadata,
75         ).finalize()
76
77     if safe_step <= config.time_tolerance:
78         # The query is closer than the declared temporal resolution. Returning
79         # INCONCLUSIVE is safer than silently stepping past a possible contact.
80         return CollisionCertificate(
81             CCDStatus.INCONCLUSIVE,
82             pair,
83             backend,
84             toi_lower=max(previous_t, t0),
85             toi_upper=min(t + config.time_tolerance, t1),
86             sample=sample,
87             iterations=iteration,
88             reason="advancement stalled at temporal resolution before contact tolerance",

```

11.5 C++ and GPU slices

The C++17 directory implements the exact sphere–sphere query and builds without external dependencies. The CUDA directory contains a source-only batched exact sphere kernel. It was not compiled in the release environment because no CUDA compiler or target GPU was available. The report therefore makes no CUDA performance or equivalence claim.

Test plan and included results

12.1 Test layers

The package contains 22 Python tests spanning exact roots, high-speed tunneling, no-hit, initial overlap, AABB slabs, true-SDF conservative advancement, sphere radius against an SDF, quadratic translation, invalid Lipschitz values, hint invariance, interval split policies, broad-phase pruning, capacity failure, unsupported pairs, deterministic digests, and invalid configuration. A C++ smoke test independently checks the sphere–sphere TOI.

12.2 Generated differential benchmark

The reproducible benchmark generates 5,000 linear sphere–sphere queries with a fixed seed. The exact quadratic backend is treated as ground truth and compared with the conservative-advancement oracle. It also evaluates a deliberately weak detector that checks only $t = 0$ and $t = 1$.

Table 12.1: Bundled generated benchmark summary. Timings are machine-specific CPython measurements and not product claims.

Metric	Result
Queries	5,000
Exact hits	2,502
Hits missed by endpoint-only discrete check	2,500
Conservative false negatives	0
Conservative false positives	0
Conservative inconclusive	0
Mean / maximum CA iterations	7.332 / 43
Mean TOI upper error	3.362e-10
Maximum TOI upper error	1.789e-08
Exact / CA queries per second	64,173 / 13,796

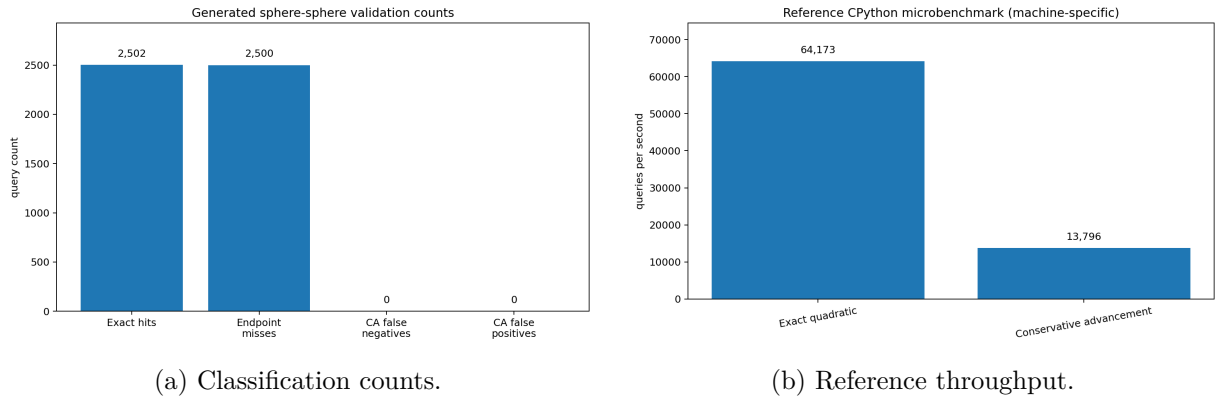


Figure 12.1: The benchmark verifies the included sphere profile and demonstrates the endpoint-tunneling failure. It does not validate mesh or GPU completeness.

12.3 Interpretation

The zero false-negative result is evidence that the implementation matches its exact oracle on this generated distribution. It is not a statistical proof over all inputs. The next gate must use exact corpora containing millions of realistic and near-degenerate feature queries, as advocated by the CCD benchmark literature (B. Wang, Ferguson, Schneider, et al., 2021; Belgrod et al., 2023).

Robustness and adversarial validation plan

13.1 Mandatory geometry cases

1. tangential contact and repeated roots;
2. near-parallel edge–edge and near-coplanar vertex–face motion;
3. contact at exactly $t = 0$ or $t = 1$;
4. zero-area triangles, duplicate vertices, and zero-length edges;
5. coincident features and ambiguous normals;
6. scale extremes and mixed units;
7. rapidly rotating shapes whose closest feature changes;
8. accelerated and nonlinear trajectories;
9. adjacency and self-collision filters;
10. multiple simultaneous impacts.

13.2 Mandatory software and resource cases

1. NaN, infinity, signed zero, overflow, underflow, and denormal behavior;
2. compiler and optimization differences;
3. deterministic tie ordering;
4. candidate, queue, trace, and output exhaustion;
5. stale generation reads and ring-buffer wrap;
6. CPU/GPU disagreement;
7. corrupted serialized scenes and invalid schemas;
8. approximate SDFs with known positive and negative error.

13.3 Differential program

Version 0.5 should ingest public exact TOI data and compare at least three independent paths: an exact/inclusion reference, the NHDF-CCD backend under test, and a second established library. It should report false negatives, false positives, TOI interval width, runtime distribution, memory, and failure status by case family. Test results should remain linked to release and compiler hashes.

Falsifiable NHDF hypotheses inside CCD

14.1 Core hypotheses

Table 14.1: CCD-centered falsifiable hypotheses.

ID	Hypothesis	Required comparison
H-CCD-1	Log-polar bins reduce divergence or improve cache/work-queue locality without changing classification.	Equal queries, equal backend, linear/Cartesian or distance-only bucketing.
H-CCD-2	A parity-derived anomaly signal predicts queries that require escalation.	Threshold, residual magnitude, random bit, and no-hint baselines; ROC and cost.
H-CCD-3	$\Delta^2\phi$ improves motion-bound selection for rapidly changing approach direction.	Actual orientation/curvature bounds and fixed conservative bounds.
H-CCD-4	Golden-ratio interval splitting reduces nodes or time for a declared workload.	Midpoint, secant, adaptive, and earliest-first strategies at equal correctness.
H-CCD-5	Bounded self-referential feedback improves next-frame broad-phase coherence.	Open-loop refit/rebuild and standard temporal coherence baselines.
H-CCD-6	SDF-specific backends improve quality-per-cost for complex implicit geometry.	Triangle mesh, convex proxy, and recent triangle-SDF/SDF-SDF methods.
H-CCD-7	GPU implementation preserves all CPU statuses and TOI bounds while improving throughput or latency.	Deterministic replay on exact and adversarial corpora.

14.2 Kill criteria

An NHDF-specific operator should be removed from the CCD path when it fails equal-budget ablations, increases false negatives, widens TOI bounds without compensating value, creates excessive divergence, or cannot be explained independently of the source metaphor. The architecture is stronger when unnecessary operators are deleted.

Part IV

Hardware, integration, and lifecycle

GPU roadmap

15.1 Semantics first

GPU acceleration begins only after a CPU backend passes exact differential tests. The GPU is not asked to execute “topology” or “SDF zero” by name; it executes concrete bounds, roots, interval predicates, scans, queues, distance queries, and reductions. Matrix or ray hardware is used only where exposed semantics match the subproblem.

15.2 Proposed pipeline

1. upload or generate structure-of-arrays body state;
2. compute swept bounds;
3. run a conservative broad phase;
4. compact candidates with prefix sums while returning required count and overflow;
5. bucket by backend and motion type;
6. execute batched exact primitive kernels;
7. execute robust mesh/inclusion work queues;
8. reduce earliest TOI with a deterministic tie policy;
9. emit certificates and bounded traces;
10. replay selected samples on CPU.

15.3 Floating-point policy

Fast-math modes can alter predicate signs and root intervals. The GPU profile must classify operations into performance-tolerant arithmetic and correctness-critical predicates. A faster kernel that silently changes HIT to NO_HIT is not an optimization. Exact/inclusion methods and outward rounding may be required for robust backends.

15.4 NHDF scheduling experiments

Log-polar separation/approach bins could group similar iteration counts; $\Delta^2\phi$ could identify changing motion; parity could request replay; golden splits could shape work queues. Each experiment must preserve the candidate set and certificate semantics. The source CUDA/Tensor claims are therefore replaced by measurable scheduling hypotheses rather than assertions that Tensor Cores implement parity or topology.

15.5 Acceptance metrics

- zero false negatives on declared exact corpora;
- CPU/GPU status equality;
- TOI bracket containment and width;
- overflow and unsupported-state equality;
- latency percentiles, not only average throughput;
- memory and queue high-water marks;
- energy and thermal behavior on the actual device;
- deterministic mode and documented non-deterministic high-throughput mode.

Application integration and response boundary

16.1 Step protocol

A conservative event-driven simulation step is:

1. predict but do not commit motion;
2. find candidates and query CCD;
3. select the earliest conservative TOI;
4. group near-simultaneous contacts;
5. advance to the allowed time;
6. solve response or constraints;
7. repeat for the remaining substep under a bounded event count;
8. commit state and telemetry.

16.2 Rollback and safe stop

The caller must have policies for initial overlap, inconclusive queries, unsupported shapes, and capacity exhaustion. In safety-relevant systems, the default should be reduced speed, reduced step, larger clearance, or stop – not an optimistic no-hit.

16.3 Public-safety and autonomous-agent boundary

CCD has legitimate uses in emergency vehicles, robots, drones, camera rigs, and mechanical safety. It does not establish intent, identity, guilt, threat, or proportionate force. A policing adapter must include lawful purpose, data minimization, human control over consequential decisions, independent privacy/bias/security review, conventional evidence cryptography, uncertainty display, and safe-stop behavior.

Cradle-to-grave gates

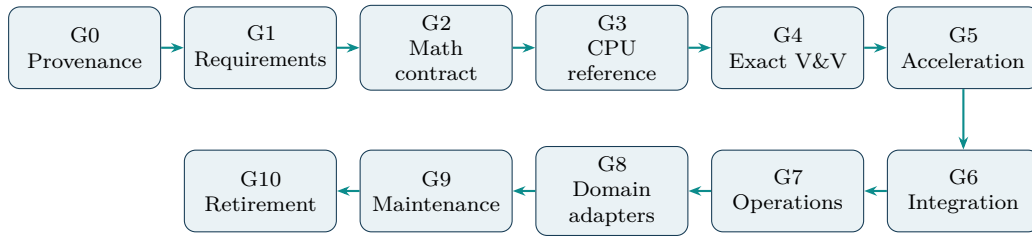


Figure 17.1: Lifecycle gates. Flow runs left-to-right on the first row and right-to-left on the second. An adapter may enter only after the core evidence it depends on exists; retirement remains a designed stage.

17.1 Gate G0 – provenance and claim boundary

Input: supplied documents and external research. **Output:** source register, traceability matrix, privacy policy, and disposition classes. **Pass:** every statement is sourced, reasoned, or labeled hypothesis.

17.2 Gate G1 – requirements and hazards

Declare shapes, motion class, clearance, units, tolerances, missed-contact cost, false-positive cost, determinism, latency, and caller failure policy. **Pass:** supported and unsupported profiles are unambiguous.

17.3 Gate G2 – mathematical contract

Define separation/inclusion objects and valid error/motion bounds. **Pass:** every no-hit statement has a proof obligation and heuristics have no hidden safety authority.

17.4 Gate G3 – deterministic CPU reference

Implement the smallest clear semantics. **Pass:** unit/property tests, reference vectors, schemas, failure preservation, and deterministic replay.

17.5 Gate G4 – robust differential validation

Use exact/adversarial data and independent implementations. **Pass:** zero false negatives in the declared profile and bounded TOI error. This release has not passed G4 for triangle meshes.

17.6 Gate G5 – accelerated backend

Port verified semantics to C++/GPU/other hardware. **Pass:** equivalence plus measured performance, memory, energy, and failure behavior.

17.7 Gate G6 – application integration

Add response, rollback, uncertainty, and hardware-in-loop testing. **Pass:** end-to-end tunneling and failure recovery succeed under realistic timing.

17.8 Gate G7 – deployment and operations

Add signed releases, configuration control, telemetry, alarms, incident replay, and safe disable. **Pass:** divergence, overflow, invalid bounds, and unsupported inputs are detected operationally.

17.9 Gate G8 – domain adapters

Only now map the core into a domain with its own laws. **Pass:** units, invariants, uncertainty, and baselines are preserved, and operator equivalence is measured.

17.10 Gate G9 – maintenance

Every predicate, tolerance, compiler mode, queue, and broad-phase change triggers regression evidence. **Pass:** change record, tests, migration, and rollback exist.

17.11 Gate G10 – retirement

Freeze unsafe or obsolete backends, migrate callers, archive reproducibility artifacts, revoke deployment, and delete personal/sensor data according to policy. **Pass:** no silent fallback to discrete endpoint checks.

Operations, incident response, and change control

18.1 Runtime telemetry contract

At minimum, record

$$\{g, \Delta t, N_b, N_c, N_{\text{backend}}, [t^-, t^+], d_{\text{min}}, k, \tau_d, \tau_t, N_{\text{overflow}}, N_{\text{unsupported}}, N_{\text{inconclusive}}, T_{\text{run}}, h\}. \quad (18.1)$$

This separates geometry, numerical, resource, and application failures.

18.2 Alarm conditions

- any replayed false negative;
- repeated inconclusive results at an unacceptable rate;
- capacity high-water or overflow;
- NaN/Inf or invalid motion/error bound;
- CPU/GPU divergence;
- unsupported shape/motion entering a safety path;
- trace or configuration mismatch.

18.3 Incident package

Preserve the exact scene, units, configuration, build and compiler hashes, hardware/runtime versions, certificate, bounded trace, random seed, and caller action. Screenshots or identity metadata are not substitutes for cryptographic and reproducible evidence.

18.4 Change categories

A documentation typo is minor. A change to a tolerance, broad-phase bound, root formula, field interpolation, floating-point mode, queue capacity, tie order, or motion model is correctness-affecting and requires revalidation. A new adapter is not “just configuration” if it changes state, units, or invariants.

Table 18.1: Risk register summary.

ID	Risk	Severity	Likelihood	Primary mitigation
R-001	False-negative broad phase	Critical	Medium	Use conservative swept bounds; exact dataset; never use learned/parity pruning without verification
R-002	Distance oracle overestimates true separation	Critical	Medium	Require true/conservatively biased distance and calibrated Lipschitz/error bounds
R-003	Closure-speed bound too low	Critical	Medium	Derive per motion/shape; test acceleration/rotation; fail invalid bounds
R-004	Floating-point degeneracy	High	High	Inclusion/exact predicates; scale normalization; adversarial corpus; multiple precision modes
R-005	Iteration or queue exhaustion	High	Medium	Explicit INCONCLUSIVE/CAPACITY_EXCEEDED; step reduction or robust backend
R-006	Contact response conflated with detection	High	Medium	Separate APIs and certificates; response solver owns impulses/friction
R-007	Parity/golden topology treated as proof	High	High	Keep outside safety authority; mandatory ablations
R-008	GPU fast-math changes signs	Critical	Medium	Predicate-specific compile modes; CPU replay; disable backend on divergence
R-009	Approximate/neural SDF lacks error envelope	Critical	High	Do not certify; add conservative error bound or mesh/exact fallback
R-010	Medical or public-safety overclaim	Critical	Medium	Simulation-only labels, governance, clinical/regulatory evidence gates
R-011	Personal identifiers used as keys	High	Medium	Omit from runtime; use standard cryptography and release hashes
R-012	Domain adapter erases constitutive laws	High	High	Typed adapter contract with units/invariants/baselines

Part V

Separated domain adapters

Uniform adapter contract

19.1 Definition

A domain adapter is

$$\mathcal{A}_d = (X_d, \Phi_d, g_d, E_d, O_d, D_d, I_d, M_d, F_d), \quad (19.1)$$

where X_d is domain state, Φ_d accepted evolution, $g_d = 0$ an event/contact/constraint surface, E_d, D_d encoder/decoder, O_d the implemented operator subset, I_d units and invariants, M_d metrics/baselines, and F_d failures/stops.

A meaningful equivalence claim requires

$$\sup_{x \in \mathcal{D}_d} \|D_d(\mathcal{N}_{\Delta t}(E_d(x))) - \Phi_d^{\Delta t}(x)\|_{W_d} \leq \epsilon_d \quad (19.2)$$

with calibrated uncertainty and declared domain $\mathcal{D}_d$. Reusing the symbols ρ , ϕ , parity, SDF, or tree does not prove this inequality.

19.2 Relationship classes

- **Literal CCD:** moving geometric bodies and contact.
- **Boundary-event reuse:** a continuous threshold crossing uses bracketing/inclusion logic but is not called geometric collision.
- **Scheduling analogy:** queues, coordinates, and feedback are reused without physical equivalence.
- **Research hypothesis:** no transfer equivalence has yet been demonstrated.

Table 19.1: Separated domain adapters and evidence gates.

Adapter	CCD relationship	Maturity	Next evidence gate	Hard boundary
Graphics and game engines	Literal CCD	D2 executable scaffold	Exact/inclusion CCD datasets; response integration; GPU equivalence	Parity/golden/Klein are optional scheduling meta-data
Robotics and public safety	Literal CCD	D1 architecture map	Certified motion bounds, sensor uncertainty, safe stop, governance	No automated identity, suspicion, or use-of-force authority

Adapter	CCD relationship	Maturity	Next evidence gate	Hard boundary
Measurement, sensors, actuators	Literal contact plus boundary-event reuse	D1 contract	Traceability, uncertainty budget, actuator dynamics, hardware-in-loop tests	SDF is a calibrated geometric model, not the material itself
SAR and wave sensing	Boundary-event reuse	D0-D1 hypothesis	Measured focus metrics against established autofocus baselines	A focus zero set is not a rigid-body collision
GPU and edge AI	Scheduling analogy plus literal CCD acceleration	D1 design	Exact replay, no-false-negative broad phase, quality/performance baseline	Learned or parity proposals may not prune safety candidates
Optical and photonic computing	Operator-equivalence research	D0-D1 hypothesis	Calibrated transfer error, drift, bandwidth, energy, repeatability	Propagation direction is causal order, not zero latency
Quantum and cross-Kerr	Research hypothesis	D0	Gate/process tomography, noise model, fidelity, classical baseline	Ideal cross-Kerr parity is not assumed in biology or hardware
Chemical reaction networks	Boundary-event reuse	D0-D1 hypothesis	Mass/charge balance, calibrated rate laws, reactor experiments	Equilibrium is not automatically an SDF and parity is not universal
Biology and medicine	Simulation-only boundary events	D0	Preclinical/clinical evidence and regulatory pathway for any intervention	No treatment, regeneration, dementia reversal, or crash-immunity claim
Fabrication and 3D photonics	Literal geometry checks plus design rules	D1 map	PDK/DRC compliance, optical simulation, metrology, fabricated test coupons	Polar coordinates do not bypass fabrication tolerances
Cosmology and holography	Analogy only	D0	Derive a real duality and compare predictions	SDF boundaries and projection do not establish AdS/CFT-like holography

Graphics, robotics, measurement, and public safety

20.1 Graphics and game engines

This is the direct application. The pairwise contact zero set, swept bounds, TOI, witnesses, and response handoff are literal. Version 0.4 supports only a primitive slice, so the next work is robust triangle feature CCD, rotation, deformables, self-collision, simultaneous contacts, and engine response. Recent exact and inclusion datasets provide the required validation foundation (B. Wang, Ferguson, Schneider, et al., 2021; Belgrod et al., 2023).

The source projections – pyramid side, circle, sphere – become calibration geometry, not guaranteed outputs. Analytic cones may serve as swept support or collision cones, but they must be conservative for the actual shape and motion.

20.2 Robotics

Robot and vehicle planning uses CCD to validate link and environment trajectories. Motion uncertainty, control latency, localization error, and braking distance must enlarge the swept geometry or enter the bound. Hardware-accelerated ray-tracing research is promising for batched robotic collision queries, but representation accuracy and conservative coverage remain explicit tradeoffs (Sui et al., 2024).

20.3 Measurement, sensors, and actuators

The whole-substrate document’s broad measurement chain is retained in disciplined form: measurand, transduction, indication, calibration, uncertainty, and interpretation. Metrology terminology and traceability are standardized independently of NHDF (Joint Committee for Guides in Metrology, 2012). NHDF may provide a common data/control record, while each sensor retains its transfer function, units, bandwidth, noise, drift, hysteresis, and environment.

CCD is literal for moving stages, shutters, rotors, valves, robotic arms, scanner heads, and mechanical envelopes. For temperature, pressure, or voltage limits, the interval machinery may detect a threshold crossing but should be named boundary-event detection. Actuator response follows electromechanical or material dynamics; parity may request a safety state but is not a restoring force in matter.

20.4 Public safety

A collision engine can support emergency-vehicle navigation, robot safety, mechanical camera rigs, and evidence-device integrity checks. The adapter must not extend geometric prediction into claims of hidden

intent or automatic enforcement. Conventional cryptography and chain-of-custody practice, not personal metadata or a geometric phase lock, must secure evidence.

SAR, GPU/AI, and wave sensing

21.1 SAR and ionospheric disturbance

The source maps echo phase and timing residuals into a log-polar LUT, platform motion into kinematics, and focused output into projection. Version 0.4 preserves this as a wave-sensing adapter:

$$X_{\text{SAR}} = (s_{\text{echo}}, x_{\text{nav}}, \hat{\phi}_{\text{error}}), \quad (21.1)$$

$$g_{\text{SAR}}(t) = \text{declared focus or phase-consistency residual}. \quad (21.2)$$

A conservative threshold-crossing solver may bracket when the residual leaves an admissible region; bounded trees may partition range, azimuth, scale, or hypotheses. This is not rigid-body collision and does not remove the need for propagation, calibration, autofocus, and image-quality models.

Metrics must include phase RMSE, point-target resolution, peak and integrated sidelobe ratios, image entropy/contrast, compute, memory, dropped frames, and controlled disturbance. “G3 immunity” is rejected; the correct behavior is measured degradation, safe mode, stronger integrity codes, and explicit failure.

21.2 GPU CCD

The GPU adapter is a direct acceleration map discussed in Chapter 13. It must preserve every candidate and status. Current CCD work demonstrates both scalable parallel algorithms and the importance of exact validation; the main lesson is that speed and correctness must be measured together (Belgrod et al., 2023).

21.3 Edge AI

The AI material becomes a separate codec/routing hypothesis. Log-polar coding, local constraints, mixed precision, sparse routing, and bounded feedback may be tested against established quantization, sparsity, and distillation baselines. A learned broad phase may prioritize candidates, but a safety profile cannot allow it to drop pairs unless a conservative verification layer proves the drop safe.

Optical, photonic, quantum, and Cross-Kerr adapters

22.1 Optical operator equivalence

Programmable optics can implement selected linear functions and self-configuring transfer networks (Miller, 2013). The NHDF question is therefore concrete: does a physical transfer T_{phys} reproduce a declared operator O within error,

$$\|T_{\text{phys}}(E_{\text{in}}) - O(E_{\text{in}})\| \leq \epsilon? \quad (22.1)$$

A first experiment should implement only log-polar encoding plus a local projection or constraint stage. It must report phase/amplitude error, drift, calibration, bandwidth, repeatability, and total energy including conversion.

For CCD, a photonic stage might accelerate a distance transform, convolution, or candidate ranking. It cannot certify no-hit until its worst-case error is conservatively bounded under fabrication variation, temperature, source/detector noise, aging, and calibration failure. A digital verifier should remain in the loop.

22.2 Photonic waveguides and direct writing

Curved and three-dimensional waveguides can be fabricated, but “native polar geometry” does not abolish discretization or process windows. A tool path still has finite resolution; optical performance still depends on refractive index, mode confinement, bend loss, roughness, coupling, material absorption, and packaging. The fabrication adapter in Chapter 22 defines the required sequence.

22.3 Quantum profile

A quantum implementation requires state preparation, Hilbert space, Hamiltonian/channel, gates, noise, measurement, readout, and resource scaling. A classical zero-set is not automatically a quantum code. A quantum parity measurement is not the same object as payload XOR parity, topological orientation parity, or CCD root parity.

22.4 Cross-Kerr boundary

The idealized interaction

$$H = \hbar\kappa\hat{n}_a\hat{n}_b \quad (22.2)$$

can produce a conditional phase under ideal assumptions. Traveling-wave single-photon gates face response and phase-noise limits; detailed analyses show that causality-induced noise can preclude high-fidelity π

phase shifts in important regimes (Dove et al., [2014](#)). Version 0.4 therefore treats Cross-Kerr parity as a laboratory hypothesis, not as a CCD primitive, a guaranteed chip component, or a biological mechanism.

Chemical, biological, and medical adapters

23.1 Chemical reaction networks

A chemical state is not a geometry word. It is governed by stoichiometry and kinetics, commonly

$$\dot{c} = Sv(c, t), \quad (23.1)$$

with diffusion, temperature, activities, and thermodynamics as required. A concentration threshold or phase boundary $g(c, t) = 0$ may use conservative event detection. Log scaling may be useful over wide concentration ranges. A branch graph may represent reactions or compartments.

Programmable BZ-reaction processors demonstrate that hybrid digital-chemical computation is physically real, but they use explicit reactor dynamics, cells, hysteresis, and digital control rather than a universal parity law (Sharma et al., 2024). Any NHDF chemical adapter must conserve mass/charge, preserve positivity, calibrate rate laws, and compare with standard solvers and experiments.

23.2 Biological modeling

CCD has direct roles in tissue-tool contact, joint surfaces, catheter navigation, surgical simulation, cells/devices, and anatomical mesh self-collision. Detection must be coupled to biomechanics, fluid dynamics, constitutive behavior, and uncertainty.

Vascular branching is not established as universal golden-ratio scaling. Murray's minimum-work work provides a historically important, assumption-dependent alternative model (Murray, 1926). Any proposed branching law must be dimensionally valid and tested against data.

23.3 Biophotonics

Ultraweak photon emission is measurable, and theoretical models have proposed myelinated axons as possible optical waveguides (Kumar et al., 2016). Disease-model studies have reported altered ultraweak photon activity and spectral shifts (Z. Wang et al., 2023; Sefati et al., 2024). These observations do not demonstrate neural NHDF computation, tubulin photonic-bandgap logic, mitochondrial Cross-Kerr parity, or instant rerouting around damaged tissue.

23.4 Medical stop line

No therapeutic claim

This package does not claim to prevent or reverse dementia, regenerate neural tissue, restore swallowing, bypass vascular lesions, stop sepsis, or make an organism immune to failure. A mathematical reparameterization is not an intervention. Any medical claim requires a defined device or treatment, biological mechanism, dose/exposure, toxicology, preclinical replication, clinical trials, ethics, and regulatory review. Current authoritative health information does not support a general dementia cure claim (National Institute on Aging, [2026](#)).

Fabrication and 3D photonics

24.1 What CCD contributes literally

CCD can validate moving tool heads, stages, wafer handling, robot paths, and additive-manufacturing trajectories. Static intersection and minimum-separation checks can validate waveguide crossings, ring resonators, electrodes, vias, packages, and curved mask geometry. These are concrete design-rule and motion-planning uses.

24.2 What coordinate choice does not remove

Polar or spherical parameterization may describe rings and spirals efficiently, but fabrication equipment consumes finite-resolution layouts or paths. Process design kits, minimum width/space, etch bias, line-edge roughness, overlay, stitching, material deposition, coupling, thermal drift, and metrology remain. ASML's current High-NA EUV platform is a lithography scanner with 0.55 numerical aperture and an advertised 8 nm resolution; it is not a custom chip designer or complete fab (ASML, [2026](#)).

24.3 Credible cradle-to-device sequence

1. define optical/electrical function and process platform;
2. select materials and a foundry/process design kit;
3. perform electromagnetic and thermal simulation;
4. generate curvilinear layout with explicit approximation tolerance;
5. run design-rule and connectivity checks;
6. run lithography/process-window simulation where needed;
7. fabricate test coupons;
8. measure geometry, loss, phase, bandwidth, and drift;
9. calibrate the model and repeat;
10. only then integrate a computing or sensing demonstration.

Direct laser writing can alter the manufacturing route, but it does not “bypass fabrication” in the sense of removing materials, equipment, process calibration, packaging, or metrology. It is a different fabrication stack.

Cosmology and the limit of the substrate claim

25.1 Holographic word versus holographic principle

Projection, wavefront processing, or distributed storage may be called holographic in engineering. The holographic principle in quantum gravity is a precise relation between theories, observables, degrees of freedom, entropy, and dynamics; AdS/CFT is the canonical example (Maldacena, [1998](#)). An SDF boundary and a projection operator do not by themselves establish such a duality.

25.2 Permitted use

CCD and event-surface machinery may appear inside numerical relativity, particle simulation, ray propagation, or astrophysical software. That computational reuse does not imply that spacetime fundamentally executes the NHDF chain. The cosmology adapter remains D0 until it states two theories and derives a testable correspondence or new prediction.

Integrated adapter roadmap

26.1 Order of work

The separate materials should not all advance in parallel at equal priority. A rational sequence is:

1. complete triangle/convex/SDF CCD validation;
2. build a GPU-equivalent backend;
3. integrate one graphics or robotics application;
4. test NHDF scheduling ablations;
5. implement a narrow optical operator-equivalence experiment;
6. pursue fabrication only after simulation and process rules are selected;
7. keep chemical, biological, medical, quantum, and cosmological work as independent collaborations with domain specialists and separate evidence plans.

26.2 Why this preserves the entire project

Separating adapters is not a retreat. It gives every domain a common systems contract while preserving the constitutive laws that make the domain real. The shared contribution can be an architecture for constrained state, continuous event detection, bounded routing, observation, and feedback. A universal ontological claim is unnecessary and currently unsupported.

Part VI

Release roadmap, risks, and appendices

Version roadmap to 1.0

Table 27.1: Proposed release sequence.

Version	Center	Exit evidence
0.4	Typed CCD scaffold	Current PDF/ZIP; primitive exact/CA reference; tests; lifecycle and adapters.
0.5	Robust linear triangle CCD	Exact TOI corpus, vertex-face and edge-edge backends, degeneracy catalog, zero false negatives in declared profile.
0.6	Convex and rigid motion	GJK/support mapping or cone casting, rotational bounds, curved trajectory handling, independent differential tests.
0.7	SDF production profile	Certified/biased fields, triangle-SDF and SDF-SDF comparison, rotation, adaptive sampling, error envelopes.
0.8	GPU backend	CPU/GPU equivalence, deterministic mode, overflow equality, measured latency/memory/power.
0.9	Application proof	One graphics/robotics integration outperforming or complementing a selected baseline without correctness regression.
1.0	Stable core	Frozen API, supported profiles, external review, continuous exact-data testing, operations and retirement support.
1.x-adapter	Domain-specific	Separate version, domain laws, uncertainty, ethics/regulation, and measured operator equivalence.

27.1 Definition of done for 1.0

- public supported-profile matrix;
- exact and adversarial continuous test service;
- independent reproduction on at least two platforms;
- complete failure-state and resource semantics;
- CPU and accelerated backends with equivalence reports;
- stable serialization and API;
- one validated application integration;
- security, privacy, operations, incident, change, and retirement documentation;
- removal or isolation of operators that fail ablations.

Conclusions

The earlier work can be made stronger by narrowing its first proof target. CCD provides a literal and demanding realization of the central idea: valid states lie on local constraint surfaces; motion is continuous and causal; swept bounds conservatively restrict work; bounded refinement isolates events; projection emits an observable certificate; and feedback updates the next generation.

The executable v0.4 release demonstrates this architecture for exact linear spheres, sphere–plane contact, swept AABs, and conditional point/sphere–SDF conservative advancement. It also demonstrates the discipline that a complete engine needs: explicit unsupported cases, initial overlap, inconclusive results, capacity failure, deterministic traces, schemas, tests, benchmarks, and a lifecycle beyond the algorithm.

The source-specific log-polar, parity, phase, golden-ratio, and topology operators remain in the research program, but behind a correctness firewall. They must earn a place through ablation and cannot override geometry. This is the key turn from a broad substrate narrative to a testable computational contribution.

The optical, quantum, chemical, biological, medical, SAR, fabrication, and cosmological material has not been discarded. It has been mapped into independent adapters that preserve domain laws and define what evidence would be required. That separation is the cradle-to-grave foundation for responsible expansion: one verified core, many explicitly bounded adapters, and no silent leap from shared notation to universal physics.

Normative minimum specification

An implementation may call itself an NHDF-CCD conforming engine only if:

1. it defines continuous step time and monotonic generation semantics;
2. it defines supported shape and motion profiles;
3. it defines conservative broad-phase bounds and visible capacity behavior;
4. every supported pair has a declared exact, conservative, or inclusion backend;
5. every no-hit result is justified by a valid bound or exact predicate;
6. it distinguishes hit, no-hit, initial overlap, inconclusive, unsupported, invalid input, and capacity exceeded;
7. it outputs a TOI point or bracket with tolerance and backend identity;
8. it separates detection from response;
9. it bounds iterations, queues, traces, and workspace;
10. it supports deterministic replay for validation;
11. it does not let log-polar, parity, golden-ratio, phase, or topology hints suppress candidates or certify outcomes without a separate proof;
12. it exposes telemetry sufficient to reproduce a failure;
13. any accelerated backend is compared with a verified semantic reference;
14. any domain adapter declares state, laws, units, invariants, uncertainty, metrics, and failure conditions.

Symbol table

Table B.1: Principal symbols.

Symbol	Meaning
$B_i = (S_i, M_i)$	Body with shape and motion.
$t \in [0, 1]$	Normalized time over one proposed simulation step.
$d_{ij}(t)$	Pairwise separation or backend-specific contact function.
t_{impact}	Earliest time of impact.
$[t^-, t^+]$	Conservative time-of-impact bracket.
L	Valid Lipschitz/closure-speed bound for separation over an interval.
η	Conservative advancement safety factor in $(0, 1]$.
τ_d, τ_t	Distance and time tolerances.
Σ	Complete typed runtime state threaded through operators.
E_{scene}	Input validation and encoding.
C_{pair}	Conservative broad-phase candidate generation.
B_{bound}	Narrow-phase oracle and bound construction.
A_{CA}	Conservative advancement.
R_{refine}	Exact/inclusion/interval earliest-contact refinement.
Π_{cert}	Certificate emission.
U	Telemetry, cache, and next-generation update.
ρ, θ	Log-separation and approach-direction scheduling coordinates.
$\Delta^2 \phi$	Wrapped second finite difference of phase/direction metadata.

Package manifest overview

Table C.1: Top-level ZIP layout.

Path	Purpose
00_provenance/	Source register, evidence boundary, privacy/authorship policy, and mounted copies of sources S1–S3; source S4 remains a registered conversation attachment.
01_problem_definition/	Requirements and release acceptance criteria.
02_formal_specification/	Typed operator contract, certificate semantics, JSON schemas, and state machine.
03_reference_implementation/	Python package, examples, C++17 slice.
04_verification/	Tests, adversarial catalog, plan, and bundled verification report.
05_benchmarks/	Deterministic generators, reference vectors, JSON/CSV results.
06_gpu_port_plan/	GPU architecture and uncompiled CUDA exact-sphere skeleton.
07_integration/	Engine integration, response boundary, public-safety governance.
08_operations/	Runbook and change control.
09_retirement/	Decommissioning and migration plan.
10_domain_adapters/	Uniform contract, matrix, and eleven separated adapter documents.
11_report/	LaTeX source, figures, references, generated tables, and final PDF.
scripts/	Full verification, benchmark-asset generation, report build, and package hash checks.
SHA256SUMS.txt	Hashes for deliverable files after package verification.

Reproduction commands

Listing D.1: Full local verification.

```
1 python -m pip install -e .
2 pytest
3 python 05_benchmarks/generate_reference_vectors.py
4 python 05_benchmarks/run_benchmarks.py
5 python 11_report/generate_report_assets.py
6 cmake -S 03_reference_implementation/cpp -B build/cpp \
7     -DCMAKE_BUILD_TYPE=Release
8 cmake --build build/cpp
9 ctest --test-dir build/cpp --output-on-failure
10 bash scripts/build_report.sh
11 python scripts/package_check.py
```

The benchmark seed, test files, schemas, and generated reference vectors are included. The external exact TOI datasets and third-party libraries are not redistributed; their project pages and papers are cited for the next validation gate.

References

- ASML (2026). *TWINSCAN EXE:5000 High-NA EUV Lithography System*. Official product documentation; 0.55 numerical aperture and 8 nm resolution.
- Belgrod, David, Bolun Wang, Zachary Ferguson, Xin Zhao, Marco Attene, Daniele Panozzo, and Teseo Schneider (2023). *Time of Impact Dataset for Continuous Collision Detection and a Scalable Conservative Algorithm*. arXiv: [2112.06300](https://arxiv.org/abs/2112.06300) [cs.GR].
- Chen, Xuwen, Cheng Yu, Xingyu Ni, and Baoquan Chen (2024). “A Time-Dependent Inclusion-Based Method for Continuous Collision Detection between Parametric Surfaces”. In: *ACM Transactions on Graphics* 43.6. DOI: [10.1145/3687960](https://doi.org/10.1145/3687960).
- Dove, Justin, Christopher Chudzicki, and Jeffrey H. Shapiro (2014). “Phase-noise Limitations on Single-photon Cross-phase Modulation with Differing Group Velocities”. In: *Physical Review A* 90, p. 062314. DOI: [10.1103/PhysRevA.90.062314](https://doi.org/10.1103/PhysRevA.90.062314).
- Ferguson, Zachary et al. (2021). “Intersection-free Rigid Body Dynamics”. In: *ACM Transactions on Graphics*.
- IPC-Sim Project (2026). *IPC Toolkit: Reusable Broad- and Narrow-Phase Continuous Collision Detection*. Project documentation and source repository. Accessed 1 September 2026.
- Joint Committee for Guides in Metrology (2012). *International Vocabulary of Metrology – Basic and General Concepts and Associated Terms, 3rd edition*. JCGM 200:2012. BIPM. DOI: [10.59161/JCGM200-2012](https://doi.org/10.59161/JCGM200-2012).
- Klootwijk, Tom (2026a). “Commute design transcript: log-polar kinematics, register operators, and CCD”. Supplied 40-page source document, 31 August 2026.
- (2026b). “Non-Euclidean Holographic Data Fields: Formal Operator Specification, Version 0.1”. Supplied 26-page formal specification, 31 August 2026.
- (2026c). “Original NHDF operator-chain prompt transcript”. Supplied 30-page source document, 31 August 2026.
- (2026d). “Spherical-Cartesian whole-substrate extension”. Supplied 70-page source document.
- Kumar, S., K. Boone, J. Tuszynski, P. Barclay, and C. Simon (2016). “Possible Existence of Optical Communication Channels in the Brain”. In: *Scientific Reports* 6, p. 36508. DOI: [10.1038/srep36508](https://doi.org/10.1038/srep36508).
- Li, Minchen, Danny M. Kaufman, and Chenfanfu Jiang (2021). “Codimensional Incremental Potential Contact”. In: *ACM Transactions on Graphics* 40.4.
- Maldacena, Juan M. (1998). “The Large N Limit of Superconformal Field Theories and Supergravity”. In: *Advances in Theoretical and Mathematical Physics* 2, pp. 231–252. eprint: [hep-th/9711200](https://arxiv.org/abs/hep-th/9711200).
- Miller, David A. B. (2013). “Self-configuring Universal Linear Optical Component”. In: *Photonics Research* 1.1, pp. 1–15.
- Mirtich, Brian Vincent (1996). “Impulse-based Dynamic Simulation of Rigid Body Systems”. PhD thesis. University of California, Berkeley.
- Murray, Cecil D. (1926). “The Physiological Principle of Minimum Work Applied to the Angle of Branching of Arteries”. In: *Journal of General Physiology* 9.6, pp. 835–841. DOI: [10.1085/jgp.9.6.835](https://doi.org/10.1085/jgp.9.6.835).
- National Institute on Aging (2026). *How Is Alzheimer’s Disease Treated?* Official health information; accessed 1 September 2026. U.S. National Institutes of Health.

- Nielsen, Rasmus Gabelgaard and Kenny Erleben (2026). “Alternating Spatial-Temporal Optimization for Continuous Collision Detection of Signed Distance Fields”. In: *Computer Graphics Forum*. DOI: [10.1111/cgf.70570](https://doi.org/10.1111/cgf.70570).
- Pelletier-Guénette, Joël, Alexandre Mercier-Aubin, and Sheldon Andrews (2025). “Real-Time Triangle-SDF Continuous Collision Detection”. In: *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 8.4. DOI: [10.1145/3747862](https://doi.org/10.1145/3747862).
- Sefati, N. et al. (2024). “Monitoring Alzheimer’s Disease via Ultraweak Photon Emission”. In: *iScience*.
- Sharma, Abhishek, Marcus Tze-Kiat Ng, Juan Manuel Parrilla Gutierrez, Yibin Jiang, and Leroy Cronin (2024). “A Programmable Hybrid Digital Chemical Information Processor Based on the Belousov-Zhabotinsky Reaction”. In: *Nature Communications* 15, p. 1984. DOI: [10.1038/s41467-024-45896-7](https://doi.org/10.1038/s41467-024-45896-7).
- Sui, Sizhe, Luis Sentis, and Andrew Bylard (2024). *Hardware-Accelerated Ray Tracing for Discrete and Continuous Collision Detection on GPUs*. arXiv: [2409.09918](https://arxiv.org/abs/2409.09918).
- Wang, Bolun, Zachary Ferguson, Xin Jiang, Marco Attene, Daniele Panozzo, and Teseo Schneider (2022). “Fast and Exact Root Parity for Continuous Collision Detection”. In: *Computer Graphics Forum* 41.2, pp. 355–363. DOI: [10.1111/cgf.14479](https://doi.org/10.1111/cgf.14479).
- Wang, Bolun, Zachary Ferguson, Teseo Schneider, Xin Jiang, Marco Attene, and Daniele Panozzo (2021). “A Large-scale Benchmark and an Inclusion-based Algorithm for Continuous Collision Detection”. In: *ACM Transactions on Graphics* 40.5. DOI: [10.1145/3460775](https://doi.org/10.1145/3460775).
- Wang, Z. et al. (2023). “Reduced Biophotonic Activities and Spectral Blueshift in Alzheimer’s Disease and Vascular Dementia Models with Cognitive Impairment”. In: *Frontiers in Aging Neuroscience*.
- Yuan, Xiaodi, Fanbo Xiang, Yin Yang, and Hao Su (2025). “C5D: Sequential Continuous Convex Collision Detection Using Cone Casting”. In: *ACM Transactions on Graphics* 44.4. DOI: [10.1145/3731151](https://doi.org/10.1145/3731151).
- Zhang, Xinyu, Minkyung Lee, and Young J. Kim (2006). “Interactive Continuous Collision Detection for Non-convex Polyhedra”. In: *The Visual Computer* 22, pp. 749–760. DOI: [10.1007/s00371-006-0060-0](https://doi.org/10.1007/s00371-006-0060-0).